Intelligent Reasoning Systems Final Project

Project Title: AniLogic -- Your animation, comics, games (ACG) friend.

Group Number (Registered in Canvas): Group 18

Group Final Presentation:

- Promotion video: https://www.youtube.com/watch?v=ONuUsmzcIac
- System demonstration video: https://www.youtube.com/watch?v=SCX9MGig88I

Group Members' Names, NUS Student ID or Masked ID:

Name (ID)	System contribution	Report and other contributions
Gao Xinghe (A0305470M)	1. Development and comparative analysis of Algorithm 3 2. Data processing for Algorithm 3 3. Development of the main system functions 4. Auxiliary data collection and processing	1. Wrote Chapters 1, 5, 8, 13 and 14 and Section 7.3 of the report 2. Co-wrote Chapters 8, 10, 11 and 12 3. Wrote the scripts for the business and technical presentations and recorded them
Lin Kai (A0352380J)	1. Development and comparative analysis of Algorithm 4 2. Data processing for Algorithm 4 3. Collection and processing of the main data 4. Development of auxiliary system functions	1. Wrote Chapter 6 and Section 7.4 of the report 2. Co-wrote Chapters 10, 11 and 12 3. Responsible for standardising and typesetting the report 4. Compiled and accepted the data package documentation, the system handover manual and the review records
Sun Yancheng (A0335090L)	1. Development and comparative analysis of Algorithm 1 2. Data processing for Algorithm 1 3. Collection and processing of the main data 4. Development of auxiliary system functions	1. Wrote Chapters 2, 3, 4 and 6 and Section 7.1 of the report 2. Co-wrote Chapters 10, 11 and 12
Yu Yuchen (A0356318Y)	1. Development and comparative analysis of Algorithm 2 2. Data processing for Algorithm 2 3. Development of the main system functions 4. Auxiliary data collection and processing	1. Wrote Section 7.2 and Chapter 9 of the report 2. Co-wrote Chapters 8, 10, 11 and 12 3. Narrated and recorded the system demonstration video 4. Edited and laid out all videos in post-production

1 Introduction

With the continuous development of digital content platforms, both the volume of anime content and the size of its audience have expanded significantly. Identifying content that aligns with users' interests and viewing preferences from a large collection of candidate titles has therefore become an important application of personalized recommendation and intelligent information retrieval. According to data released by Netflix in July 2025, more than half of its global members had watched anime, with anime content accumulating over one billion views on the platform. These figures reflect the growing demand for anime within the digital entertainment market. However, existing content platforms still rely primarily on structured attributes, such as titles, genres, and languages, for content retrieval. Such mechanisms are often insufficient to represent users' multidimensional preferences regarding narrative style, thematic characteristics, content length, and complexity. In particular, when user requirements involve both subjective semantic descriptions and objective attribute constraints, traditional keyword-based retrieval and individual recommendation methods may struggle to produce sufficiently relevant results. Therefore, integrating historical user behavior, content knowledge, and natural language requirements to achieve accurate, explainable, and diverse anime recommendations constitutes the primary motivation of this project.

To address these challenges, AniLogic divides the anime content discovery task into three interrelated research problems. The first is **personalized preference matching**, which aims to estimate the relevance of candidate anime titles to individual users based on historical ratings, favorite lists, and content attributes. The second is **recommendation list optimization**, which seeks to improve diversity, novelty, and long-tail content coverage while maintaining recommendation accuracy, thereby reducing excessive concentration on popular or highly similar titles. The third is **natural language perception and semantic retrieval**, which focuses on understanding user preferences, underlying intentions, and explicit attribute constraints expressed through natural language, and transforming unstructured linguistic information into semantic representations suitable for knowledge retrieval and recommendation generation. These three problems involve user preference modeling, multi-objective ranking optimization, and cognitive language perception with knowledge-enhanced reasoning, respectively. Each requires different algorithmic approaches and evaluation criteria. Consequently, this project adopts a hybrid intelligent recommendation architecture that integrates multiple reasoning techniques to address content discovery requirements across different application scenarios.

In terms of system design, AniLogic applies four categories of reasoning techniques to four functional scenarios. The anime detail page employs **Knowledge-based Recommendation**, which constructs a multidimensional similarity model using attributes such as genres, themes, target demographics, production studios, and content length to identify related anime titles. The favorites page adopts **Rule- and Knowledge-based Recommendation**, which first filters candidate titles according to user-defined constraints, including keywords, minimum ratings, and maximum episode counts, and subsequently applies a learning-based ranking model to optimize recommendation order. The homepage implements a two-stage recommendation architecture combining **Item-based Collaborative Filtering (ICF) and a Genetic Algorithm (GA)**. Collaborative filtering is responsible for candidate retrieval, while the genetic algorithm performs multi-objective optimization of the recommendation list to balance relevance, diversity, novelty, and recency. The chatbot page integrates **Cognitive Language Perception and Retrieval-Augmented Generation (RAG)**. A language model is employed to interpret natural language

queries and identify user preferences, query intentions, and content constraints. Relevant anime information is then retrieved from an external knowledge base to provide factual evidence for response generation. By combining natural language understanding, semantic information retrieval, and knowledge-enhanced generation, this module supports complex queries involving both subjective preferences and objective requirements, producing contextually relevant, factually accurate, and traceable recommendations and natural language responses. The complete system is implemented using an HTML frontend, a FastAPI backend, and a SQLite database, with all functional modules integrated into a single-process service architecture.

Unlike system development approaches that focus primarily on implementing recommendation functionalities, this project places systematic algorithm comparison and experimental validation at the center of its research methodology. For each of the four recommendation scenarios, multiple candidate algorithms with different modeling mechanisms are designed and evaluated. Comparative experiments are conducted across key components, including similarity computation, feature representation, ranking strategies, candidate retrieval, recommendation list re-ranking, language model selection, and knowledge-enhanced retrieval. Rather than evaluating only the final recommendation performance, the experiments also employ baseline comparisons, component substitution, and ablation analysis to investigate the contributions of individual algorithmic components. For knowledge-based recommendation, the experiments examine how different attribute combinations, feature weights, and similarity computation strategies affect recommendation quality. For rule- and knowledge-based recommendation, alternative candidate filtering and learning-based ranking strategies are evaluated. For homepage recommendation, the candidate retrieval capability of collaborative filtering and the additional benefits of genetic algorithm-based list optimization are analyzed separately. For cognitive language perception, the experiments compare the performance of different language models in processing natural language queries and further investigate changes in response quality, semantic relevance, and factual accuracy before and after the introduction of RAG. To ensure experimental comparability, evaluation metrics and experimental protocols are defined before model selection, and comparisons are conducted under consistent data partitions, candidate sets, and evaluation conditions wherever possible. The final algorithm configurations are selected based on experimental evidence rather than subjective assessments of individual recommendation examples. Through this methodology, the project evaluates the applicability of different reasoning techniques and investigates the trade-offs among recommendation accuracy, content coverage, semantic understanding, and generation reliability.

Data collection, preprocessing, and multi-source information integration also constitute a substantial component of the project's technical work. The project primarily utilizes a 2023 snapshot of the MyAnimeList dataset (dbdmobile, 2023), which contains anime metadata and approximately 24 million user rating records. As the original dataset presents limitations in attribute completeness, data consistency, and compatibility with different recommendation tasks, extensive data processing is conducted, including data cleaning, missing attribute completion, entity association, and structured information integration. Particular attention is given to enriching knowledge attributes such as anime themes, target demographics, character information, and relationships between titles. Based on these processed data, a unified data representation and storage structure is established, while anime attributes, user interactions, and knowledge retrieval resources are organized according to the input requirements of different algorithms. Specifically, the knowledge-based recommendation module requires structured anime attributes and feature representations for similarity computation. The collaborative filtering module requires the organization of large-scale user rating records to construct user-item interaction relationships. The

rule- and knowledge-based recommendation module requires consistent representations of anime attributes and user-defined constraints. Meanwhile, the cognitive language perception and RAG module requires a searchable knowledge corpus to support natural language queries and factual information retrieval. The resulting data resources contain 25,019 anime entries, a serviceable catalog of 14,202 titles, and a retrieval corpus consisting of working documents and scenario records. This data processing pipeline not only provides a consistent foundation for algorithm evaluation but also enables heterogeneous techniques, including collaborative filtering, knowledge-based matching, constraint-aware ranking, and language perception with knowledge-enhanced generation, to be experimentally evaluated and integrated within a unified system.

Preliminary experimental results demonstrate the practical value of the selected reasoning techniques in their respective application scenarios. For the anime detail page, the fixed five-dimensional weighted similarity method achieves the best performance across all evaluated metrics in an experiment involving 1,000 target anime titles. For the favorites page, the learning-based ranking model improves Hit Rate at 8 (HR@8) from 0.021 to 0.132, indicating that data-driven ranking strategies can effectively improve the prioritization of candidate titles. For homepage recommendation, the genetic algorithm increases long-tail content exposure by approximately 51%, at the cost of a reduction of approximately one percentage point in Hit Rate at 10 (HR@10). This result illustrates a measurable trade-off between recommendation accuracy and content coverage. For cognitive language perception and natural language interaction, introducing RAG increases the average evaluation score across six language models from 3.50 to 4.05, while improving episode-count accuracy from 0.39 to 0.86. These findings suggest that external knowledge retrieval can enhance the factual reliability of language model outputs. Collectively, these results provide experimental evidence for the selection of individual algorithmic components and demonstrate that the value of integrating multiple reasoning techniques extends beyond functional coverage. Different recommendation objectives can be addressed more effectively through task-specific modeling and experimental optimization.

Overall, the primary research contributions of AniLogic lie in **its multi-technique architecture designed for different recommendation tasks, systematic algorithm comparison based on unified data resources, and data-driven model selection supported by experimental validation**. By integrating large-scale user interaction records with multidimensional anime knowledge, the project establishes a common data foundation for evaluating different recommendation approaches. Through comparative experiments involving candidate algorithms, key components, and optimization strategies, the project investigates the applicability and limitations of different techniques in terms of recommendation accuracy, diversity, long-tail coverage, and natural language interaction. Compared with systems relying on a single recommendation model, the proposed architecture enables the selection of appropriate reasoning mechanisms for different user scenarios while providing a reusable experimental foundation for future algorithmic improvements and system extensions.

The remainder of this report is organized as follows. Chapters 2 to 4 introduce the project background, related research, target users, and project scope. Chapter 5 presents the four categories of reasoning techniques and their corresponding application scenarios. Chapter 6 describes the data collection, cleaning, attribute completion, and dataset construction processes. Chapter 7 focuses on the candidate algorithms, experimental designs, comparative results, and model selection criteria for each recommendation module. Chapters 8 to 11 discuss the system architecture, implementation, system-level evaluation, and interim project outcomes. Chapter 12 analyzes the current technical limitations and unresolved challenges. Chapter 13 outlines future

development and improvement plans, while Chapter 14 summarizes the main research findings and practical contributions of the project.

2 Project Background

2.1 Background and motivation

AniLogic is our anime discovery and recommendation project. We focus on the decision a viewer makes before watching: finding a title that fits their interests, deciding whether it is worth the time, and choosing among several reasonable options. Streaming services provide access to anime, while catalogue sites provide ratings, synopses and watchlists. Even with these resources, a viewer may need to move between several services to answer a fairly simple question: what should I watch next?

The size of the audience makes this a useful problem to work on. In July 2025, Netflix reported that more than half of its global members watched anime. It also reported over one billion anime views in 2024 and a threefold increase in anime viewership over five years (Netflix, 2025). These figures describe activity on Netflix, rather than the entire anime market, but they show that anime discovery matters to a substantial audience. They also suggest that a discovery tool should accommodate casual viewers who do not already know the terminology, studios or titles used by experienced fans.

A viewer's preferences are often more specific than a genre label. Someone who enjoyed Attack on Titan might want another action series with a serious story, but with fewer episodes. Another viewer might like several fantasy shows yet want their next recommendation to feel different. A high average rating alone cannot resolve either request. Our starting point is therefore the gap between the information available in an anime catalogue and the way people actually describe what they want to watch.

2.2 The discovery problem

We identify two related recommendation problems. The first concerns individual fit: whether a title matches a user's preferences and stated requirements. The second concerns the list as a whole. Ten relevant recommendations may still offer little choice if most share the same themes or belong to the same franchise. For AniLogic, relevance matters, but so does whether the results give the user a useful range of options. Diversity and novelty need to be considered alongside predicted interest, rather than added as an afterthought.

Retrieval presents a different difficulty. Users do not always remember a title, and a description such as "a short mystery anime with a dark atmosphere" mixes subjective preferences with a measurable length constraint. Structured filters can handle some of this request, but they require the user to translate it into catalogue fields. Natural-language retrieval can reduce that effort, provided that the response stays grounded in information the system can retrieve and checks explicit constraints against the underlying metadata.

These problems led us to a hybrid design. Attribute matching supports similar-title recommendations, rules enforce filters, and item-based collaborative filtering uses rating patterns to generate personalised candidates. A genetic algorithm then considers combinations of candidates when selecting the final list. The conversational component uses retrieval-augmented generation (RAG) to connect a user's description with the anime knowledge base.

3 Market Research

3.1 Market evidence and its implications

We approach AniLogic as an MVP product, so our market research focuses on existing services, evidence of demand and a realistic position for a small prototype. Netflix's viewership figures provide one indication of sustained interest. Sony also reported that Crunchyroll had more than 17 million paid members as of 31 March 2025 (Sony Group Corporation, 2025). Together, these figures show that anime attracts both audiences on general streaming services and subscribers to a dedicated platform.

For our project, the opportunity lies in helping viewers make better choices within an already active market. AniLogic does not need to acquire streaming rights to explore this opportunity. Public catalogue metadata and rating records provide a basis for recommendations, and users can watch the selected titles through their existing services. A useful product would need to offer enough additional value to justify another step in a viewer's routine.

We therefore treat the market evidence as support for the relevance of the problem, rather than proof of product adoption. The more practical question is whether combining preference matching, list diversity and conversational search can reduce the work involved in choosing an anime.

3.2 Existing services and competing approaches

Streaming platforms already provide substantial discovery support. Netflix's standard search accepts genres, names, languages and other terms as well as titles (Netflix, n.d.-b). Its help centre also describes an opt-in beta on iPhones and iPads that lets a limited group of members search in their own words and refine results according to mood and taste (Netflix, n.d.-a). Natural-language discovery is therefore an existing competitive direction. AniLogic's position needs to be more specific than simply allowing users to type a sentence.

Catalogue and community sites are equally relevant competitors. MyAnimeList offers recommendations and personal lists, while AniList explicitly supports discovery through reviews and recommendations (AniList, n.d.). These services give users substantial information and community knowledge. The remaining question for AniLogic is whether this information can be organised around a user's immediate requirements and whether a recommendation list can be assessed as a combined set of choices.

A MyAnimeList forum thread from June 2025 illustrates some concerns about that experience. One participant complained about being recommended titles already marked as completed. Another described suggestions that included sequels to shows they had not watched and questioned how personalised the selection was (MyAnimeList, 2025). These comments are individual accounts, rather than a representative survey or a verified description of MyAnimeList's algorithm. They nevertheless identify useful requirements for our own design: consider known viewing history, avoid unnecessary repetition, and check franchise dependencies when relationship data is available.

Specialist tools also address parts of the problem. trace.moe identifies anime from screenshots and can return the episode and exact moment of a matching scene (Soruly, n.d.). Screenshot identification is therefore not an unserved market. It also requires suitable visual reference data. For AniLogic, the more defensible opportunity is to connect several discovery tasks within one anime-focused workflow, with recommendations informed by user preferences and an explicit objective for the overall list.

3.3 Target users and product opportunity

Our initial target users are anime viewers who want help choosing their next title and are willing to provide a few ratings or favourite shows. A student looking for a shorter series is one plausible use case. Another is an experienced viewer who wants suggestions outside the genres they usually watch. Newcomers may benefit from conversational search because they can describe a mood or story preference without knowing the catalogue's labels.

The product's value should be visible in ordinary decisions. A user browsing an anime detail page should receive similar titles with an understandable basis for the match. A user applying a genre or episode-count filter should receive results that respect it. On the homepage, the recommendation list should retain relevant options while avoiding excessive repetition. In chat, explanations should refer to retrieved anime information and make uncertainty clear when the available description does not support a confident match.

There is also a practical barrier to adoption. Many viewers already maintain a watchlist elsewhere, so asking them to recreate a long rating history would add friction. Favourites and simple filters provide a more accessible starting point, while collaborative filtering becomes useful when sufficient rating information is available. We position AniLogic as a companion to existing streaming and catalogue services. Whether viewers would continue using it, or pay for it, requires separate validation. Those questions cannot be answered from market size or the quality of a classroom demonstration alone.

4 Project Scope

4.1 Functional scope

The core scope of AniLogic is a web-based anime discovery prototype. It covers registration and login, catalogue browsing, title search, detail pages, user ratings and favourites. These functions provide the information and preference inputs needed by the recommendation modules. Although the project uses ACG branding, this version covers anime; manga and game recommendations are outside its scope. The architecture follows the proposal's HTML frontend, FastAPI backend and SQLite database, which are sufficient for a prototype at this scale.

The reasoning component covers the four approaches defined in the proposal. Knowledge-based matching compares anime attributes to recommend similar titles. The favourites module combines a preference profile with rule-based filters. For homepage recommendations, item-based collaborative filtering produces candidates from rating patterns, and a genetic algorithm selects a list using objectives for relevance, personalisation, diversity and novelty. The RAG component retrieves anime text to support natural-language search and basic recommendation explanations. RAG provides a way to condition generated responses on external information, although retrieval alone does not guarantee factual accuracy (Lewis et al., 2020).

We distinguish this core scope from the extensions listed in the proposal. Optimising a recommendation list does not, by itself, produce a viewing plan within a time budget. That extension requires dependable duration information and an explicit feasibility check. Knowledge-graph reasoning, detailed scene search and screenshot identification also require additional data and implementation. They remain extension areas, together with richer feedback and adaptive recommendations. Anime playback, content licensing and commercial operation are outside the project's scope.

4.2 Data and academic scope

The data scope centres on the 2023 MyAnimeList dataset cited in the proposal, which contains anime metadata, user ratings and user profile records (dbdmobile, 2023). Anime and user identifiers connect the relevant tables. Synopses support text retrieval, while attributes such as genres and episode counts support matching and filtering. The snapshot provides a stable basis for experiments, but it cannot represent every new release or subsequent change in ratings. Jikan v4 is an optional source of supplementary catalogue information. It is an unofficial MyAnimeList API and does not support authenticated updates to users' lists (Jikan, n.d.).

Academically, the project applies established reasoning and recommendation methods to a shared product problem. Our focus is their integration and the trade-off between individual relevance and the quality of the complete recommendation list.

An appropriate evaluation would compare the hybrid approach with simpler alternatives, such as popularity ranking, attribute matching and collaborative filtering without GA optimisation. Ranking measures should be considered alongside diversity and novelty, since a gain in one may come at the expense of another. Constraint violations, duplicate suggestions and response time also matter to the user experience. The conversational component needs separate checks for retrieval relevance, factual support and compliance with explicit requests.

4.3 Limitations and market applicability

The main data limitation is uneven coverage. Missing scores, incomplete attributes and sparse user ratings restrict which titles can be compared reliably. Excluding unrated entries may improve consistency while reducing exposure to newer or less widely rated works. MyAnimeList users are also a self-selected group, and their ratings do not necessarily represent casual viewers or audiences in every region. Personalisation is especially uncertain for users with little history.

There are limits to the optimisation and retrieval methods as well. A GA searches for good combinations within the candidate pool and the chosen fitness function; it does not guarantee a global optimum. Its output depends on the objective weights and cannot recover a suitable title that candidate generation has already excluded. Catalogue synopses may support broad story queries but omit the exact scene a user remembers. The base dataset contains cover images rather than a frame-level reference collection, so it is insufficient for reliable screenshot matching.

Market applicability also depends on information beyond recommendation scores. A relevant title may be unavailable in the user's region or on their subscriptions. Community ratings indicate reported preferences rather than objective quality. API availability, access conditions and the rights associated with metadata and images would need review before public deployment. User ratings and account information would also require appropriate protection, while LLM cost and latency could limit routine use.

These constraints keep our project focused on an MVP whose recommendation choices can be inspected and evaluated. The immediate aim is to demonstrate a coherent anime discovery workflow. Broader claims about adoption, long-term satisfaction or commercial viability require testing with real users and a more current, operational data pipeline.

5 Reasoning Technology

In this experiment, we design the following four algorithms to implement the core functionalities of our system, covering key reasoning techniques across four areas:

1. **Decision automation:** Business rule- and knowledge-based reasoning
2. **Business resource optimization:** Evolutionary computing techniques (e.g., Genetic Algorithms)
3. **Recommendation:** Item-based Collaborative Filtering (ICF), Neural ICF (NeurICF), etc.
4. **Cognitive-enabled interaction:** Intelligent chatbot systems

This section introduces the fundamental functions of each algorithm. Detailed technical implementations, along with comprehensive comparative analyses and ablation studies, are presented in **Section 7: Algorithm Comparison and Model/Task Level Evaluation**.

Algorithm 1: Similar Anime Recommendation Algorithm for Anime Detail Pages
Technology Used: Knowledge-based reasoning

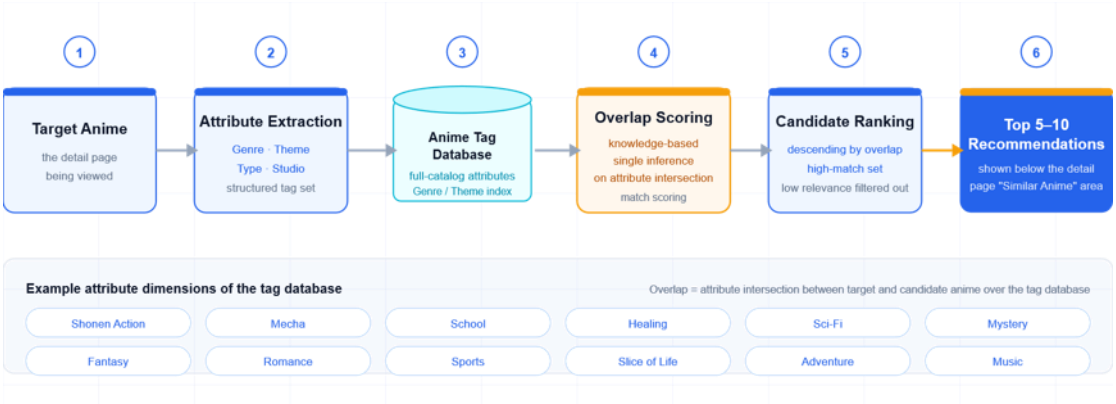


Figure 1. Algorithm 1: Similar Anime Recommendation Algorithm for Anime Detail Pages

Detailed Description: At the bottom of each anime detail page, the system recommends 5–10 similar results based on attribute matching with the target anime. The system establishes an anime tag database containing attributes such as Genre and Theme, and uses knowledge-based single reasoning to calculate the attribute overlap between the target anime and all anime in the database, thereby generating candidate works with high matching scores.

Algorithm 2: Hybrid Recommendation Algorithm Based on User Favorites and Keyword Filtering for the User Favorites Page

Technology Used: Knowledge-based + Rule-based fusion design

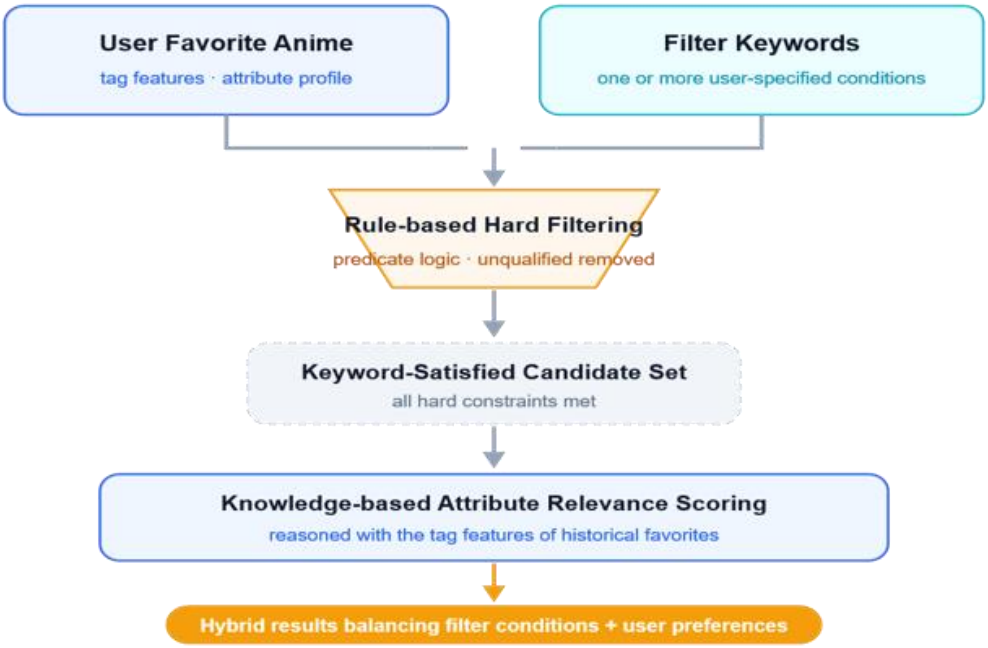


Figure 2. Algorithm 2: Hybrid Recommendation Algorithm Based on User Favorites and Keyword Filtering

Detailed Description: Personalized recommendations are generated based on the attributes of the anime in the user's favorites and the selected keywords. After satisfying the keyword requirements, the system performs similarity-based recommendations according to the attributes of the user's favorite anime. The system adopts a rule-and-knowledge fusion architecture: first, Rule-based predicate logic is used to apply hard filtering to one or more keywords selected by the user; subsequently, Knowledge-based reasoning calculates the attribute associations between the filtered set and the tag features of the user's historical favorites, generating hybrid recommendation results that consider both filtering conditions and preference relationships.

Algorithm 3: Main Page Recommendation Algorithm Based on All User Ratings and Genetic Algorithm Optimization

Technology Used: Item-based CF + GA

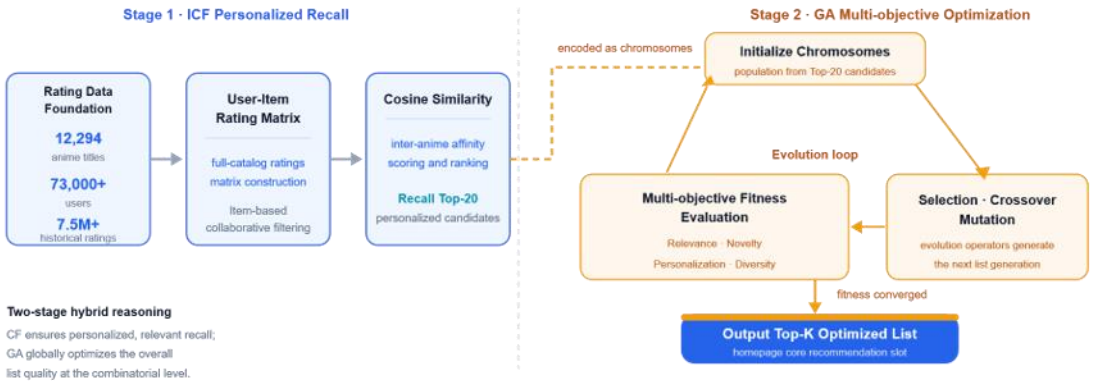


Figure 3. Algorithm 3: Main Page Recommendation Algorithm Based on All User Ratings and GA Optimization

As the core recommendation algorithm of the main page, a hybrid reasoning mechanism combining Item-based Collaborative Filtering (ICF) and Genetic Algorithm (GA) is adopted. First,

a User-Item rating matrix is constructed from the rating records described in Section 4, after the filtering and sampling outlined there. Cosine similarity is then used to calculate the associations between anime titles, retrieving the Top-20 personalized candidate items.

The candidate set is subsequently modeled as the chromosomes of the GA. A multi-objective fitness function incorporating Relevance, Personalization, Diversity, and Novelty is designed. Through evolutionary iterations involving selection, crossover, and mutation, the recommendation list is globally optimized, producing the final Top-K optimized recommendation list.

Algorithm 4: RAG-based Language-aware Search System
Technology Used: Cognitive LLM + Knowledge base + RAG

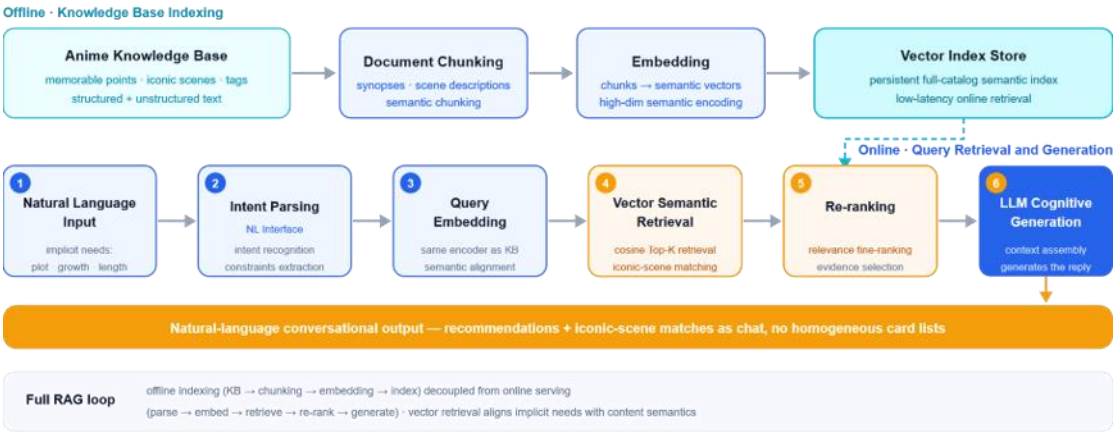


Figure 4. Algorithm 4: RAG-based Language-aware Search System

Detailed Description: Based on a cognitive computing architecture, the system integrates a Large Language Model (LLM), an anime knowledge base containing memorable moments, iconic scenes, tag attributes/structured knowledge graphs, and RAG technology to construct a natural language conversational search system. The system receives and analyzes users' implicit requirements through a Natural Language Interface, such as plot style, character development, and length constraints. RAG is used to perform vector-based semantic retrieval over unstructured anime text and iconic scene descriptions, while the relational topology of the knowledge graph is incorporated for structured logical reasoning. Recommendation results and iconic scene matches are presented in the form of natural language chat conversations, enhancing interaction awareness and avoiding homogenization with traditional card-based list formats.

6 Data Collection and Preparation

We prepare one data package for all four algorithms, so that titles, tags and sources are identified in the same way everywhere. It is built between 16 and 20 September 2026 from six CSV files and consists of a title catalogue, a user–title rating matrix, a relation graph, a set of verified scenes, and a retrieval corpus with its own evaluation queries. **Figure 5** shows the stages; the sections below follow them.

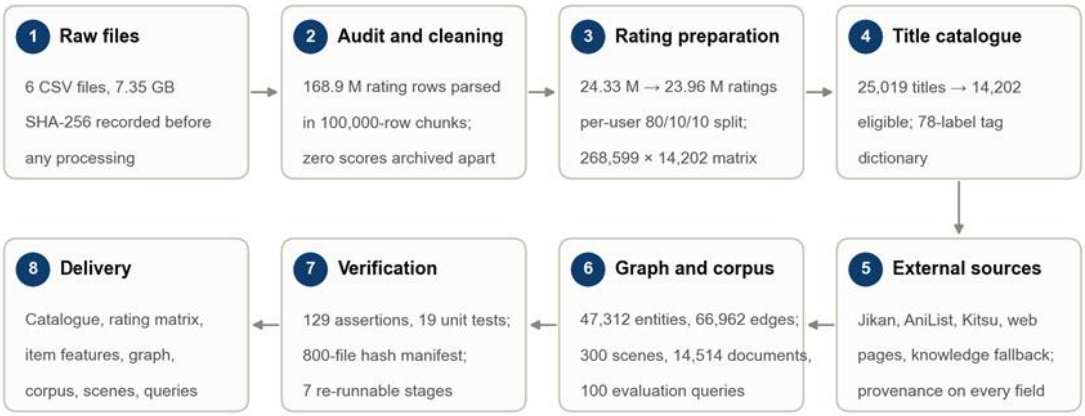


Figure 5. From six raw files to the delivered data package. The counts in the boxes are the ones reported in this section.

6.1 Data and knowledge sources

Table 1 lists what we starte from and what we added. The three 2023 files are the MyAnimeList snapshot published on Kaggle (dbdmobile/myanimelist-dataset, ODbL licence). The three older files carry no date or version; we recorded their version as unconfirmed and used them only for cross-checks and as archived ratings.

Table 1. Data and knowledge sources

Source	Content	Size	How we used it
anime-dataset-2023.csv	MyAnimeList title table	24,905 titles	Primary title records
users-score-2023.csv	User–title ratings	24,325,191 ratings	Primary ratings
users-details-2023.csv	User profile table	731,290 profiles	User table; identity checks
anime-filtered.csv	Older title table	14,952 titles	Cross-checks; 97 IDs absent from the 2023 table
user-filtered.csv	Older ratings	109,224,747 rows, 46,827,035 zeros	Archived apart, never merged
final_animedataset.csv	Older wide table	35,305,695 rows, 13,743,594 zeros	Archived apart; title attributes extracted
Jikan API	Title details, characters	200 + 7 records obtained	Planned main source; service down
AniList GraphQL	Genres, themes, relations, main characters, studios	18,038 titles	Main enrichment source
Kitsu API	Episodes, duration, type, status, synopsis	2,537 titles	Fills physical fields
MyAnimeList pages, Wikipedia	Demographics, themes, relations	14,407 stored records	Supplements with source URL and date
Wikipedia and TheTVDB episode guides	Episode lists and summaries	300 scenes of 100 titles	Evidence for scene records

The snapshot leaves three gaps. Themes, demographics, main characters and series relations are incomplete in the title table; the ratings carry no timestamps, so a temporal split is impossible; and no file holds anything at the level of a single scene. External sources were added for the first gap and episode guides for the third.

Working with these sources meant respecting rate limits, not negotiating access. Expertise is harder to find. We have no anime editor or licensed catalogue available, and the nearest substitutes

are community-maintained and sometimes disagree with each other (MyAnimeList, AniList, Wikipedia). We therefore treated every external value as evidence with an origin: each field records its source, and a value that could not be tied to a page or an API response is labelled as inferred. A commercial service would license its metadata and pay editors for tags and scene summaries; the 300-scene set is where that difference shows most. On privacy, birthdays and locations were not carried into the working tables, the rating inputs use numeric user IDs only, explicit titles were removed together with their ratings, and the data is used for recommendation research only.

6.2 Audit and cleaning of the raw files

Before touching any file we recorded its size and SHA-256 hash, then parsed every record with a strict CSV reader in chunks of 100,000 rows (the largest file is 4.5 GB, and 168.9 million rating rows were parsed in all). The audit found 9,213 titles without a score, 14,577 without an English name and 4,535 with the placeholder synopsis "No description available for this anime." in the 2023 title table. The older table gives 2,620 titles a score of exactly 6.51, which looks like a fill value (1,245 of them have no score in the 2023 table), so older scores were never used to fill gaps. A total of 542 ratings point to 17 title IDs that the 2023 table does not contain, and 521,412 ratings come from users without a profile row. We kept both groups and followed them to the end of the pipeline.

The rating rules were fixed before the full pass. A rating must be an integer from 1 to 10. A zero in the older files is neither a dislike nor a favourite, so the 60,570,629 zero records were archived on their own and kept out of the explicit-rating matrix. The older files number their users differently from the 2023 file (sampled IDs matched while the user names did not), so each source was archived separately and never merged into the 2023 ratings. Duplicate or conflicting (user, title) pairs were searched across chunks with an on-disk key table, and none occurred among the positive ratings of any source. Missing values were not filled with averages or defaults. The 2023 user table holds 737,254 users, the union of profile IDs and rating IDs.

6.3 Rating preparation

Table 2 follows the ratings from the raw file to the training data. Three rules remove ratings: the title is explicit (361,269 ratings), the title has no public score (8,223), or the title ID could not be matched to the title table (542). That leaves 23,955,157 ratings from 268,599 users. We repeated the subtraction on a second path from the exported files and recounted the three split files with a separate script; the totals agree exactly.

Table 2. From the raw ratings to the training data

Step	Ratings	Note
Rows in users-score-2023.csv	24,325,191	No invalid, duplicate or conflicting rows
Explicit titles removed	−361,269	Hentai, Erotica or rated Rx
Titles without a public score removed	−8,223	
Titles missing from the title table removed	−542	17 title IDs
Eligible ratings	**23,955,157**	**268,599 users, 14,202 titles**
Training	19,387,811	All statistics are computed here only
Validation	2,283,673	Held out from 193,009 users
Test	2,283,673	181,247 of these users have a rating of 8 or more

Each user with at least ten ratings gives floor(0.1n) ratings to validation and the same number to test, chosen by a stable hash of the seed (42), the user and the title, so the split is identical on every re-run. The 75,590 users with fewer than ten ratings stay in training and are not dropped. Among the 193,009 users with held-out ratings, 181,247 have at least one test rating of 8 or higher, the relevance threshold for the ranking metrics; the other 11,762 are counted separately, not as failures. User means, preferences, novelty and quality values come from the training part alone. The 160,311 ratings that become zero after mean-centring stay visible through an observation mask, so a centred zero is never confused with an unrated title. The matrix is stored sparse, and 14,086 finished titles with a reliable total duration are exported as candidates and boundary cases for the viewing-plan tests. The Algorithm 3 benchmark loads its training matrix, ID mappings, validation ratings and item features from these exports.

6.4 Title catalogue

Titles are identified by their MyAnimeList ID and are never merged on similar names. The full catalogue has 25,019 IDs: the 24,905 titles of the 2023 table, 97 IDs that only the older table has, and the 17 IDs found in the ratings. From it we derive a runtime set of 14,202 titles by rule. The projection is exact, and a check fails if a single title is added or missing. A title is excluded if it is explicit (1,543), has no public score (9,210), has an unresolved identity (114) or has no title (3). These reasons overlap, and 10,817 titles are excluded in total.

All records follow one schema (animefinder-data-v2). Titles and synopses are in English, other names are kept as aliases, and placeholder synopses count as missing. Durations such as “24 min per ep” become per-episode and total minutes, and an ambiguous duration stays unknown. Genres, themes and demographics are mapped to one dictionary of 78 labels (20 genres, 53 themes, 5 demographics), shared by the JSONL records, the CSV exports and the retrieval text. Every field carries a status (present, known_empty, missing or unverified), and every value carries its source, a locator such as a file and row, and a method (copied, parsed, translated or derived). A release fingerprint of input, cache and output hashes is stored with the catalogue. The Algorithm 1 benchmark takes its 14,202-title catalogue from this package, and Algorithm 2 takes its theme and demographic tags from it.

6.5 Filling the gaps from external sources

Jikan is the planned source for every title: a queue of detail and character requests, one request at a time at least 1.25 seconds apart, back-off on errors and a resumable state. From 17 September its full-detail, relations and characters endpoints answered with HTTP 504 (Jikan could not reach MyAnimeList) and kept doing so for more than 24 hours. A six-hour run sent 751 requests for 663 titles and obtained 67 successes, and ten scheduled retry rounds added nothing. We ended with 200 detail records and 7 character lists.

Because the processing guide named Jikan as the source, replacing it was a change of data rules. We opened a new run version (20260917_B_v2) instead of overwriting the old catalogue and added the layers in **Table 3**. They are applied in this order, and each one only fills fields that are still empty, so a later layer never overwrites an earlier value.

Table 3. External layers and what each one delivered

Layer	Supplied	Result
1 Jikan API	Details, characters	200 detail records and 7 character lists (endpoints down)
2 AniList GraphQL	Genres, themes, relations, main characters, studios	18,038 titles in 501 batches of 50 MAL IDs; 505 requests, 4 retries, no failed batch; 18,024 of 25,019 titles covered (72.0%)
3 Kitsu API	Episodes, duration, type, status, synopsis	6,995 titles looked up: 2,537 found, 4,458 not found, 0 errors
4 MyAnimeList pages and Wikipedia	Demographics, themes, relations	14,407 stored records, each with source URL and observation time
5 Knowledge fallback	Audience and theme labels of well-known titles	128 titles, each with a written rationale, labelled inferred

For 6,995 titles no layer has an entry; 147 batch requests for the 6,998 IDs missing from AniList returned nothing. They are recorded as having no external source so that nobody retries them. Demographic labels were the hardest field: AniList has no such field and Wikipedia infoboxes rarely give one. Two routes worked, Wikipedia's audience categories (including those of the light novels that many series adapt) and the MyAnimeList pages themselves. The less known the title, the more often its page carried a demographic label (71% in the last batch).

The Wikipedia route needed a second look. Its audience categories describe the manga or light novel an anime was adapted from, not the anime, and a page that covers a whole franchise can carry contradictory categories (the page for Full Metal Panic! lists both Seinen manga and Shōnen manga). A script re-fetched the live category list behind all 1,393 Wikipedia-based demographic values. For 1,386 of them the page had only manga categories or conflicting ones, and for 7 it had no audience category. All 1,393 were moved from external_verified to ai_inferred, with the reason and the original categories written into each record. Coverage did not change, but the verified demographic count fell from 6,763 to 5,370, and anyone using the data can now filter on that label.

When ten consecutive rounds had brought no new record, we closed the remaining Jikan work under the processing guide's rule for unavailable services. The 46,747 open tasks (23,277 detail and 23,470 character tasks) were marked closed_unavailable, which is not the same as success: their fields stay missing. The closure was authorised on 18 September, and the task counts before and after, together with eight pieces of evidence, are kept in external_closure_record.json. The catalogue status changed from provisional to final, and Jikan can be resumed from the saved queue if the service recovers.

Table 4 gives the result and **Figure 6** shows where each field came from. Over the web pass the demographic coverage of the 25,019 titles rose from 27.0% to 50.3%, theme coverage reached 71.2%, and 46.5% of the titles have at least one relation. The last column of the table removes inferred values, which is the view the algorithm benchmarks use.

Table 4. Share of titles with a value after enrichment

Field	All titles (25,019)	Runtime set (14,202)	Runtime set, inferred removed
Genres	82.7%	90.1%	90.1%
Themes	71.2%	83.2%	69.1%
Demographics	50.3%	49.7%	37.4%
Studios	61.6%	78.8%	78.8%
Main characters	46.0%	66.8%	66.8%
Relations (at least one)	46.5%	60.8%	60.8%
English synopsis	84.5%	99.3%	99.3%
Episode count	99.2%	99.8%	99.8%

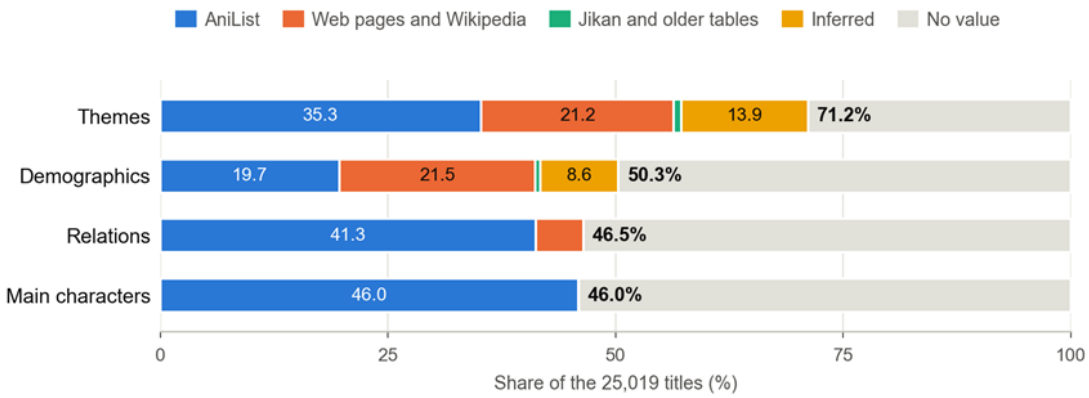


Figure 6. Where the values of four fields come from, as a share of the 25,019 titles (numbers inside the bars are percentages). Inferred covers keyword rules on title and synopsis, sequel-suffix matching, general knowledge for 128 well-known titles and the 1,393 relabelled Wikipedia values.

6.6 Relation graph, scenes and retrieval corpus

The graph has 47,312 entities (14,202 titles, 19,556 characters, 11,883 studios and 1,671 franchises) and 66,962 edges: 34,907 title–character, 12,211 title–studio, 4,569 title–franchise and 15,275 title–title relations such as sequel, prequel, side story and alternative version. Only characters with the role Main enter the displayed list; the full API response stays in the cache. Franchise groups come from explicit prequel and sequel links inside the runtime set, never from names such as “Season 2”. A relation is kept only if both ends are in the catalogue: at catalogue level 21,694 relations were kept and 1,260 that point outside were written to a pending file, so none was dropped silently or forced in, and the graph has no dangling edge.

The scene set has 300 scenes for 100 titles, exactly three per title: 60 TV series, 25 films, 6 OVA, 5 ONA and 4 specials, released between the 1980s and the 2010s. Each record has 16 fields, including an English description, keywords, characters, the episode and its sources. A scene counts as verified only when a source page was opened and the record holds its URL, the page title, the access time, a note on where in the page the evidence sits, a short verbatim excerpt and the path of the cached page. Descriptions written from memory were not accepted. Where a source gave no reliable episode number the field was left empty (24 of 300), and no timestamps were recorded.

The two halves of 150 scenes are cross-checked. Three traps turn up: Wikipedia’s list of Hellsing episodes mixes the 2001 TV series with the Ultimate OVA, the two cours of Fate/Zero are numbered continuously there but are separate entries in our catalogue, and Code Geass R2 is numbered 26–50 there and 1–25 here. Eight scenes are replaced because their evidence does not belong to the title they are filed under, two descriptions are trimmed to what the cached excerpt

supports, and the cached pages of the eight replacements were opened one by one. The rules draw from these cases are part of the verification instructions: the evidence note names the page section and the year, and an episode number is checked against the catalogue's episode count.

For retrieval we build 14,214 work documents and 300 scene documents, in English, in chunks of at most 400 words; a work document holds the title, aliases, type, genres, themes, demographics and synopsis as text, with episodes, score and duration kept as metadata for filtering. The evaluation set has 100 queries: 30 semantic descriptions, 20 hard-constraint queries, 10 without an answer and 40 scene queries. None of them enters the corpus, and a check enforces that. The expected answers of the constraint queries are computed from the structured fields, for example by running an episode limit, a score floor and a genre filter on the catalogue, not written by hand. Each of the 10 queries without an answer was run against the corpus and returns nothing. The scene queries are paraphrases of verified scene descriptions with the title words removed, and each carries the expected scene ID.

6.7 Verification and reproducibility

The rules of the processing guide are turned into automatic checks. The catalogue stage has 37 assertions, among them: IDs unique, sorted and positive; the runtime set equals the eligibility rules applied to the full catalogue; no placeholder synopsis; only characters with the role Main; no dangling graph edge; the status and method vocabularies respected; evaluation queries absent from the corpus; raw files unchanged. The base cleaning suite has 44 assertions, the joint catalogue and graph suite 18, the rating export suite 17 and the scene, corpus and viewing-plan suite 13. Together with 19 unit tests, including one for the lock that stops two processes from writing the same output, all of them passed on the runs we delivered. The delivery manifest lists the SHA-256 of 800 files; after sealing, 799 still matched, and the 800th is the running development log, which kept growing.

The pipeline runs in seven stages: profile, catalogue, web, evaluation queries, exports, verify and report. Each stage is one command, all settings are in config_b.json, and every run writes environment.json, manifest.json and a reproducibility record. When we rebuilt the catalogue in two separate processes, 378 titles differed only in the order of their aliases, because ties such as "AIR" and "Air" were sorted by an unstable set order; using the original text as a second sort key made both runs byte-identical. On our machine the catalogue stage takes 23 seconds, the rating export about 8 minutes and verification about 5 minutes, with a peak memory of 2.1 GiB.

6.8 Problems and remaining gaps

Table 5. Main problems and what we did about them

Problem	What we did	Outcome
Jikan answered 504 for more than 24 hours	Added AniList as a second source under a new run version; closed the remaining Jikan tasks with recorded evidence	18,024 of 25,019 titles covered; catalogue final
Retryable 504 errors were stored as permanent failures	Read the status code from the saved error text and reset 124 tasks	No task skipped for good
Wikipedia categories describe the manga, not the anime	Re-fetched all 1,393 pages and relabelled the values	Verified demographics 6,763 → 5,370; coverage unchanged
One source page covers several series (Hellsing, Fate/Zero, Code Geass R2)	Evidence note must name page section and year; episode checked against the catalogue	8 scenes replaced, 2 descriptions trimmed
Scene queries pointed at titles without scenes (0 of 40 could be scored)	Rebuilt them from verified scenes and added an assertion	40 of 40 anchored
Stale background crawlers overwrote the supplement file (3,885 → 3,306 records)	Stopped the processes, rebuilt the file from the catalogue, and required every verified value to be in it before a rebuild	No record lost
Values outside the status and method vocabularies	Mapped them and turned both vocabularies into assertions	Checks 32 → 37
7.35 GB of CSV on one workstation	Chunked reading, an on-disk key table and checkpoints	Peak memory 2.1 GiB

Three gaps remain. About half of the titles have no demographic label, and 6,995 titles have no entry in any external source. The Jikan work was closed, not completed. And the ratings have no timestamps and no real user collections exist, so collection-based experiments rely on proxies. These limits carry into every experiment that uses the data.

7 Algorithm Comparison and Model/Task Level Evaluation

7.1 Algorithm 1: Comparing Candidate Routes for Knowledge-Based Similar-Anime Recommendation

Key finding: On a benchmark of 1,000 targets, we compared four candidate routes. Route A, the fixed five-dimensional weighted similarity, ranks first on every metric and is therefore chosen as the final route for Algorithm 1. Its coverage@5 is 30.26%, about 859× the uniform random baseline; its hit_rate@5 is 67.50%, about 384× the baseline. The runner-up, Route D, trails A by 4.46 percentage points on coverage@5, and both the Wilcoxon and McNemar tests show that the gap is significant. Stratified by difficulty (easy, medium, hard), A still leads D by 4.3 to 4.6 percentage points. None of B, C, or D cleared the pre-registered replacement gate, so A is retained; D is kept as a semantic-enhancement candidate, to be activated only after evaluation with real users.

7.1.1 Task Definition and Experimental Setup

Algorithm 1 is a self-contained knowledge-based recommendation module. Given the MAL ID of a target anime, it returns the Top-5 similar anime from a runtime catalog of 14,202 entries. The module uses no user profiles, rating histories, favorites, or chat context. All four candidate routes read the same five projected fields and exclude the target itself, leaving 14,201 candidates per query.

Table 6. Feature projection rules and missing-value handling for the five fields

Field	Projection rule	Missing-value handling
genre	Deduplicated set, Jaccard	0 if either set is empty
theme	Deduplicated set, Jaccard	0 if either set is empty
demographic	Case-insensitive exact match	Missing values never match
studio	Case-insensitive exact match	Missing values never match
episodes	1 / (1 + abs(e1 − e2) / 12)	0.5 if missing or non-positive

Titles, synopses, ratings, relations, and user history are never used as inference inputs. The four routes are as follows.

- Route A — fixed five-dimensional weighted similarity. A = 0.35 × genre + 0.30 × theme + 0.15 × demographic + 0.10 × studio + 0.10 × length. It requires no training, and the contribution of each component can be explained directly to users.
- Route B — five-dimensional logistic regression. The same five numeric features feed a logistic regression whose ranking score is sigmoid(w^T x + b). The model uses optimization results that were frozen in advance; test labels are never used to retune it during evaluation.
- Route C — BGE semantic similarity. The five fields are concatenated into a fixed text template and encoded by BAAI/bge-small-en-v1.5 into 384-dimensional CLS vectors, then mean-centered and L2-normalized over the full catalog; ranking is by cosine similarity.
- Route D — tag-and-semantic fusion. D = 0.6 × A + 0.4 × semantic. The semantic cosine is first mapped to [0, 1] using the 1st and 99th percentiles of the full catalog, so no per-query renormalization is needed.

Ties are broken by score in descending order, then by anime_id in ascending order, which keeps the results reproducible.

Test protocol. The test set consists of 1,000 targets. We first hash the catalog IDs with SHA-256: each ID is fed into the key target:20261003:<anime_id>, the first 8 bytes of the digest are read as a big-endian integer, and the IDs are sorted by that value. This step determines the test sample only and is independent of any prediction outcome.

Ground truth is produced by a separate metadata oracle. For each target, every other catalog entry is scored as 0.50 × genre Jaccard + 0.30 × theme Jaccard + 0.10 × demographic exact match + 0.10 × length similarity, and N(0, 0.05) noise drawn from a fixed seed is added. The studio field plays no part in the oracle, so the oracle's construction differs from that of all four routes. After sorting by the noisy scores, five items are drawn without replacement under a softmax temperature of 0.035 to form the ground truth: easy, 40% of targets, drawn from a pool of 8 candidates; medium, 40%, from a pool of 20; hard, 20%, from a pool of 60. The realized split was 377 / 412 / 211.

All ground truth is generated before any model ranking is computed, and the generator records the seed, the rules, and the catalog hash. Since 949 of the 1,000 targets overlap with the weak-supervision endpoints used to train B, this set serves as a regression test rather than a fully independent, leak-free validation set.

7.1.2 Evaluation Metrics

Let T be the target's ground-truth set (|T| = 5) and R5 the Top-5 list returned by the model. All metrics are macro-averaged over targets.

Table 7. Evaluation metrics and their interpretation

Metric	Definition	Interpretation
coverage@5	abs(R5 ∩ T) / abs(T)	Recall@5; the primary metric in this report
hit_rate@5	1 if R5 ∩ T is non-empty, else 0	Standard binary hit rate
NDCG@5	Binary relevance; ideal length min(5, abs(T))	Ranking quality that accounts for hit positions
precision@5	Top-5 hits / 5	Identical to coverage@5 because abs(T) = 5
hit@1	1 if the first returned item is relevant	Quality of the top position
MRR	Mean reciprocal rank of the first relevant item	Quality of early positions

Random baseline. Drawing five items uniformly at random from the 14,201 candidates gives an expected coverage@5 of 5 / 14,201 = 0.0352% and an exact hit_rate@5 of 1 − C(14,196, 5) / C(14,201, 5) = 0.1759%. Lift is defined as the metric value divided by the corresponding random baseline. Because the oracle draws ground-truth items with score-weighted rather than uniform probabilities, the uniform baseline serves only as a conservative lower bound.

7.1.3 Overall Comparison of the Four Routes

All four routes are evaluated on the same 1,000 targets, against the same ground truth and the same catalog of 14,201 candidates.

Table 8. Overall comparison of the four routes on the 1,000 targets

Route	coverage@5	NDCG@5	hit_rate@5	hit@1	MRR	Lift (coverage@5)	Lift (hit_rate@5)
A — fixed weighted	30.26%	0.3288	67.50%	41.70%	0.5148	859×	384×
D — tag-and-semantic fusion	25.80%	0.2855	63.20%	38.70%	0.4791	733×	359×
B — logistic regression	21.92%	0.2371	54.90%	30.30%	0.3924	623×	312×
C — BGE semantic	13.54%	0.1529	42.50%	21.40%	0.2935	385×	242×

All four routes beat the uniform random baseline by two to three orders of magnitude. A leads on every metric and D is second. B and D come close to A: on average, 72.8% of the positions in B's Top-5 coincide with A's (61.7% for D), and B returns exactly A's Top-5 on 43.0% of targets (20.5% for D). B and D therefore do not fall behind because they explore other parts of the catalog; rather, when they deviate from A, they are more often wrong. C's lists vary the most — its Top-5 overlaps A's on only 31.5% of positions — and it posts the lowest metrics on this metadata-defined task.

7.1.4 Stratified Results by Sampling Group

Table 9. Stratified coverage@5 by sampling group

Route	easy coverage@5 (n=377)	medium coverage@5 (n=412)	hard coverage@5 (n=211)	Overall
A	30.13%	30.34%	30.33%	30.26%
D	25.52%	25.92%	26.07%	25.80%
B	21.75%	21.94%	22.18%	21.92%
C	13.16%	13.30%	14.69%	13.54%

Table 10. Stratified hit_rate@5 by sampling group

Route	easy hit_rate@5	medium hit_rate@5	hard hit_rate@5	Overall
A	63.66%	69.90%	69.67%	67.50%
D	62.07%	63.83%	63.98%	63.20%
B	51.19%	57.77%	55.92%	54.90%
C	41.38%	42.96%	43.60%	42.50%

The ordering of stratified NDCG@5 is unchanged across the three strata: A comes in at 0.3168 / 0.3362 / 0.3358, D at 0.2744 / 0.2896 / 0.2973, B at 0.2290 / 0.2396 / 0.2468, and C at 0.1428 / 0.1543 / 0.1683. A leads D by 4.62, 4.42, and 4.27 percentage points on easy, medium, and hard respectively, leads B by 8.2 to 8.4 percentage points, and leads C by 15.6 to 17.0 percentage points. Pool size does not change the ordering of the routes.

Stratified coverage@5 shows similarly little variation — A varies by less than 0.3 percentage points across strata. This indicates that with a softmax temperature of 0.035, the five draws concentrate in the highest-scoring part of each pool. The clearer stratification shows up in hit_rate@5: for every route, the easy stratum runs about 6 percentage points below medium and hard.

7.1.5 Statistical Significance, Overlap Analysis, and the Replacement Gate

We run the Wilcoxon signed-rank test on the per-target paired differences, defined as challenger minus A, using a normal approximation with a tie correction:

Table 11. Wilcoxon signed-rank tests of each challenger against Route A

Comparison	Metric	Non-zero pairs	z	p
A vs D	coverage@5	339 / 1,000	8.41	4.1×10^{-17}
A vs D	NDCG@5	438 / 1,000	9.31	1.3×10^{-20}
A vs B	coverage@5	301 / 1,000	14.30	2.2×10^{-46}
A vs B	NDCG@5	403 / 1,000	16.24	2.6×10^{-59}
A vs C	coverage@5	556 / 1,000	18.23	3.0×10^{-74}
A vs C	NDCG@5	633 / 1,000	19.11	2.1×10^{-81}

A and D return the same hit count on 661 of the 1,000 targets. In other words, D's semantic component changes the hit set on only about a third of the targets, and most of those changes make the results worse.

For hit_rate@5 we use the exact McNemar test, where b counts the targets that A hits and the challenger misses, and c the reverse:

Table 12. Exact McNemar tests on hit_rate@5 against Route A

Comparison	b	c	Exact p
A vs D	85	42	1.7×10^{-4}
A vs B	127	1	7.6×10^{-37}
A vs C	284	34	2.8×10^{-50}

The table below lists 95% confidence intervals for the per-target paired differences, again defined as challenger minus A; the data come from the release summary:

Table 13. 95% confidence intervals of the paired differences (challenger minus A) and the replacement-gate result

Challenger	coverage@5 Δ	95% CI (pp)	NDCG@5 Δ	95% CI	Replacement gate
D	−4.46 pp	[−5.40, −3.52]	−0.0433	[−0.0525, −0.0344]	Failed
B	−8.34 pp	[−9.34, −7.36]	−0.0916	[−0.1016, −0.0814]	Failed
C	−16.72 pp	[−18.22, −15.24]	−0.1759	[−0.1912, −0.1606]	Failed

All confidence intervals lie entirely below zero, so the challengers' deficits (that is, the shortfalls of Routes B, C, and D relative to A) are not random noise. None of B, C, or D cleared the replacement gate pre-registered against A.

7.1.6 Runtime and Reproducibility

Scoring all four routes for a single target takes 40.26 ms at p50 and 43.78 ms at p95. The benchmark ran on a single Windows 11 machine with Python 3.12.14, and the full evaluation of 1,000 targets × 4 routes took 40.5 seconds. The production endpoint runs A only, so its per-query cost is even lower than these evaluation figures.

To reproduce the experiments, run the following commands from the algorithm1_benchmark directory:

```
python generate_synthetic_1000.py --output results/replay_synthetic_1000
python run_checks.py --output results/replay_synthetic_1000
python verify_delivery.py
```

Replay results are written to a separate results/replay_synthetic_1000 directory and never overwrite the release directory. The release artifacts — testset.jsonl, ground_truth_used.jsonl, predictions.jsonl, per_query_metrics.jsonl, metrics.csv, summary.json, and testset_manifest.json — are sealed with a SHA-256 manifest.

7.1.7 Problems Encountered and Solutions

Problem 1 Handling incomplete metadata and tied similarity scores

A practical implementation problem is how to score anime with incomplete metadata. Some catalog entries have no genre or theme labels, no demographic or studio value, or an unknown episode count. Treating two empty fields as a match would reward the absence of information: two poorly documented titles could receive a high score without sharing any known attribute. Conversely, discarding every incomplete title would reduce the candidate catalog and change the evaluation population. Short tag lists also produce many identical feature vectors, so ties at the Top-5 boundary require an explicit ordering rule.

We address this by defining the missing-value rules in one shared feature module. Genre and theme labels are deduplicated before Jaccard similarity is calculated, and the score is zero

whenever either set is empty. Demographic and studio values are compared case-insensitively only when both values are present; missing values therefore never count as exact matches. A missing or non-positive episode count receives the fixed fallback value of 0.5, while valid counts use the same 1 / (1 + abs(e1 - e2) / 12) function in the shared numeric feature representation. This fallback is a declared scoring convention, not evidence that two unknown lengths are actually similar. We retain incomplete entries rather than inventing their metadata.

The target is excluded before selecting recommendations, and candidates are sorted by decreasing score and then increasing anime_id. This makes tied results independent of the input order. Unit tests check empty fields, duplicate labels, numeric episode distances, tied scores, and self-exclusion. The solution keeps the feature rules and ranking behavior consistent across the four routes. It does not recover missing information, but it prevents missing tag fields from creating false matches and makes the resulting comparisons reproducible.

Problem 2 Semantic enhancement did not outperform fixed weighted similarity

The second problem is that a more complex semantic route does not deliver better recommendations on the declared benchmark. Route C achieved coverage@5 of 13.54%, while the tag-and-semantic fusion in Route D achieved 25.80%, still below A's 30.26%. An important constraint is that the embedding model receives only short descriptions of the same five metadata fields. It does not receive plot summaries or additional evidence about narrative style. Moreover, the synthetic oracle is itself based on metadata overlaps, which gives directly weighted features a structural advantage. These observations provide a plausible explanation for the gap, rather than proving that semantic models are unsuitable for real-user recommendation.

We handle the engineering side by keeping the semantic representation and its scale fixed. The BGE vectors are mean-centered over the catalog and L2-normalized, while Route D uses frozen catalog-wide 1st and 99th percentile bounds to map cosine similarity into [0, 1]. We avoid per-query normalization, which would make the same cosine value contribute differently depending on the other candidates in that query. The fusion remains 0.6 times A plus 0.4 times the calibrated semantic score. This provides a consistent comparison; it does not by itself establish that the fusion improves recommendation quality.

For route selection, we compare all four methods on the same 1,000 targets instead of judging semantic enhancement from a few attractive examples. A's advantage over D remains across all three sampling groups, and the paired 95% confidence interval for D minus A on coverage@5 is entirely negative at [-5.40, -3.52] percentage points. We therefore retain A as the selected route and keep D as an experimental candidate. The immediate solution is evidence-based route selection, not a claim that semantic enhancement has been made superior. A future replacement requires evaluation against independent expert or real-user labels, because the current synthetic benchmark alone cannot establish that a route generalizes to user preferences.

7.1.8 Conclusion

On this 1,000-target benchmark, Route A — the fixed five-dimensional weighted similarity — ranks first on every metric and is selected as the final route for Algorithm 1. Its advantage over the runner-up D passes both the Wilcoxon and McNemar tests and stays stable across the easy, medium, and hard strata. None of B, C, or D meets the replacement gate, so A remains the production route; D is retained as a semantic-enhancement candidate, to be activated only after evaluation with real users. The 1,000-target benchmark will continue to serve as the regression suite for Algorithm 1.

7.2 Algorithm 2: Collection-Based Recommendation with Rules and a Learned Scorer

Key finding: Under a protocol frozen before any training, we compared the three scorer families of the proposal on 3,000 held-out users, ranking the full catalogue of 14,182 titles. A gradient-boosted tree over eight collection-profile features (B1) gave the best Top-8: HR@8 0.132 against 0.021 for the original formula. It passed every pre-registered check on the test set (+11.1 pp HR@8 over the formula, 95% CI [+9.8, +12.3]; AUC 0.953; zero rule violations) and is now the served scorer in real-data mode, with the formula as an automatic fallback.

7.2.1 Task and Candidate Scorers

On the collection page, Algorithm 2 recommends eight titles from what the user has saved, under filters the user sets: keywords, a minimum score and a maximum episode count. As planned in the proposal presentation, the pipeline has a **fixed rule layer** and a **replaceable scorer** (**Figure 7**). The rule layer removes saved and explicit titles and every title that fails a filter, and a missing value never satisfies a filter. Only the scorer changes between routes, so any difference in the results comes from the scorer.

We keep the proposal's three candidate families and added two controls, so that each step changes one thing:

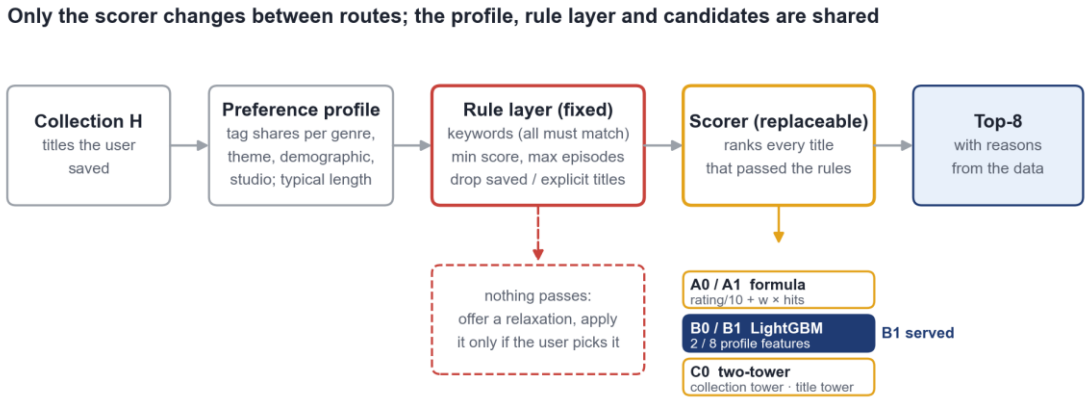


Figure 7. Algorithm 2 pipeline. The rule layer is fixed; the five scorers are interchangeable, and B1 is served.

7.2.2 Ground Truth and Protocol

The proposal presentation names real user collections as the primary ground truth. The application has none yet, so we use a proxy with the same leave-one-out design: for each Kaggle user, the titles rated 8 or higher stand in for a collection, one of them is hidden, and the scorer must rank it into the Top-8 of the whole catalogue. Unrated titles act as weak negatives, and positives and negatives pass the same rule filter, as the presentation required.

Users are split 70/15/15 at user level (24,000 users; 3,000 main-group test users with at least five visible titles). B0, B1 and C0 share the same 247,380 training pairs (one positive, four weak negatives) and are tuned on validation users only. Before any training we froze the selection rules: the presentation's replacement rule (AUC $\geq$ 0.65 and HR@8 at least 2 pp above A0), plus zero rule violations and paired 95% bootstrap intervals above zero against both A0 and A1. The test set is scored once, after the models were frozen.

7.2.3 Results

Table 14. Test results (3,000 users, no filter, full-catalogue ranking; learned models at the deployed seed)

Scorer	HR@8	NDCG@8	AUC	ΔHR@8 vs A0, pp [95% CI]
A0 original formula	2.1%	1.1%	87.0%	–
A1 tuned formula	3.2%	1.8%	90.8%	+1.1 [+0.6, +1.6]
B0 LightGBM, 2 features	5.4%	3.2%	92.1%	+3.3 [+2.5, +4.2]
B1 LightGBM, 8 features (served)	**13.2%**	**7.2%**	**95.3%**	**+11.1 [+9.8, +12.3]**
C0 two-tower	10.8%	6.2%	96.4%	+8.7 [+7.5, +9.9]

All five scorers beat a random Top-8 (about 0.06%) by one to two orders of magnitude. **Figure 8** splits B1's lead into steps, using the mean of three training seeds: tuning the weight adds 1.1 pp, learning the scoring function from the same two inputs adds 2.1 pp, and the six profile features add 7.9 pp, with every interval above zero. C0 has the highest AUC but is 1.9 pp below B1 on HR@8 [−3.0, −0.7]: it ranks the target well against the whole catalogue but less often into the eight positions the page shows, and HR@8 is named the primary metric in advance.

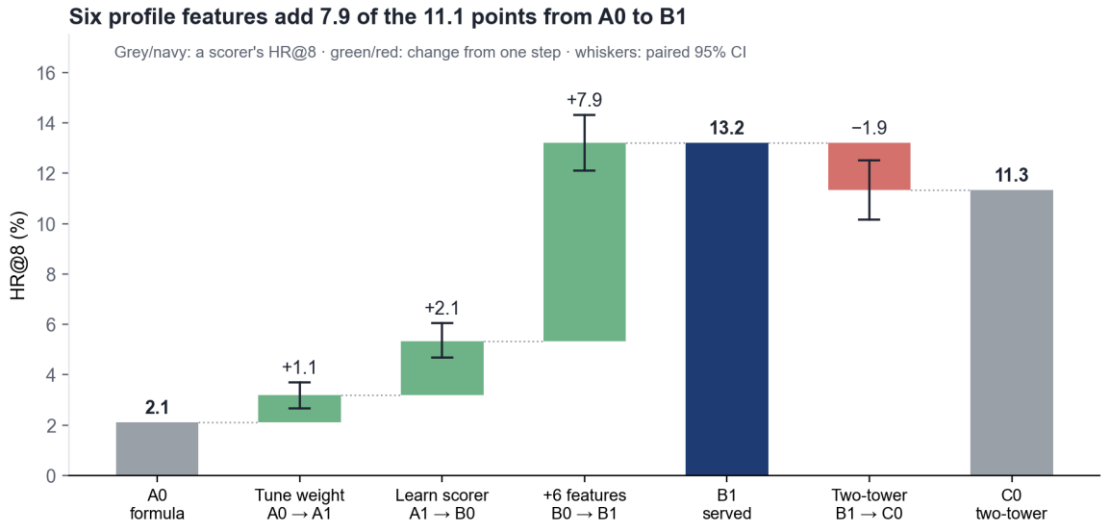


Figure 8. Where B1's gain comes from: paired differences between neighbouring scorers on the test set.

7.2.4 Subgroups and Failure Cases

B1 leads in every subgroup (**Figure 9**). Filters make the task easier when the hidden title passes them, because fewer titles compete, while long histories and mixed tastes make it harder; for mixed tastes B1 and C0 are almost level. Strict filters can also exclude the hidden title (in 48% of keyword queries). Counting those as misses, B1 still has the highest end-to-end hit rate in every filter scenario, and no scorer ever recommended a title that broke a filter or was already saved.

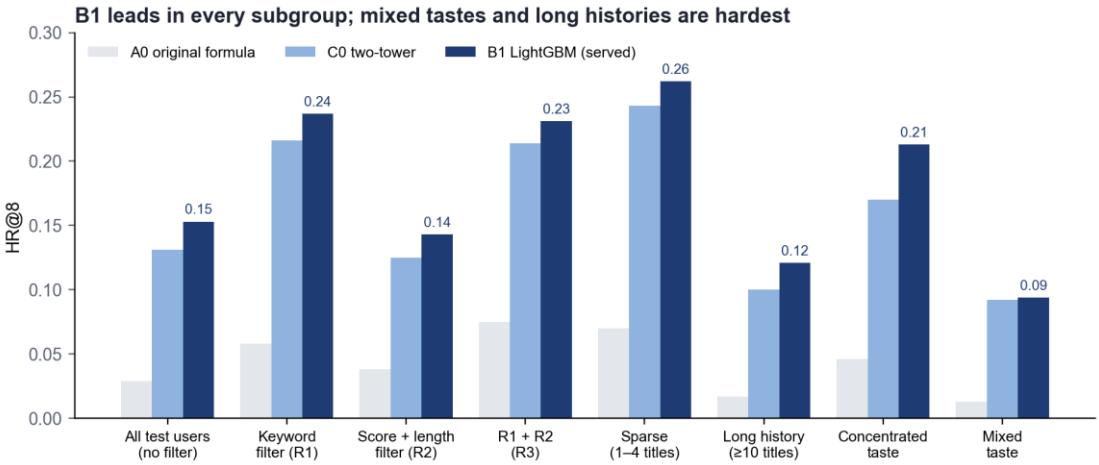


Figure 9. HR@8 by filter scenario, collection size and taste profile (test users, deployed seed).

Most misses are shared: in 2,399 of 3,600 test queries no scorer placed the hidden title in its Top-8. In 26 queries the formula found it and the learned models did not; typically the title matched the collection's tags closely but had a lower public score, which B1 weighs heavily (66% of the trees' gain).

7.2.5 Decision, Integration and Limitations

B1 passes all five confirmation checks and replaced the formula as the default. A run with only 150 test users had pre-selected C0 but failed the interval check, which shows why the rules were fixed in advance. The website calls the same profile, rule and scorer code as the experiment: on a copy of the real database the API's Top-8 matched the offline Top-8 for 1,000 of 1,000 requests, and all 14 integration checks passed. If the model file or catalogue changes, the service falls back to the formula and keeps the filters strict. B1 answers a full-catalogue request in 129 ms (p50); because LightGBM and PyTorch could not share a process on macOS, the site scores the model with its own NumPy reader, which matched native LightGBM exactly on 560,000 rows.

Three limitations remain:

- **Proxy label.** A set of titles rated 8 or higher is not a collection; the gain must be re-checked once real collections exist.
- **Popularity bias.** Most hidden titles are popular and highly scored, which favours the public-score feature; obscure titles rarely reach the Top-8.
- **Scope.** The test uses the static 2023 snapshot, and C0's ability to score unseen titles was not tested separately.

7.3 Algorithm 3 ICF and GA Experiment (Recall Comparison · Reranking Gains)

Comparative conclusion: ICF is responsible for recalling the correct items (295× for HR@10 and 431× for HR@5), while GA is responsible for reallocating these 10 positions. Under the full-catalog setting, the number of long-tail item positions in users' Top-10 recommendations increased from 1,829 to 3,161 (+72.8%), and the proportion of users exposed to long-tail items rose from 2.33% to 3.45% (222 additional users), at the cost of a decrease in HR@10 from 20.93% to 19.51%.

GA does not simply "shuffle the list." When 10 items are randomly selected from exactly the same candidate window used by GA (Window Random arm), the hit rate is only 13.67%; GA achieves

1.43 times this value. This indicates that the gains from GA come from its reranking capability rather than randomness.

7.3.1 Experimental Setup

1. Leave-one-out: For each user, one highly rated record (≥ 8.0) is held out from the training set. It appears neither in training nor in recall and is used only as the ground-truth answer. Eligible users must have at least 8 ratings in the training set. A fixed random sample of 20,000 users is then drawn from the eligible user pool.
2. Candidate-set construction: The single held-out item is mixed with (N−1) items that the user has never rated, with 10 candidate-pool sizes: N = 20 / 50 / 100 / 200 / 500 / 1000 / 2000 / 5000 / 10000 / full catalog. All users share the same queue and sampling seed, so every configuration is compared using the same users and the same candidates.
3. Recall engines (3): ICF item-based collaborative filtering, SVD matrix factorization, and NCF neural collaborative filtering. They score and rank items in the candidate set → reported in "Part I."
4. Reranking arms (4; using the same candidate pool and the same recall result, changing only "how these 10 items are selected"): Greedy (selecting the Top-10 purely by relevance, i.e., GA's internal baseline), Window Random (randomly selecting 10 items from the relevance Top-20), Pool Random (randomly selecting 10 items from the entire candidate pool without considering relevance), and GA (0.4 relevance + 0.3 diversity + 0.2 novelty + 0.1 freshness) → reported in "Part II."

7.3.2 Evaluation Matrix

To comprehensively evaluate recommendation accuracy, long-tail item exposure, and the optimization effect of the Genetic Algorithm (GA), this study uses Hit Rate (HR), Random Baseline, Lift, Novelty, Long-Tail Ratio, Long-Tail Exposure, Long-Tail Reach Rate, and GA Objective Score as the main evaluation metrics.

Recommendation System Evaluation Metrics

To comprehensively evaluate recommendation accuracy and the effect of GA on long-tail recommendation and recommendation quality, the following evaluation metrics are used.

Hit Rate HR@5: HR@5 represents the proportion of users whose ground-truth interacted item appears in the Top-5 recommendation list and is used to evaluate recommendation accuracy at higher-ranked positions.

$$\text{HR@5} = \text{Number of users with a Top-5 hit} / \text{Total number of users}$$

Hit Rate HR@10: HR@10 represents the proportion of users whose ground-truth interacted item appears in the Top-10 recommendation list. Because GA ultimately generates Top-10 recommendation results in this study, this metric is used as the primary recommendation-accuracy metric.

$$\text{HR@10} = \text{Number of users with a Top-10 hit} / \text{Total number of users}$$

Random Baseline: The random baseline represents the theoretical hit rate when K items are randomly selected from N candidate items and is used as a basic reference for recommendation-algorithm performance.

$$\text{Random@K} = K / N$$

Lift: Lift measures the performance improvement of the recommendation algorithm relative to the random recommendation baseline and reduces the effect of different candidate-pool sizes on hit-rate comparisons.

$$\text{Lift@K} = \text{HR@K} / (K / N)$$

Novelty nov@K: nov@K measures the overall novelty of the Top-K recommendation results. A higher value indicates that the recommendations tend to contain more low-popularity items.

$$\text{novelty}(i) = 1 - \log(1 + \text{popularity_i}) / \log(1 + \text{popularity_max})$$

$$\text{nov@K} = \text{Average novelty}(i) \text{ of Top-K items}$$

Long-Tail Ratio lt@10: lt@10 represents the proportion of long-tail items in the Top-10 recommendation results. In this study, items with novelty(i) ≥ 0.6156 are defined as long-tail items.

$$\text{lt@10} = \text{Number of long-tail items in Top-10} / 10$$

Long-Tail Exposure LTE@10: LTE@10 represents the cumulative number of appearances of long-tail items in the Top-10 recommendation results of all users and is used to measure the overall exposure received by long-tail items.

$$\text{LTE@10} = \text{Sum of long-tail items in all users' Top-10 lists}$$

Long-Tail Reach Rate LTR@10: LTR@10 represents the proportion of users whose Top-10 recommendation results contain at least one long-tail item. It is used to measure user-level coverage of long-tail recommendations and serves as the primary metric for evaluating long-tail recommendation effectiveness in this study.

$$\text{LTR@10} = \text{Number of users receiving at least 1 long-tail item recommendation} / \text{Total number of users}$$

GA Objective Score GOS@10: GOS@10 jointly measures GA's optimization effect on recommendation diversity, novelty, and freshness, corresponding to optimization objectives with a combined weight of 0.6 in the GA fitness function.

$$\text{GOS@10} = 0.3 \times \text{Diversity@10} + 0.2 \times \text{Novelty@10} + 0.1 \times \text{Freshness@10}$$

The remaining weight of 0.4 corresponds to recommendation relevance, while HR@5 and HR@10 are used to monitor possible changes in accuracy during GA optimization.

7.3.3 Definition of Novelty

In this study, we use a continuous Novelty metric to define long-tail items rather than directly dividing items according to sales or popularity rankings. The novelty of item (i) is defined as:

$$\text{novelty}(i) = 1 - \log(1 + \text{popularity_i}) / \log(1 + \text{popularity_max})$$

Here, popularity_i represents the number of times item (i) is rated in the training set, and popularity_max represents the maximum number of ratings received by any item in the dataset. The logarithmic transformation reduces the influence of highly popular head items on the numerical distribution and improves the discriminability of novelty across different items. The metric ranges from [0, 1], with higher values indicating lower item popularity.

In this study, we use the median novelty value of all items in the catalog, 0.6156, as the long-tail threshold and defines items with novelty(i) ≥ 0.6156 as long-tail items. Among all 14,202 items, 7,110 (50.1%) are classified as long-tail items. Meanwhile, item interactions show clear head concentration: the top 1% of items account for 24.9% of all ratings, while the top 10% account for

75.8%. Using the median as the threshold reduces the subjectivity introduced by manually selecting a threshold and allows automatic adjustment according to the distribution of different datasets.

The average novelty of items in users' historical interactions is only 0.148, with long-tail items accounting for 0.1%, indicating that users' historical behavior is mainly concentrated on popular items. Therefore, increasing the proportion of long-tail items in recommendations may affect recommendation accuracy to some extent, requiring a trade-off between long-tail recommendation capability and the degree of match with user preferences.

It should be noted that higher novelty only indicates fewer interactions with an item in the training set and does not imply lower item quality. All items in the dataset used in this study have valid metadata such as genre, year, and number of episodes; therefore, long-tail items remain valid recommendation candidates with complete information.

7.3.4 GA Weight Settings

GA constructs the fitness function using a weighted sum. Each component is normalized to [0, 1] within the candidate pool, and the specific weights are consistent with the parameter settings in config.py.

$$\text{Fitness} = 0.4 \times \text{Relevance} + 0.3 \times \text{Diversity} + 0.2 \times \text{Novelty} + 0.1 \times \text{Freshness}$$

Relevance has a weight of 0.4, representing the degree of match between recommended items and user preferences, and is normalized using the Min-Max method. Relevance is primarily ensured by recall models such as ICF, SVD, and NCF, while HR@5 and HR@10 are used to monitor the effect of GA optimization on recommendation accuracy.

Diversity has a weight of 0.3 and is calculated according to the degree of difference among item genre labels in the recommendation list, with the aim of reducing the homogeneity of recommended content.

Novelty has a weight of 0.2. The novelty metric defined above is used, and the average novelty of the Top-10 recommended items is calculated to increase recommendation opportunities for low-popularity items.

Freshness has a weight of 0.1 and is calculated from the normalized broadcast year of each item, with the aim of increasing the weight of newer items in the recommendation results.

Diversity, novelty, and freshness are the main optimization objectives of GA, with a combined weight of 0.6; relevance is used to constrain the optimization process and prevent an excessive decline in recommendation quality.

7.3.5 Recall Engine Comparison Experiment

Table 15. HR@5 (Proportion of Positive Items Entering the Top 5)

Candidate Pool N	Random Baseline 5/N	ICF	SVD	NCF
20	25.00000%	84.88%	77.27%	**99.20%**
50	10.00000%	83.67%	72.97%	**96.43%**
100	5.00000%	81.23%	70.00%	**92.80%**
200	2.50000%	74.83%	66.07%	**80.27%**
500	1.00000%	55.77%	44.00%	**61.69%**
1000	0.50000%	41.84%	33.38%	**43.01%**
2000	0.25000%	**33.01%**	22.81%	33.45%
5000	0.10000%	**24.20%**	17.53%	16.41%
10000	0.05000%	**17.82%**	10.83%	7.65%
147097	0.03547%	**15.35%**	8.63%	5.74%

Table 16. HR@10 (Proportion of Positive Items Entering the Top 10)

Candidate Pool N	Random Baseline 10/N	ICF	SVD	NCF
20	50.00000%	85.05%	79.89%	**99.88%**
50	20.00000%	84.93%	75.66%	**98.67%**
100	10.00000%	84.00%	72.01%	**96.07%**
200	5.00000%	80.87%	70.50%	**90.94%**
500	2.00000%	70.78%	62.09%	**70.09%**
1000	1.00000%	55.61%	44.46%	**57.99%**
2000	0.50000%	**42.73%**	33.03%	46.37%
5000	0.20000%	**31.49%**	23.75%	25.80%
10000	0.10000%	**23.94%**	16.31%	15.79%
147097	0.07094%	**20.93%**	14.12%	12.26%

Table 17. Lift Relative to the Fully Random Baseline

Candidate Pool N	ICF @5	ICF @10	SVD @5	SVD @10	NCF @5	NCF @10
20	3.40×	1.70×	3.09×	1.60×	**3.97×**	2.00×
50	8.37×	4.25×	7.30×	3.78×	**9.64×**	4.93×
100	16.25×	8.40×	14.00×	7.20×	**18.56×**	9.61×
200	29.93×	16.17×	26.43×	14.10×	**32.11×**	18.19×
500	55.76×	35.39×	44.00×	31.04×	**61.69×**	35.05×
1000	83.67×	55.61×	66.77×	44.46×	**86.02×**	57.99×
2000	132.03×	85.47×	91.24×	66.07×	**133.78×**	92.73×
5000	**241.95×**	157.42×	175.27×	118.74×	164.12×	129.01×
10000	**356.50×**	239.40×	216.60×	163.12×	153.05×	157.90×
147097	**432.91×**	294.97×	243.45×	199.12×	161.83×	172.83×

All 30 experimental combinations (3 recommendation engines × 10 candidate-pool sizes) perform substantially better than the random baseline. The lift for HR@5 ranges from 3.09× to 432.91×, while the lift for HR@10 ranges from 1.60× to 294.97×.

The performance of the recommendation engines varies across candidate-pool sizes. When the candidate pool is relatively small (≤ 2000), NCF performs better overall; when the candidate-pool

size is relatively large (≥ 5000), ICF performs better. Overall, ICF achieves the best result in 3 settings, NCF in 7 settings, while SVD does not achieve the best performance in any setting.

7.3.6 Comparison Experiment of the Rerank Layer

To verify the optimization effect of GA and exclude the influence of random reranking, this study compares four recommendation strategies: Greedy baseline, Window Random, Pool Random, and GA.

The Greedy baseline directly ranks items by relevance and selects the Top-10 from the 20 candidates with the highest relevance, without reranking. It serves as the baseline for the original recommendation results.

Window Random uses the same Top-20 candidate set as GA but randomly selects 10 items from it. This control is used to exclude the effect of random selection and thereby verify whether GA can obtain results superior to random reranking through fitness optimization.

Pool Random randomly selects 10 items from the complete candidate pool of 14,098 items without considering relevance. This method provides a reference under random recommendation conditions and reflects exposure characteristics close to the full-catalog distribution.

The GA method is likewise based on the relevance Top-20 candidate set and selects the final Top-10 recommendation results using a fitness function containing 0.4 relevance, 0.3 diversity, 0.2 novelty, and 0.1 freshness. Comparison with the above control methods evaluates GA's ability to optimize recommendation-list structure while maintaining recommendation relevance.

Table 18. Cost Side: Hit Rate HR@10

Candidate Pool N	Winning Engine	Greedy	Window Random	Pool Random	GA	GA ÷ Window Random	GA ÷ Pool Random
5000	ICF	31.49%	20.14%	0.21%	29.73%	1.48×	142×
10000	ICF	23.94%	15.65%	0.12%	22.37%	1.43×	183×
147097	ICF	20.93%	13.67%	0.05%	19.51%	1.43×	390×

Table 19. Gain Side: Long-Tail Exposure LTE@10 (items)

Candidate Pool N	Greedy	Window Random	Pool Random	GA	LTR@10 Greedy	Window Random	Pool Random	GA
5000	2,258	2,820	101,170	4,494	3.53%	5.04%	99.90%	**6.39%**
10000	1,858	2,171	100,550	3,457	2.65%	3.45%	99.91%	**4.14%**
147097	1,829	2,018	101,067	3,161	2.33%	2.92%	99.92%	**3.45%**

7.3.7 Gains and Costs of the GA Algorithm

Table 20 compares the experimental results of the ICF baseline and ICF + GA under the same user conditions.

Table 20. Experimental results of the ICF baseline and ICF + GA under the same user conditions

Metric	Category	CF BaselineI	ICF + GA	Change
LTR@10 Long-Tail Reach Rate	Gain	2.33%	3.45%	+222 users (+47.5%)
LTE@10 Long-Tail Exposure	Gain	1,829 items	3,161 items	+1,332 items (+72.8%)
GOS@10 GA Objective Score	Gain	0.2799	0.2923	+0.0124 (+4.4%)
HR@10	Cost	20.93%	19.51%	-1.42pp (-6.8%)
HR@5	Cost	15.35%	14.53%	-0.83%

GA substantially improves the long-tail exposure structure of the recommendation results. LTR@10 increases by 47.5%, adding 222 users reached by long-tail items; LTE@10 increases by 72.8%, adding 1,332 long-tail exposures; and GOS@10 increases by 4.4%. Correspondingly, HR@10 and HR@5 decrease by 1.42 and 0.83 percentage points, respectively. The smaller decrease in HR@5 indicates that GA has a relatively limited impact on the top portion of the recommendation list.

Gain metrics are tested using the Wilcoxon signed-rank test, while hit rates are tested using the McNemar test. The difference in GOS@10 is statistically significant ($p < 1e\text{-}300$).

Under the full-catalog setting, each 1-percentage-point decrease in HR@10 corresponds to approximately a 51% increase in long-tail exposure. For 20,000 users, GA trades the loss of hits for approximately 283 users for 1,332 additional long-tail exposures and 222 additional users reached by long-tail items.

From a per-user perspective, the average number of long-tail items in each user's Top-10 recommendation list increases from 0.091 to 0.158, indicating that GA can effectively increase exposure opportunities for long-tail items.

7.3.8 Conclusion

The experimental results show that the recall and GA reranking mechanisms constructed in this study can respectively perform candidate screening and recommendation-list optimization. During the recall stage, ICF, SVD, and NCF all clearly outperform the random baseline across all candidate-pool sizes, with maximum lifts of 432.91× and 294.97× over the random baseline for HR@5 and HR@10, respectively. NCF performs better overall for small candidate pools, whereas ICF performs better when the candidate-pool size reaches 5000 or above, indicating that different recall models are suitable for different candidate scales.

During the reranking stage, GA improves the long-tail exposure structure of the recommendation list while maintaining relatively high recommendation accuracy. Under the full-catalog setting, LTE@10 increases from 1,829 to 3,161, a 72.8% increase; LTR@10 rises from 2.33% to 3.45%, adding 222 users reached by long-tail items; and GOS@10 increases by 4.4%. Correspondingly, HR@10 decreases from 20.93% to 19.51%, a decline of 1.42 percentage points, while HR@5 decreases by 0.83 percentage points. The results show that GA can increase long-tail item exposure and user coverage at a certain cost in accuracy.

The random-control experiment further verifies the effectiveness of GA reranking. Under the full-catalog setting, GA and Window Random use exactly the same Top-20 candidate set, but GA achieves an HR@10 of 19.51%, clearly higher than Window Random's 13.67% and approximately 1.43 times as high. At the same time, GA increases long-tail exposure from 2,018 under Window Random to 3,161. Therefore, GA's performance gains mainly come from multi-objective selection driven by the fitness function rather than random reranking.

7.4 Algorithm 4: Comparing Template, LLM-Only and Retrieval-Augmented Routes for Chat-to-Find Anime

Key finding: On 80 test queries and six LLMs, we compared three routes for producing the reply of the chat page. Route R, retrieval-augmented generation, beats the LLM alone (Route N) for every model on all three headline measures. The judge score rises by 0.41 to 0.67 points, from a mean of 3.50 to 4.05 on a 5-point scale, while the template reply (Route T) scores 2.44; the stated episode matches the knowledge base in 86% of the R replies against 39% of the N replies; and 94–100% of the R replies name only knowledge-base titles against 20–38% for N. R is therefore chosen as the route for Algorithm 4. GLM-5.3-Flash, GLM-5.3 and Kimi-K3 form the top tier of the six models and are the shortlist, and the template reply is kept only as an offline fallback. One boundary applies: single-fact questions that name a title are answered worse with retrieval than without it (Section 7.4.5).

7.4.1 Task Definition and Experimental Setup

The chat page (Page 4) lets the user describe what to watch in free text, for example “12 episodes or fewer, score above 8.4, comedy” or “which episode has L and Light talking through the glass”. We compared three routes for producing the reply.

- Route T — template reply of the first prototype. Lexical retrieval over the knowledge base and a fixed reply text; no LLM.
- Route N — LLM only. The question goes to an LLM with a one-line role description and nothing else.
- Route R — retrieval-augmented generation. The retrieval step selects entries from the knowledge base, the entries are placed in the prompt, and the model is told to answer from them.

The comparison answers two questions: should the chat page retrieve at all, and which LLM should sit behind it.

System under test. We froze the Page 4 pipeline of AnimeFinder v1.4.1 (rag.py: lexical scoring, scene-intent threshold 0.34, constraint extraction and prompt assembly) and checked the frozen copy function by function against the repository (12 of 12 identical). Its knowledge base has 24 well-known titles, 137 aliases and 12 curated scenes. The chat module of the final system (AniLogic) is not covered by the numbers below; the protocol carries over to it, the numbers do not.

Queries. We wrote 80 test queries, 74 in Chinese and 6 in English, in the six types of **Table 21**. Each scene question exists in a plain version and in a paraphrased version that avoids the keywords of the stored description.

Table 21. The 80 test queries

Type	Queries	A good reply
Scene identification	24 (12 plain, 12 paraphrased)	Names the right title, with the episode as a bonus, and no wrong title
Constraint filtering	16	Recommends only titles that meet every limit on episodes, score and genre
False premise or unknown title	12	Says the title cannot be found instead of inventing a plot
Similar titles	10	Suggests close titles from the knowledge base, not the reference itself
Mood	10	Suggests fitting titles from the knowledge base and gives reasons
Single fact	8	States the studio, year or episode count that was asked for

Models and runs. Six LLMs answered every query with retrieval (R) and without it (N): Hy3, DeepSeek-V4.1-Flash, GLM-5.3, GLM-5.3-Flash, MiniMax-M3 and Kimi-K3. That makes 960 generation calls and 240 judge calls, roughly 6.4 million tokens in all, with one run per cell. T was produced by running the prototype's own template code on all 80 queries.

7.4.2 Evaluation Metrics

Table 22 lists the measures. Rule-based measures are computed against the knowledge base; the judge score comes from three LLM judges.

Table 22. Evaluation measures

Measure	Definition	Interpretation
Judge score	Mean of relevance, faithfulness, usefulness and tone (1–5) over the judges; a judge never scores a model of its own vendor	Primary quality measure
Centered judge score	Judge score minus that judge's overall mean	Used to rank the models
Episode accuracy	Share of scene replies that state the knowledge-base episode of the identified title (22 questions with a checkable episode)	Rule-based; independent of the judges
In-KB citation rate	Share of replies that name at least one title and no title outside the knowledge base	Scope compliance; T meets it by construction
Lift (R − N)	Paired per-question difference between R and N, with a 95% bootstrap interval	Effect of retrieval
Hit@1, Hit@3	The right scene entry is the first, or among the first three, retrieved	Retrieval layer only
p95 time, replies > 20 s	Response time of the R group through the command-line channel	Relative cost between models
Net input tokens	Input tokens minus the fixed overhead of the channel, estimated by regression	Cost of retrieval

Three LLM judges (Hy3, DeepSeek-V4.1-Flash and GLM-5.3) scored 13 candidate replies per question, T plus six models each with and without retrieval. The order of the candidates was shuffled for every judge. The judges differ in strictness (mean scores 3.57, 3.68 and 3.82), so each judge's scores were centered on that judge's own mean before models were ranked. Intervals are 95% bootstrap intervals over questions (10,000 resamples, seed 42), and differences between R and N are paired per question.

7.4.3 Overall Comparison of T, N and R

Retrieval raises the judge score of every model, by 0.41 to 0.67 points on the 5-point scale (all six 95% intervals lie above zero), from a mean of 3.50 for N to 4.05 for R. The template replies of the prototype scored 2.44 on the same scale, judged on the reply text only. On scene questions the stated episode matched the knowledge base in 86% of the R replies against 39% of the N replies, and the share of replies that cite only knowledge-base titles rose from 0.20–0.38 to 0.94–1.00. **Figure 10** shows the three measures for each model and **Table 23** compares the three routes.

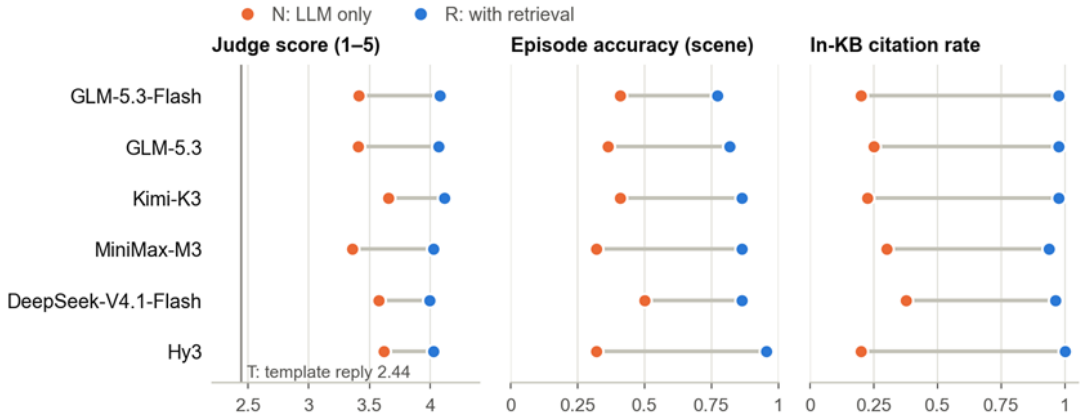


Figure 10. N and R for each model on three measures, ordered by centered judge score. The gray line marks the template reply T. Intervals for the judge-score lift are in Table 7.4-4.

Retrieval itself put the right scene entry first for 21 of the 24 scene questions (87.5%) and among the first three for 22 (91.7%). It costs about 400 extra input tokens per question (+383 to +406 for five of the six models; the estimate for MiniMax-M3 is unreliable). Output length fell for all six models, but mostly because the models reasoned less, and as N and R use different prompts we do not read this as a saving.

Table 23. T, N and R on the 80 queries. A value followed by brackets is a mean over the six models, with the range over models in the brackets.

Measure	T template	N LLM only	R retrieval + LLM
Judge score, 1–5	2.44	3.50 (3.36–3.65)	4.05 (3.99–4.12)
Episode accuracy, scene questions	–	0.39 (0.32–0.50)	0.86 (0.77–0.95)
Replies citing only knowledge-base titles	1.00	0.20–0.38	0.94–1.00
Extra input tokens per question	0	0	about +400
LLM required	no	yes	yes

The two rule-based measures do not depend on any judge, and both favor R over N by a wide margin: episode accuracy rises from 0.39 to 0.86 (pooled lift +0.47, 95% CI [+0.30, +0.63], over 22 questions and six models) and is still +0.43 [+0.24, +0.61] when the four queries that rely on two scene entries that may not match the broadcast episodes are left out. Part of the judge gain is built in, because the faithfulness dimension rewards replies that use the supplied entries; with that dimension removed the gain is still significant for three of the six models, and after controlling for reply length for four.

7.4.4 Model Comparison and Significance

Table 24 compares the six models behind R. Their raw judge scores lie in a narrow band (3.99–4.12) and change order once each judge is centered, so we rank on the centered score. The ranking

has two tiers. GLM-5.3-Flash, GLM-5.3 and Kimi-K3 are each significantly above MiniMax-M3, DeepSeek-V4.1-Flash and Hy3: 9 of the 15 pairwise gaps are significant, and they are exactly the nine that cross the tiers, while no gap inside a tier is significant (no correction for multiple comparisons was applied). **Figure 11** adds response time.

Table 24. The six models behind R, in order of centered judge score

Model	Judge score, R (raw)	Centered score	Lift R − N [95% CI]	p95 (s)	Replies > 20 s	Output tokens per reply
1. GLM-5.3-Flash	4.08	0.45	+0.67 [0.41, 0.91]	20.5	6/80	313
2. GLM-5.3	4.07	0.44	+0.66 [0.40, 0.92]	15.6	2/80	305
3. Kimi-K3	4.12	0.43	+0.47 [0.21, 0.70]	19.7	3/80	277
4. MiniMax-M3	4.02	0.33	+0.67 [0.39, 0.93]	32.4	12/80	528
5. DeepSeek-V4.1-Flash	3.99	0.30	+0.42 [0.10, 0.73]	8.1	0/80	628
6. Hy3	4.03	0.28	+0.41 [0.12, 0.68]	19.1	4/80	790

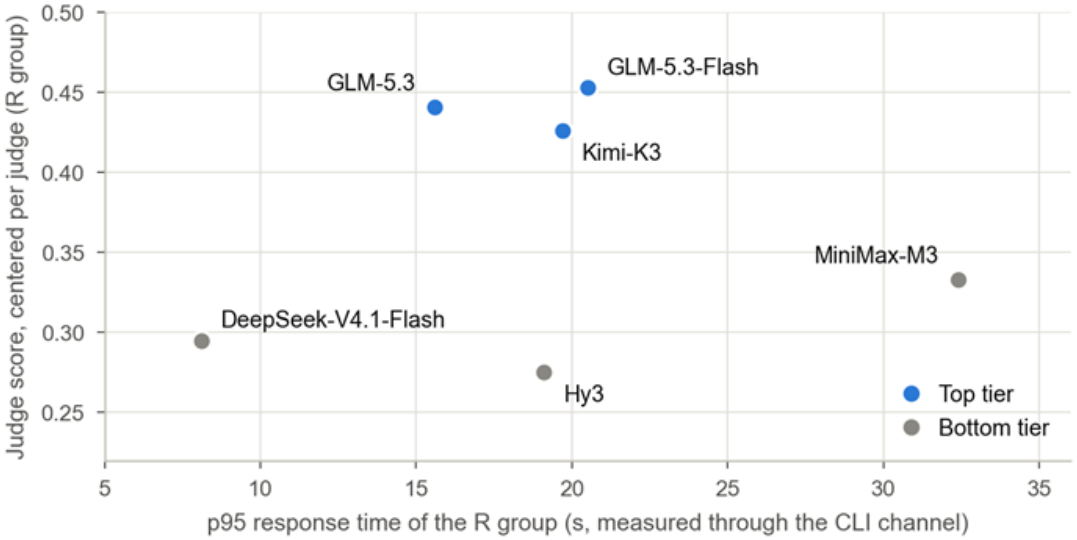


Figure 11. Centered judge score against the 95th-percentile response time of the R group. The two tiers are the groups that differ significantly from each other; response times were measured through the command-line channel and are comparable between models only.

7.4.5 Boundary of the Decision: Single-Fact Questions

Single-fact questions that name a title (8 of the 80 queries) are the exception to the result above: R answered 19 of 48 correctly against 48 of 48 for N. The cause lies in the ranking step, not in generation. For queries that read like a request for recommendations, rag.py moves the title named in the query to the end of the candidate list (lines 146–153) so that it is not recommended back, and the same rule also fires on fact and scene questions. The named title was missing from the prompt in 5 of the 8 fact questions, and every one of the 29 replies that were right without retrieval and wrong with it had its title missing. The final chat module resolves named titles in a separate step (alias, prefix and fuzzy matching) before retrieval, but we have not repeated this fact-lookup comparison on it.

7.4.6 Scope, Validity and Reproducibility

Scope. These numbers describe the prototype pipeline with its 24-title knowledge base. The chat module of the final system and the 14,202-title corpus described in Chapter 6 have not been measured with this protocol. The model answers and the judgments came through a command-line

channel instead of vendor API keys, so latencies are comparable between models but are not end-to-end figures, and temperature and token limits could not be set. Each cell is a single run. The judges are models and no human ratings were collected; the rank correlation between judges within a question is only 0.54–0.73. The knowledge base holds famous titles that the models already know, which mutes the measurable benefit of retrieval for lesser-known titles.

Reproducibility. The R and N prompts rebuilt from the frozen code are byte-identical to the 160 prompts that were sent, the template code reproduces all 80 T replies, and re-aggregating the saved judge outputs reproduces all 960 per-reply scores. The frozen pipeline, the queries, the saved outputs, the scorers and 100 unit tests are in algorithm4_benchmark/ of the team repository.

7.4.7 Conclusion

On this 80-query benchmark, Route R scores higher than Route N for all six models on the judge score, on episode accuracy and on the in-KB citation rate, and higher than the template route T on the judge score (4.05 against 2.44). R is selected as the route for Algorithm 4. The template reply stays in the system only as the offline fallback for installations without an LLM key.

Among the models, GLM-5.3-Flash and GLM-5.3 are the quality-first choice (centered scores 0.45 and 0.44). Kimi-K3 is a steady alternative: it has the shortest replies (277 output tokens on average) and only 3 of 80 replies took longer than 20 seconds. Because the three top models are statistically tied, price and availability can decide among them. DeepSeek-V4.1-Flash is the option when response time matters more than quality (p95 8.1 s, no reply over 20 s). We would not use MiniMax-M3 for real-time chat, where 12 of 80 replies took longer than 20 seconds, and Hy3 gives no quality advantage over the other bottom-tier models while producing the longest replies (790 output tokens).

The chat module of the final system is built on the same principle. It retrieves first (full-text and dense search, fused by reciprocal rank fusion), composes the reply only from retrieved evidence and verified facts, and checks the item keys, cited documents and numbers of an LLM reply before showing it; without an LLM it falls back to a deterministic template over the same evidence. Which model sits behind it remains a configuration item, and the evaluation package can be reused to repeat the comparison on that module.

8 System Design

AniLogic keeps the architecture of the proposal, an HTML frontend, a FastAPI backend and SQLite, and runs it as one process that serves four reasoning modules. Heavy work such as cleaning 24 million ratings and training models runs offline. A request only looks things up and runs light computations, and every hard rule goes through one shared rule engine.

8.1 Architecture Overview

Figure 12 extends the architecture slide of the proposal presentation. Pages are thin: each page is one HTML template plus one script, and all data comes from a versioned JSON API. The backend checks every request (schema, session and CSRF token), and the services apply ownership checks, caching and run records before calling a reasoning module. The modules read an in-memory serving state (catalogue arrays, trained models and the vector index) and the SQLite database. An LLM is optional and sits outside the core path.

Offline, the pipeline turns the Kaggle snapshot and the team data package (Chapter 6) into a versioned bundle of data, models and indexes, each file recorded with its SHA-256 hash.

Activation loads the bundle into SQLite in one transaction and only then switches the active version.

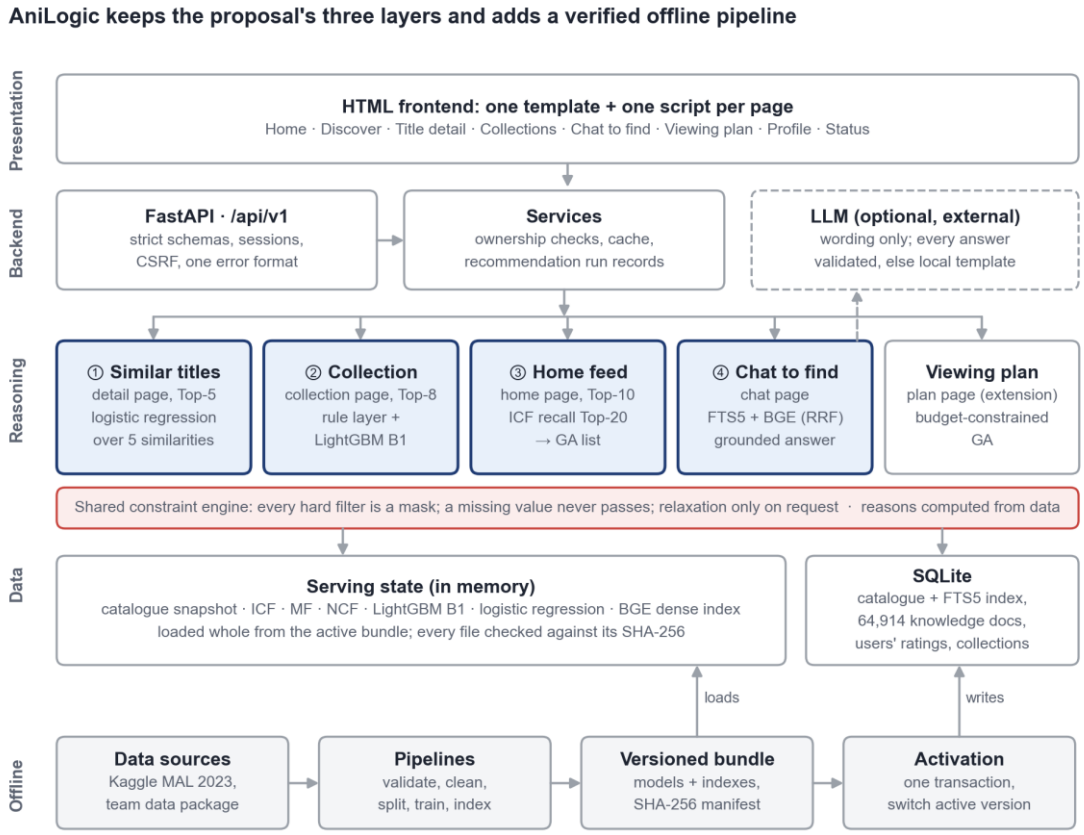


Figure 12. System architecture: presentation, backend, reasoning and data layers, fed by the offline pipeline.

8.2 Reasoning Modules

Each page has its own pipeline (**Figure 13**), following the fixed-and-replaceable design of the proposal presentation. Fixed steps carry the product rules; replaceable steps were compared in Chapter 7 and chosen by a rule written before the result was known.

- **Similar titles** (detail page): five attribute similarities (genre, theme, demographic, studio, length). Logistic regression over these features (B) is served; the weighted formula (A) is the fallback.
- **Collection** (collection page): preference profile, rule layer and the LightGBM scorer B1 (Section 7.2).
- **Home feed**: a cold-start gate (cold, partial, warm at eight ratings), ICF recall of 20 candidates, and a GA that picks ten for relevance, diversity, novelty and freshness (weights 0.4, 0.3, 0.2, 0.1).
- **Chat to find**: deterministic intent parsing, full-text and dense retrieval fused by reciprocal rank fusion (RRF), and an answer built only from retrieved evidence; an LLM may word it after validation.
- **Viewing plan** (extension): a GA that selects titles within a time budget and schedules prequels first.

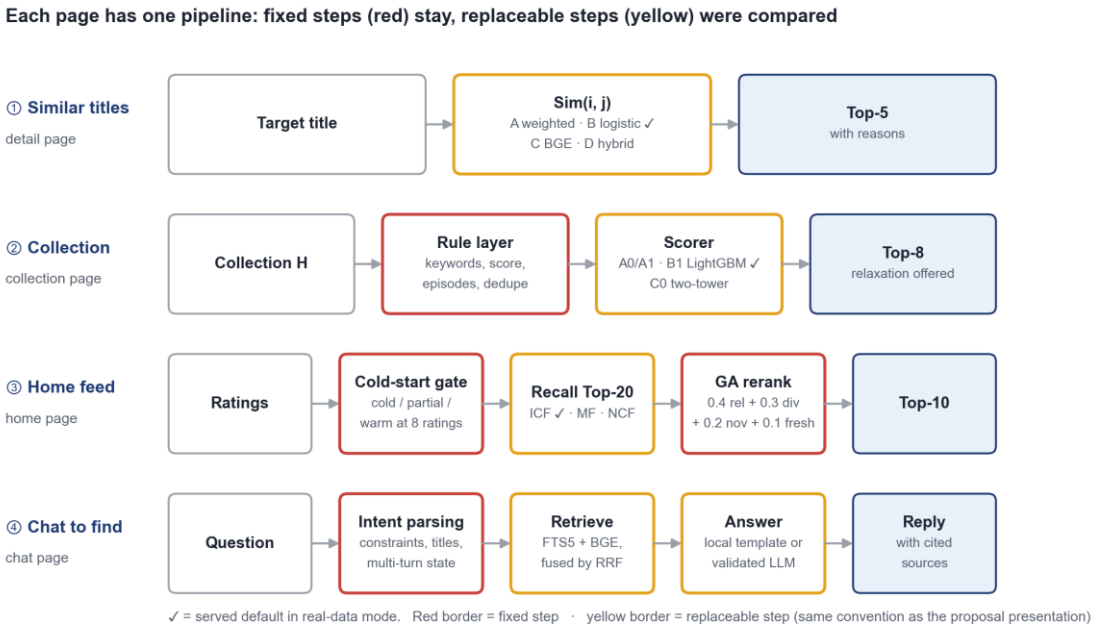


Figure 13. The four reasoning pipelines. Red: fixed steps; yellow: replaceable steps; ✓: served default.

8.3 Shared Components

Three components are shared by all modules. The **rule engine** turns every hard filter into a mask over the catalogue, so search, collections, the home feed, chat and the planner apply a filter in the same way; a missing value never passes, and a filter is relaxed only when the user picks a suggested relaxation. The **explanation layer** attaches reasons computed from data, such as shared tags, the user's own supporting ratings or what a title adds to the list; an LLM never writes them. The **knowledge base** holds 64,914 documents (attributes, synopses, relations, characters and 300 sourced scenes), indexed for full-text search in SQLite and for dense search with BGE embeddings.

8.4 One Request, End to End

Figure 14 follows one collection request. The API checks the session and schema; the service reads the collection (only its owner may), builds the profile, lets the rule engine select eligible titles and asks B1 to score them. The reply is either eight titles with reasons or, when no title passes, the rule that excluded everything together with relaxation options. Every run is recorded and cached under the user's profile version, which any new rating or collection change increments, so a stale list is never served.

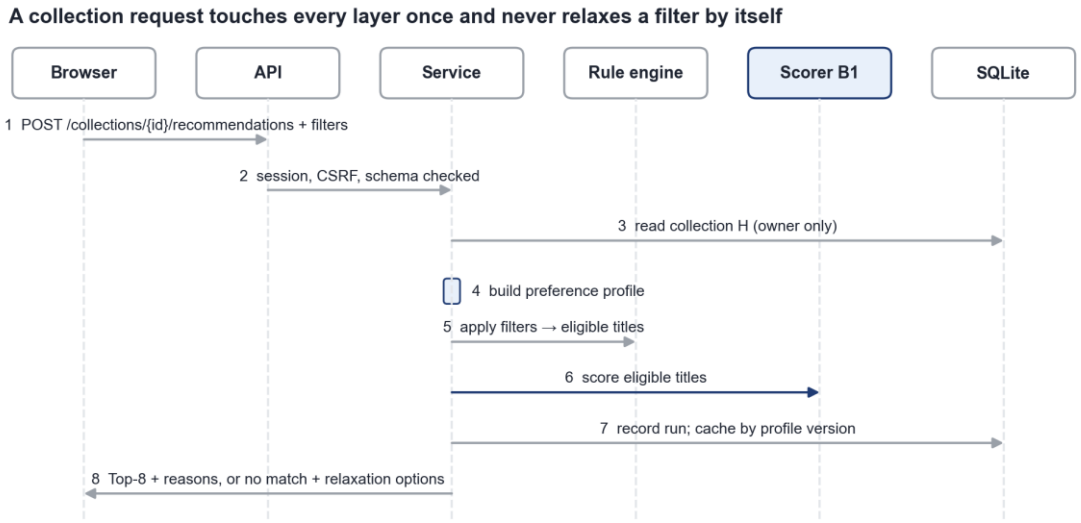


Figure 14. Sequence of a collection recommendation request.

8.5 Reliability and Design Trade-offs

Models, indexes and ID mappings are only valid together, so each request uses one consistent serving state, and a failed check degrades one capability instead of the site. For example, a changed Algorithm 2 model makes the collection page fall back to the formula, and a failed dense encoder leaves chat on full-text search; the status page shows the state of each capability. **Table 25** lists the main design decisions.

Table 25. Main design decisions

Decision	Choice	Reason
Structure	One FastAPI process with strict module boundaries	One team and one machine; boundaries keep each algorithm testable
Storage	SQLite for user data, Parquet for the 24 M ratings	Small, mostly single-writer data; 90 concurrent writes gave 0 failures
Similar titles	B (logistic regression) served, A as fallback	Passed the proposal's replacement rule on MyAnimeList recommendations (validation Hits@5 0.167 vs 0.049) with A's own interpretable features
Collection	B1 served, formula as fallback	Confirmed on test by the frozen protocol (Section 7.2)
Home feed	ICF recall + GA list	Best personalised engine, serves new users without retraining; GA adds diversity for 1.5 pp Hits@10
Chat	Retrieval first, LLM optional	RAG beat LLM-only for all six models (Section 7.4); works without credentials

Section 7.1's synthetic-oracle benchmark retains route A for similar titles; the served real-mode choice follows the proposal's replacement rule on the MyAnimeList recommendation labels, and A remains the fallback.

9 Evaluation

Following the evaluation plan of the proposal presentation, we evaluated the integrated system at two levels. The **task level** asks whether each page does its job (Hits@5, MRR@10 and task-success rate). The **system level** covers efficiency and cost, robustness and reliability, and explainability and human factors. Chapter 7 chose each algorithm in its own benchmark; this chapter checks them together, through the code the website runs.

9.1 Evaluation Design

Figure 15 maps the plan to the evidence we used: labelled queries, held-out ratings, user tasks and failure scenarios. All runs used real-data version real-034ebd6a74 (23,363 served titles), worked on throwaway copies of the database and ran chat in local mode, so no paid model was called. Every run stores its configuration, dataset version, per-case rows and report.

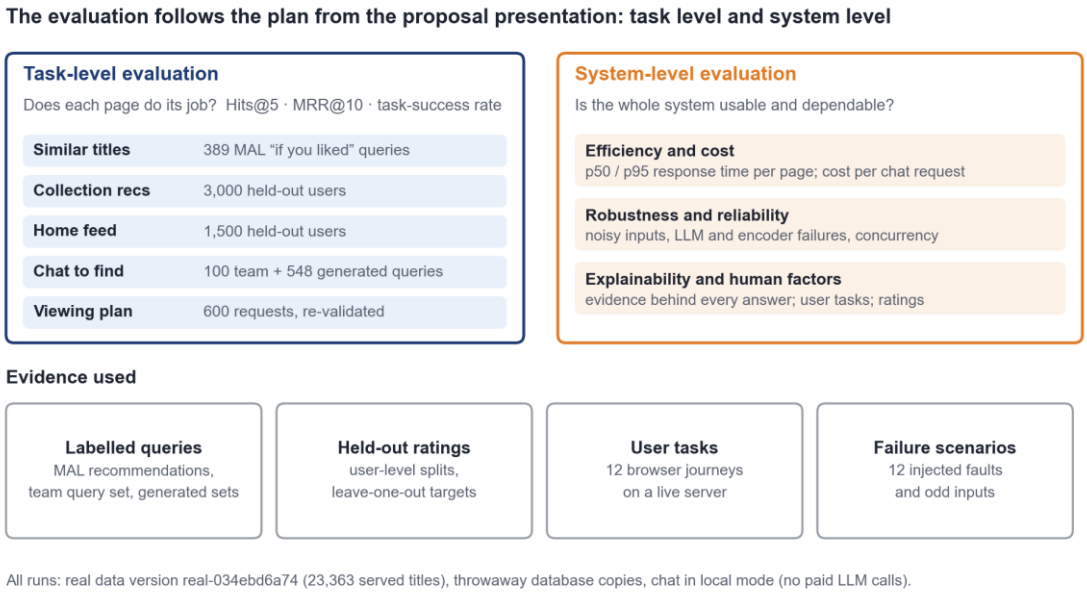


Figure 15. Evaluation framework, following the plan in the proposal presentation.

9.2 Task-level Evaluation

Table 26. Task-level results in the served configuration (baseline in brackets)

Task (test set)	Hits@5	MRR@10	Task success
Similar titles (234 MAL test queries)	0.197 (A: 0.081)	NDCG@5 0.052 (A: 0.019)	–
Collection (3,000 held-out users)	0.100 (A0: 0.015)	0.055 (A0: 0.008)	0 rule violations
Home feed (1,500 held-out users)	0.040 (greedy: 0.047)	0.029 (greedy: 0.032)	GA objective +0.021
Chat (100 team queries)	1.00	0.93–0.98	100 / 100
Chat (548 generated queries)	0.97–1.00	–	0.97–1.00 per set
Viewing plan (600 requests)	–	–	0 infeasible plans

Hits@5 for the recommendation tasks is low in absolute terms because each user has one hidden title among about 14,000, where a random Top-5 hits 0.04% of the time; the gains over the baselines are what matter. On the home feed a popularity list reaches Hits@10 0.137 against 0.062 for ICF, because most held-out titles are popular, but it covers about 30 times less of the catalogue, so the home page shows a separate, labelled popular list. The GA gives up 1.5 pp Hits@10 for clearly more diverse, novel and fresh lists, the same trade-off reported in Section 7.3.

Chat answered all 100 queries of the team's set: 40 scene, 30 semantic, 20 constraint and 10 unanswerable. The scene records of the data package make the difference for scene questions (**Figure 16**): with synopses alone the right title is in the Top-5 for 43% of them.

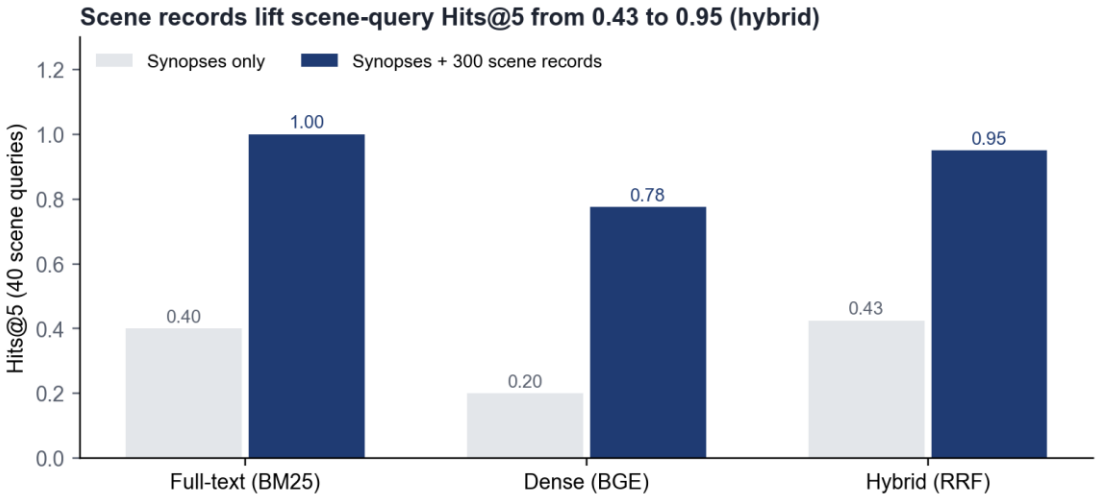


Figure 16. Scene queries: retrieval with and without the 300 scene records.

9.3 Efficiency and Cost

Every request answered within 0.35 s at p95 on one laptop (**Figure 17**). The collection page is the slowest recommendation call because B1 scores the full catalogue, and the planner runs a GA with a 1.5 s budget. Five concurrent clients saw a p95 of 45.8 ms with no errors. In local mode a chat request costs no paid call; with an LLM it makes at most two calls (30 s timeout), and retrieval adds about 400 input tokens per question (Section 7.4). The full offline pipeline takes about 16 minutes, and the server needs about 1.1 GB of memory with all models loaded.

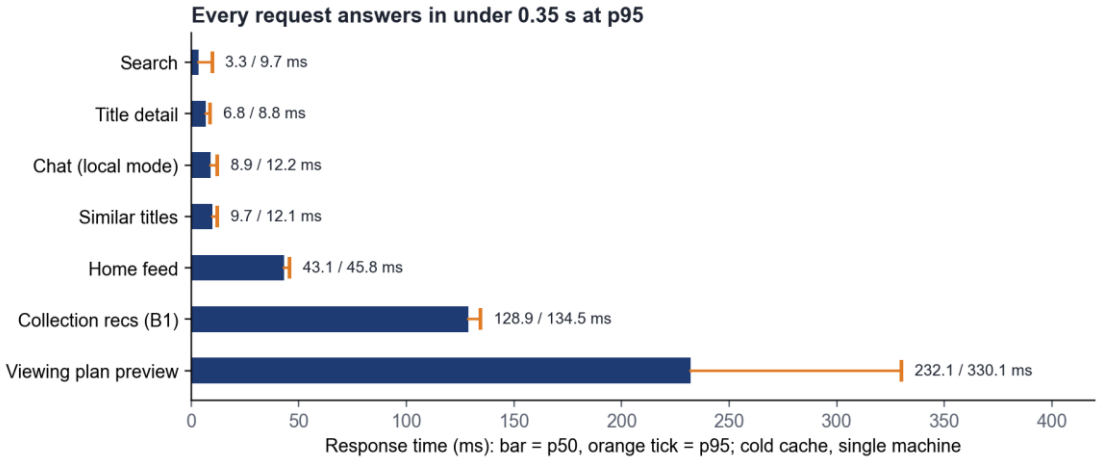


Figure 17. Response time per endpoint (p50 and p95, cold cache, 30 calls each).

9.4 Robustness and Reliability

All 12 injected failures and unusual inputs were handled without an error page:

- **LLM failures** (timeout, rate limit, invalid JSON, empty reply, refusal): 5 of 5 fall back to the local answer with a warning.
- **Dense encoder failure**: chat continues with full-text search and says so.
- **Noisy input**: a 1,000-character message is handled and an over-long one rejected (HTTP 422); HTML is treated as text; contradictory filters trigger a clarifying question; impossible filters return the reason instead of an error.
- **Concurrency**: 90 simultaneous rating writes, 0 failures.

- **Tampered or mismatched model files**: refused, and only the affected capability degrades.

Underneath, 185 automated tests pass on the final version: 177 unit and integration tests, including real-data checks, and 8 browser tests that cover 12 user journeys; the 21 Algorithm 2 checks pass as well.

9.5 Explainability and Human Factors

Every recommendation card shows reasons computed from data, and every chat answer lists its sources. In local mode, chat gave 0 fabricated episode numbers for 30 unanswerable episode questions and 0 false "verified scene" claims across 100 questions designed to trigger one. In Section 7.4's LLM comparison, retrieval raised the share of replies citing only knowledge-base titles from 0.20–0.38 to 0.94–1.00.

For user tasks, the 12 browser journeys (new user, ratings from cold to warm, strict collection filters and relaxation, chat refinement, a 15-hour plan with edits and export, data isolation) all succeed on a live server and found three defects that were fixed. The user helpfulness ratings planned in the presentation were not collected: the protocol is written but no user study has run, so human factors are only partly evaluated.

9.6 Summary

At task level every page meets its checks on real data, with zero rule violations, zero infeasible plans and zero fabricated episodes. At system level the system answers interactively and degrades safely. The open gaps are human judgement, LLM answers in the final chat module and real collection data.

10 Implementation

The complete system runs on the full real dataset on one laptop: 14 pages, a versioned JSON API, an offline pipeline from raw files to an activated database, and an evaluation framework.

10.1 Technology Stack and Code Structure

The stack follows the proposal: an HTML frontend (Jinja page shells and plain JavaScript modules), a FastAPI backend in Python 3.12 and a SQLite database (Alembic migrations, FTS5 full-text index). We added Parquet for the offline data, NumPy and SciPy for ICF and the GA, LightGBM for Algorithm 2, PyTorch for MF, NCF and the two-tower model, and BGE-small for dense retrieval. The code has about 24,700 lines of Python in five folders: app/ (website and algorithms), pipelines/ (offline jobs), evaluation/ (experiments), algorithm2_benchmark/ and tests/, plus about 1,900 lines of templates, scripts and styles. Tests use pytest and Playwright.

10.2 Delivered Scope

Figure 18 compares the delivery with the MVP and additional features of the proposal (**Table A-1**). Every MVP feature is delivered and verified. Three extensions are limited by data: relations cover 10,883 of 23,363 titles, scenes cover 100 titles, and adaptive feedback logs events and applies exclusions without retraining. Screenshot identification is built but switched off because no frame images exist.

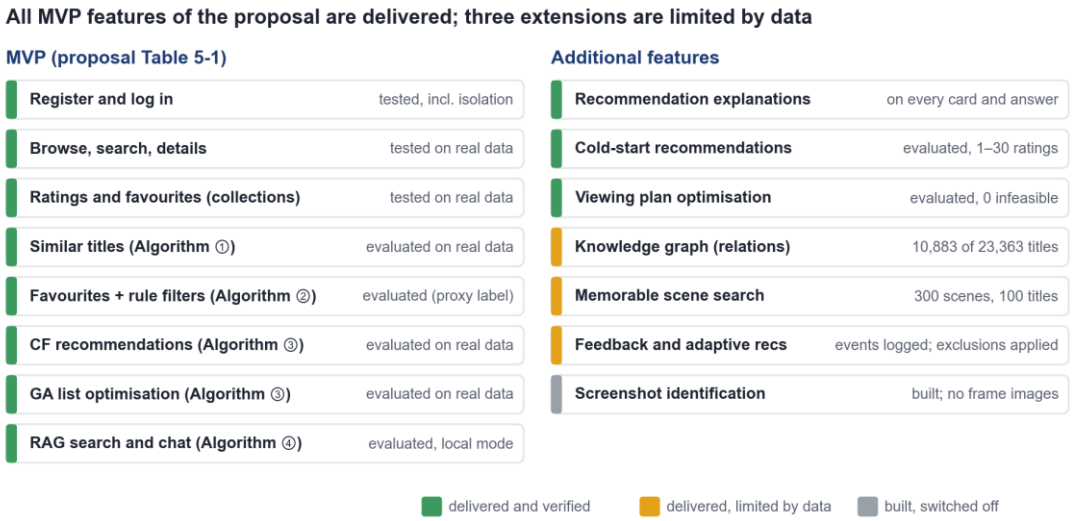


Figure 18. Proposal scope against the delivered system.

10.3 System Walkthrough

The screenshots below come from the real-data site (version real-034ebd6a74) for a test account that rated ten titles and saved five psychological thrillers to a collection.

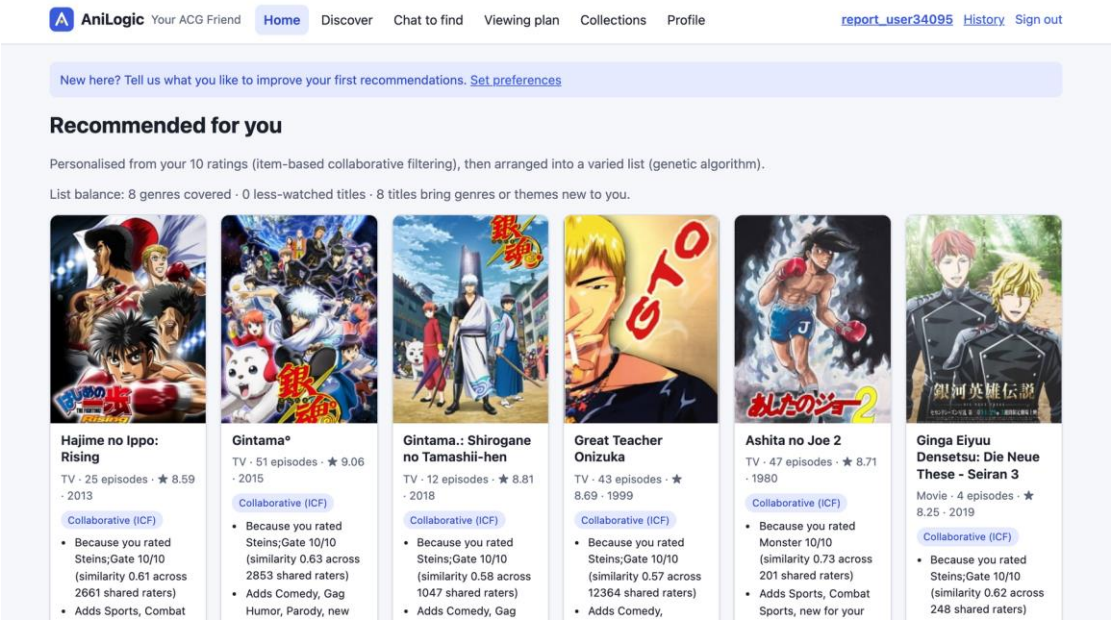


Figure 19. Home page: ten titles from ICF recall arranged by the GA, each with the rating that supports it and what it adds to the list.

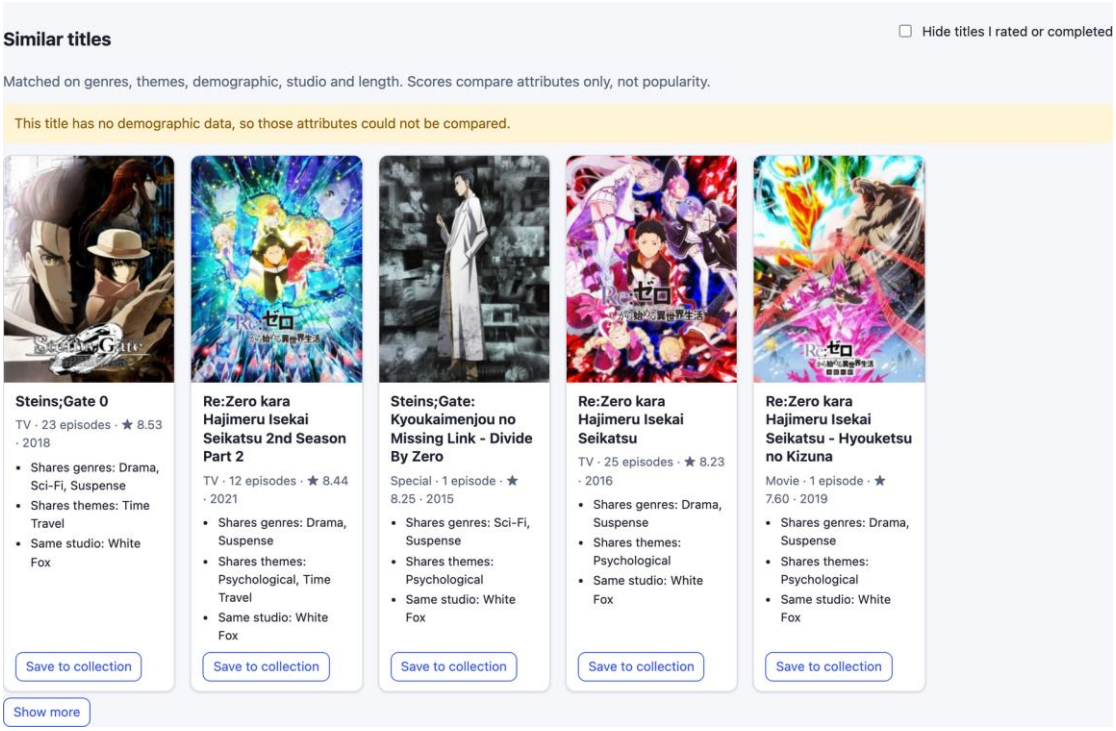


Figure 20. Similar titles on the Steins;Gate detail page, with shared genres, themes and studio as reasons and a note on missing data.

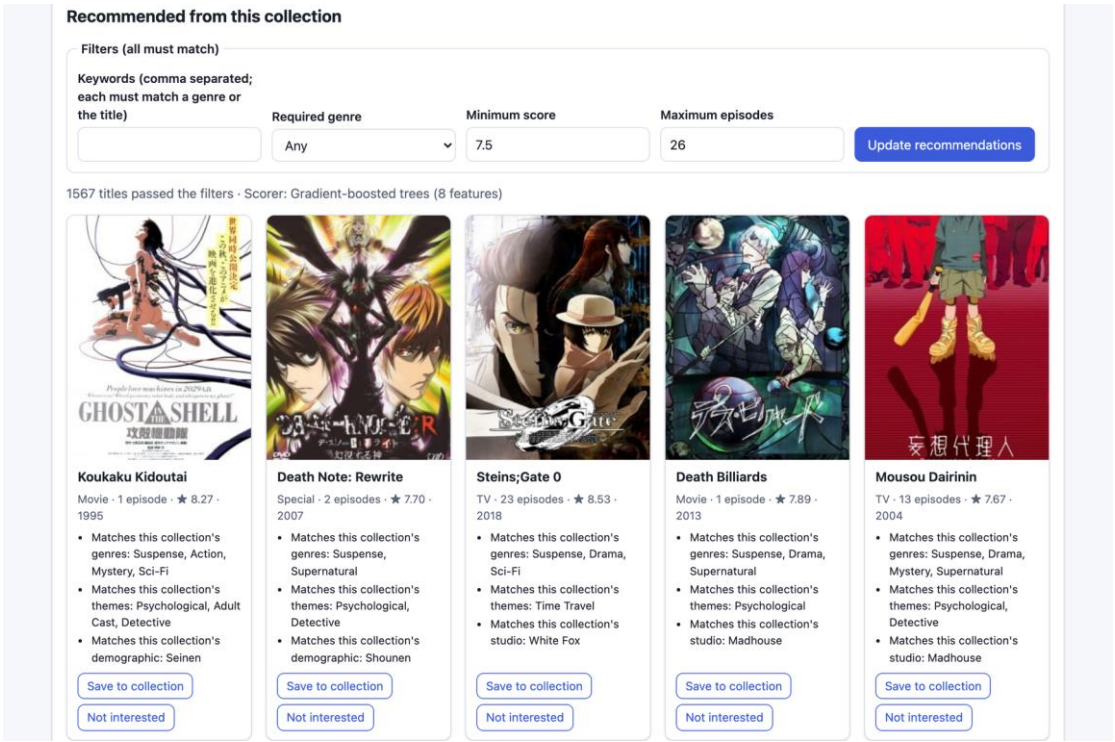


Figure 21. Collection page: strict filters (score ≥ 7.5, at most 26 episodes) and the Top-8 from scorer B1.

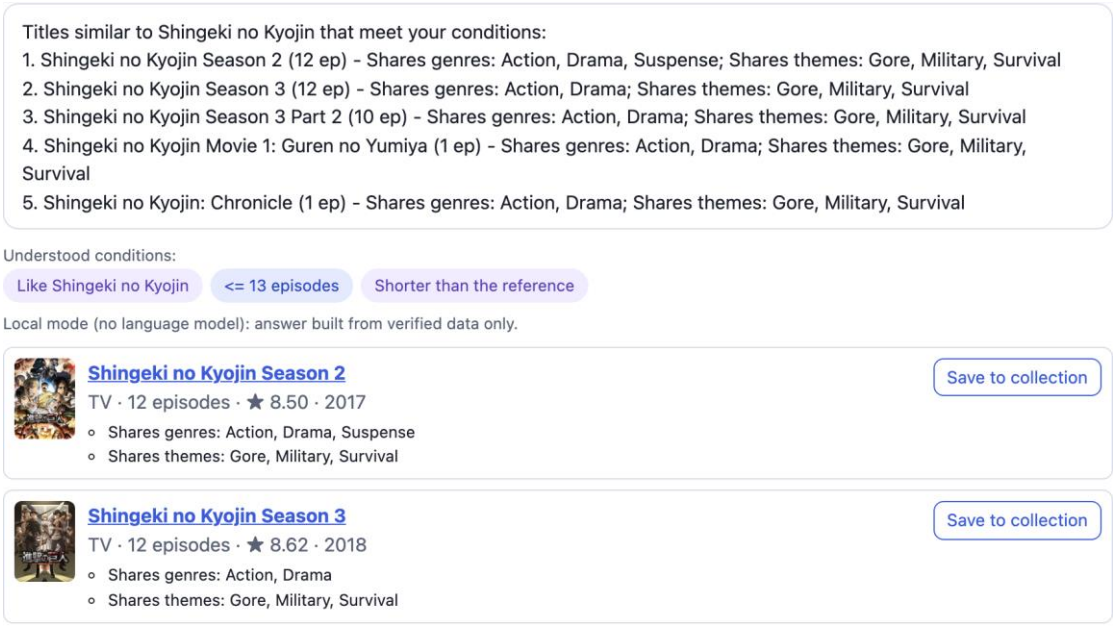


Figure 22. Chat to find: "something like Attack on Titan but shorter, at most 13 episodes" shown as understood conditions.

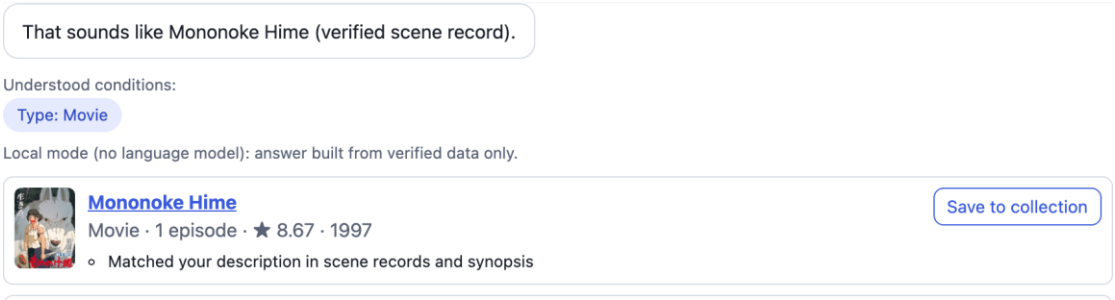


Figure 23. Scene lookup: a plot description identified as Mononoke Hime from a verified scene record.

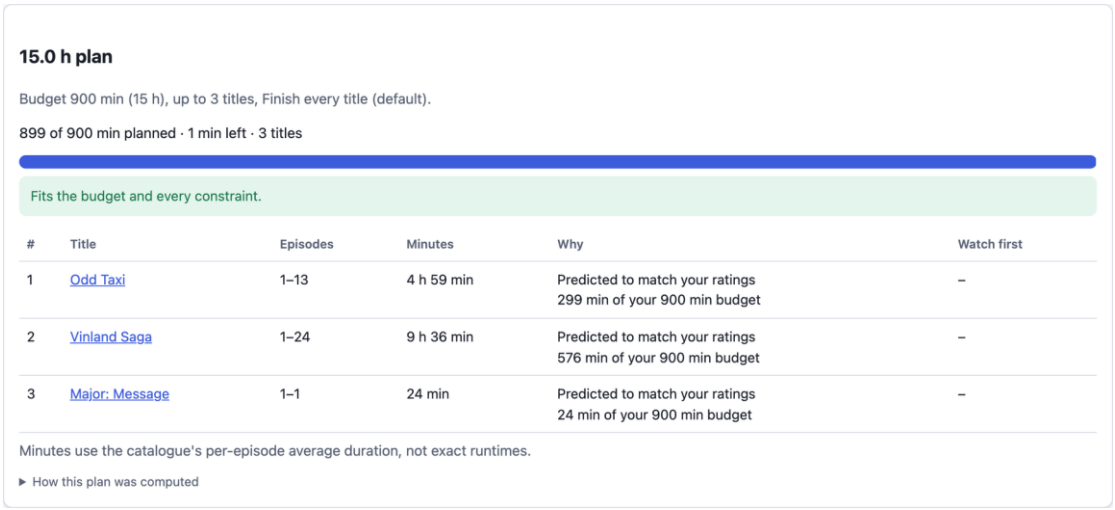


Figure 24. Viewing plan: three titles fill 899 of 900 minutes and the plan is checked against the budget.

10.4 Technical Challenges and Solutions

- **Unsafe prototype.** The first prototype trusted the client's user ID and computed results over 24 hand-written titles. We rebuilt it with hashed sessions, CSRF tokens and ownership checks; a second account cannot see another's data.

- **24 million ratings on a laptop.** Chunked cleaning, partitioned Parquet and blockwise sparse ICF computed offline; the full pipeline takes about 16 minutes.
- **LightGBM and PyTorch in one process.** Their two OpenMP runtimes froze the process on macOS, so training runs in separate subprocesses and the site scores LightGBM with a NumPy reader that matches native scores exactly.
- **Mismatched versions.** Versioned bundles with SHA-256 manifests and atomic activation ensure a new model is never paired with an old index.
- **Cold-start quality.** ICF predicted the same score for every title when a new user rated everything 9, and tag matching alone rarely found a liked title. A user-mean prior and a popularity weight, both tuned on validation, fixed this (validation Hits@10 0.010 → 0.041 for cold start).
- **Wrong scene claims.** Chat sometimes credited another title's scene to a plot question; a scene is now called verified only when its record ranks first in both retrieval channels (wrong claims 13 of 60 → 0 of 60).

10.5 Feasibility for Real Use

A fresh clone serves the real-data site with one command, because the repository ships the data version, models and databases through Git LFS (about 1.8 GB). The site needs no paid service: the LLM is optional and validated. For a public service, SQLite and one process would need a database server and more workers, the 2023 snapshot would need regular refreshes, and the use of MyAnimeList data and cover images would need a rights review.

11 Results and Progress

11.1 Key Results

- All four algorithms of the proposal and the viewing-plan extension run together on the full MyAnimeList 2023 data (23,363 served titles, 24 million ratings).
- Algorithm 2: B1 raised HR@8 from 0.021 to 0.132 and was confirmed on test by rules fixed before training.
- Similar titles: the served logistic route lifted test Hits@5 from 0.081 to 0.197 on MyAnimeList recommendation labels.
- Chat answered 100 of 100 team queries in local mode, with zero fabricated episode numbers; viewing plans were always feasible.
- Every page answers within 0.35 s at p95, and all 12 injected failures were handled.

11.2 Chat Module and Data Package Results

On 80 test queries and six LLMs, retrieval-grounded generation (R) raised the judge score from 3.50 for the LLM alone (N) to 4.05 on a 5-point scale, and the 95% interval of the gain lies above zero for every model (**Figure 25**). The template reply of the first prototype scored 2.44, judged on the reply text only. On scene questions, 86% of the R replies gave the episode stored in the knowledge base against 39% of the N replies, and 94–100% of the R replies named only titles that exist in the catalog against 20–38% of the N replies. GLM-5.3-Flash, GLM-5.3 and Kimi-K3 form a top tier that is significantly above the other three models, and they are the shortlist for the final module (Section 7.4).

The last number matters most for the product. A chat reply is useful only if every title it names can be shown as a card with its metadata; grounding makes that true for almost every reply, while the LLM alone leaves the catalog in most of them. As an experiment, the comparison is paired, quantitative and replayable offline.

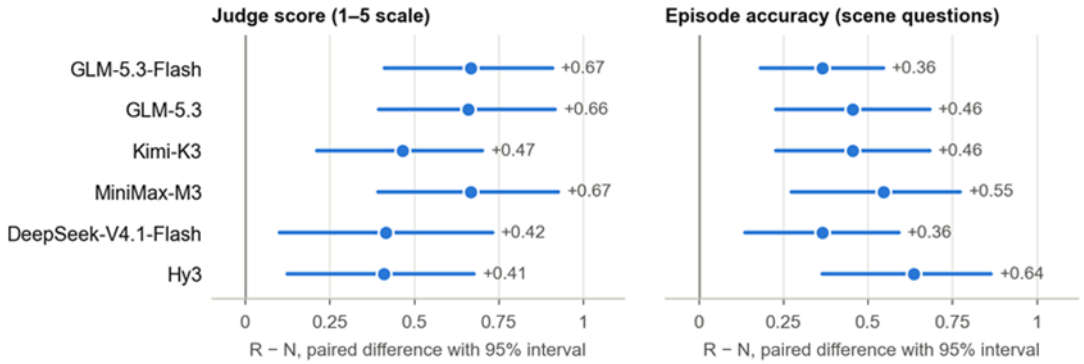


Figure 25. Gain of retrieval-grounded generation (R) over the LLM alone (N) for each model, as a paired difference with a 95% bootstrap interval. Every interval lies above zero, so the gain holds for all six models; the intervals overlap, so the size of the gain does not separate the models.

The data package supports the other algorithms as well: the 14,202-title catalog for Algorithm 1, its theme and demographic tags for Algorithm 2, and a rating matrix from the full rating file for Algorithm 3 (23.96 million eligible ratings, 19.4 million of them for training, no sampling). It also delivers the data for two features the proposal listed as optional, scene search (300 verified scenes for 100 titles) and the knowledge graph (47,312 entities), plus 100 evaluation queries with expected answers. **Table 27** sets the proposal against what happened.

Table 27. Proposal versus outcome for the data and the chat module

Item	Proposal	Outcome
Metadata sources	Jikan API for themes, demographics, characters and relations; a review dataset for the chat knowledge base	Jikan answered HTTP 504 for more than 24 hours. AniList became the main source (18,024 of 25,019 titles), with Kitsu, web pages and a 128-title knowledge fallback. The review dataset was not used
Data scale	About 12,000 usable titles; users sampled if memory ran short	14,202 titles and all 23,955,157 eligible ratings, no sampling
Chat knowledge	Scene-level knowledge base, knowledge graph and vector retrieval	300 scenes and a 47,312-entity graph prepared but not yet loaded into the application; the comparison ran on the prototype's 24-title base with lexical retrieval

The Jikan outage has the longest reach. It made AniList the main metadata source and left audience labels for about half of the titles (50.3%, or 37.4% of the runtime catalog once inferred values are removed), so components that use them must handle missing values; every value carries its source, which lets experiments drop the inferred ones. For a product, the episode shows the risk of building on a free, best-effort API: a deployed system would mirror or license its metadata. The chat comparison is the deviation that limits the claim. It measures the prototype with 24 famous titles, so it shows that grounding helps but not how much it helps in the final system (Section 12.4).

11.3 Progress

The project moved from a 24-title prototype to the full real dataset in four weeks (**Figure 26**). The rebuild on 30 September replaced the prototype; the Algorithm 2 comparison and the import of the team data package followed, and every experiment was re-run on the final data version.



Figure 26. Project milestones, September to October 2026.

11.4 Deviations from the Plan

- **Rebuilt rather than extended.** The prototype could not support safe multi-user use or real data, so we rebuilt it; this cost time but made the real-data evaluation possible.
- **Algorithm 2 ground truth and scorer.** Real collections did not exist, so a rating-based proxy replaced them, and the learned scorer B1 replaced the proposal's formula.
- **Empty results.** The presentation planned to fall back to pure profile ranking when the filters remove everything. We instead show the excluding rule and offer a relaxation, because silently ignoring a stated filter breaks trust.
- **Home list.** The GA keeps the proposal's weights although validation suggests lowering novelty and freshness, and a separate popular list was added.
- **LLM optional.** The final chat works without credentials and was evaluated in local mode; the LLM comparison of Section 7.4 used the earlier prototype pipeline.

11.5 Significance

Academically, the project shows how to choose between reasoning techniques honestly: each replaceable step was compared under rules written before the results, and the served defaults follow those rules. For the market, the system behaves like a product people can trust: filters are never bent silently, every reason traces back to data, and the site keeps working when the LLM or a model file fails.

12 Challenges and Roadblocks

This chapter sets out the challenges that remain open in the project and what each of them means for a market application. It follows the order of Chapter 7: one section for each of the four algorithms, then the challenges that cut across the whole system, and finally the plan for the remaining time. Each challenge is described by its obstacle and, where the sources allow, its consequence for real use and a next step.

All four algorithms were selected on offline evidence, and for Algorithms 1, 2 and 4 that evidence is also indirect: Algorithm 1 was judged against labels generated from metadata rules, Algorithm 2 against a rating-based proxy for real collections, and Algorithm 4 on the first prototype pipeline. For Algorithm 3 the central challenge is keeping recommendation quality within a tight online time budget. At system level, the open gaps are human judgment, LLM answers in the final chat module and real collection data (Section 9.6).

12.1 Algorithm 1: Knowledge-Based Similar-Anime Recommendation

Evidence from real users. The main remaining obstacle is the lack of validation with real users. Although the benchmark contains 1,000 targets, its relevance labels are generated from metadata rules rather than user judgments. Route A's coverage@5 of 30.26% therefore measures agreement with this synthetic reference, not the proportion of users who will click, watch, or enjoy a recommendation. Because the reference uses attributes similar to the recommender's, the current ranking of routes may also change under independent judgments. Route D, which Section 7.1.8 keeps as a semantic-enhancement candidate, is likewise to be activated only after evaluation with real users. This limits the evidence available for commercial adoption: improved retention or viewing activity cannot yet be claimed. A next step is an independently labeled evaluation followed by a limited user pilot, measuring recommendation satisfaction, click-through, and subsequent viewing against the existing recommendation experience.

Limited information for recommendation. The five metadata fields cannot fully express narrative style, emotional tone, or individual preferences, and incomplete tags further weaken comparisons. Users selecting the same target receive the same list, regardless of their viewing histories. In a market application, this could produce recommendations that are technically similar but feel repetitive or personally irrelevant, reducing trust and repeat use. Further work should improve metadata quality, evaluate richer content descriptions, and consider an optional personalization stage using user-authorized feedback. Each addition needs independent testing to establish whether it improves relevance and diversity.

Operational scalability. The reported timings cover a 14,202-entry catalog on one machine; they do not establish response times under concurrent demand or a much larger catalog. Scoring every candidate increases work as the catalog grows, while metadata updates require refreshed features and, for semantic routes, embeddings. Slow responses or stale results could affect user experience and increase operating costs. Before wider deployment, the project needs load tests, a defined refresh process, monitoring, and evaluation of caching or candidate retrieval. Availability filters would also be needed where titles differ by region or platform, so that users can act on the recommendations they receive.

12.2 Algorithm 2: Collection-Based Recommendation

Proxy label and missing user feedback. Algorithm 2 was chosen on a proxy label. Real collections did not exist, so a rating-based proxy replaced them, and a set of titles rated 8 or higher is not a collection (Section 7.2.5). The learned scorer B1 passed all five confirmation checks on this proxy and replaced the proposal's formula as the default. No user helpfulness ratings were collected, although a human-evaluation protocol has been written, and there is no online feedback yet. Offline gains may therefore not carry over to real users, which limits both the academic claim and the market case. The gain must be re-checked once real collections exist: the frozen Algorithm 2 pipeline is to be re-run on real collections as they accumulate, and B1 then compared with the tuned formula online.

Popularity bias in the held-out test. Held-out accuracy rewards popular titles, so a popularity list beats every personalized engine on Hits@10. In the Algorithm 2 test, most hidden titles are popular and highly scored, which favors the public-score feature, while obscure titles rarely reach the Top-8 (Section 7.2.5). The home-feed figures and the consequence for the product are given under Algorithm 3.

Missing labels and unseen titles. About half of the titles lack a demographic label, so the components of Algorithm 2 that use the demographic labels of the data package must work with

missing values. The test also uses the static 2023 snapshot, and the ability of C0, the content two-tower scorer (Section 7.2.1), to score unseen titles was not tested separately (Section 7.2.5). For a product, this means that titles released after the snapshot are missing from the recommendations and that how well the scorer handles unseen titles is not established. Data refresh is covered under System-Level Challenges.

12.3 Algorithm 3: Main-Page Recommendation with ICF and GA

Quality against efficiency. In the two-stage architecture of Item-based Collaborative Filtering (ICF) and a Genetic Algorithm (GA), balancing recommendation quality and computational efficiency is a major technical challenge. As a stochastic optimizer, the GA depends on population size, number of generations and encoding strategy; larger settings can improve the solution but add computational overhead. This matters most online, where the home page list is re-ranked in real time and must meet strict response-time limits. To make the system deployable while maintaining recommendation quality, the project optimizes three areas: GA encoding, collaborative-filtering similarity and objective-function evaluation. Each is described below together with how it is handled and what remains open.

Search depth under the 250 ms budget. In the offline benchmark, the GA with Random-key Encoding, a population of 60 and 30 generations improved Long-tail Exposure (LTE) by 72.8% over the greedy baseline. Random-key Encoding needs sorting-based decoding and a population-wide fitness evaluation in every generation, which is difficult to fit into the low-latency requirement of online recommendation. The deployed system instead uses Set-based Encoding, which optimizes candidate solutions of K distinct items directly, with an online computation budget of 250 ms: the search runs for the remaining time, then stops and returns the best solution found so far. This makes response time more predictable, but the restricted search depth may reduce the optimization quality of the final list, and the 72.8% gain was measured offline with Random-key Encoding and a fixed number of generations. The system thus prioritizes online response efficiency while trying to preserve the optimization capability of the GA.

Similarity estimation under sparse ratings. The offline benchmark uses Standard Cosine Similarity on a column-normalized Gram matrix, which is simple, efficient and reproducible, but on a highly sparse rating matrix its estimates can be distorted by few co-rated items or extreme ratings. For this reason the deployed system retains a combination of Adjusted Cosine Similarity, Shrinkage and Prior Weighting. Adjusted Cosine Similarity reduces the influence of individual users' rating scales, Shrinkage lowers the uncertainty of similarities based on few co-ratings, and Prior Weighting adjusts the contribution of different candidate items. Item similarities are computed in offline preprocessing, so the extra complexity does not add to online request latency, and the extra offline computation is spent on more stable similarities for online candidate retrieval. The accuracy advantage over Standard Cosine Similarity is, however, so far supported only by theory and still has to be validated through comparative experiments under consistent evaluation conditions.

Cost of the diversity objective. The GA fitness jointly considers relevance, diversity, novelty and recency (whether a title was released within the preceding five years). Diversity is the pairwise dissimilarity (1 − Jaccard Similarity) of the genre and theme labels of two titles, so comparing every pair in a candidate set costs $O(n^2)$ and would raise the cost of each fitness evaluation substantially. To contain this cost, the system restricts diversity computation to the Top-20 candidate window retrieved by collaborative filtering (RERANK_DEPTH = 20), so that the pairwise matrix has at most 20 × 20 entries, and precomputes it before the search starts so that

every fitness evaluation reuses it and no label-set intersection or union is repeated. Because the GA optimizes only within this window and items outside it do not enter the re-ranking, the restriction adds no approximation error within the optimization scope, and diversity inside the window remains fully computed.

Accuracy cost of diversity. The GA gives up accuracy for diversity, novelty and freshness: it gives up 1.5 percentage points (pp) of Hits@10 once themes are filled in, in return for clearly more diverse, novel and fresh lists (Section 9.2), the same trade-off reported in Section 7.3. On the home feed a popularity list reaches Hits@10 of 0.137 against 0.062 for ICF, because most held-out titles are popular, but it covers about 30 times less of the catalog, which is why the home page shows a separate, labeled popular list. Judging recommendations by accuracy alone would push the product toward popular lists and away from what the GA is built to provide. The GA also keeps the proposal's weights although validation suggests lowering novelty and freshness (Section 11.4). The next step is to test lower novelty and freshness weights for the GA on validation users and to report coverage and novelty beside accuracy.

12.4 Algorithm 4: Retrieval-Augmented Chat

Evidence from the prototype, not the final chat module. The central obstacle is that the evidence comes from the first prototype, not from the chat module users will see. The 80-query comparison of the three routes in Section 7.4 (template reply T, LLM alone N and retrieval-augmented generation R) ran on AnimeFinder v1.4.1 with a 24-title knowledge base, lexical retrieval and mostly Chinese queries (74 of 80), whereas the final module retrieves with full-text and dense search over an English corpus that covers the whole catalog. The gain of R over N therefore shows that grounding helps, not how large it is in the shipped system; and because the 24 titles are famous, the benefit for lesser-known titles is probably understated. This limits any commercial claim, such as fewer invented titles in real conversations. The final chat module has so far been evaluated only in local mode (Section 11.4). A next step is to repeat the three-route comparison on the final module, once an evaluation budget is approved. It would use the 100 English queries prepared in the data package (30 semantic, 20 constrained, 10 unanswerable and 40 scene) and the three shortlisted models: GLM-5.3-Flash, GLM-5.3 and Kimi-K3.

Questions that name a title. In the prototype, a ranking rule meant for recommendation requests pushed the named title out of the prompt, so R answered 19 of 48 replies to single-fact questions correctly (8 queries, 6 models), against 48 of 48 for the LLM alone (Section 7.4.5). The final module resolves titles in a separate step, but fact lookup has not been retested. A chat that loses to a bare LLM on simple facts would cost trust quickly. The next step is to rerun the fact queries on the final module and, if single-fact questions still lose, to answer them from the title record before retrieval is used.

Scene and relation coverage. The 300 verified scenes and the relation graph still have to be loaded before scene and relation questions can be answered from sources, and coverage is partial: relations cover 10,883 of 23,363 titles, scenes cover 100 titles, and no frame images exist for screenshot search. Reverse lookup from a described scene is what the proposal called the most differentiated part of the system, so a narrow scene base bears directly on the product's most distinctive feature. The next steps are to load the scene and relation files and rerun the scene queries.

Limits of the measurements. The judge score comes from three LLMs with no human reference: they differ in strictness, model ranks change unless scores are centered per judge, and rank agreement between judges within a question is only 0.54–0.73. Each cell is a single run, and the

answers came through a command-line channel that does not allow setting the temperature and cannot reproduce the application's 20-second timeout, so latency and token figures are relative (Section 7.4.6). A model could be chosen on judge preferences that users do not share, and a slow or costly one could be found out only after launch. The two rule-based measures do not depend on the judges and carry the decision for now. Further work should have team members rate a small sample of replies to calibrate the judges, repeat the runs for the shortlisted models, and measure latency and cost with live API calls once a model key is available.

12.5 System-Level Challenges

Data freshness and external sources. The system rests on a 2023 snapshot that misses new releases, and the Jikan outage showed the risk of building on a free, best-effort API: it made AniList the main metadata source and left only about half of the titles with a demographic label (50.3%, or 37.4% of the runtime catalog once inferred values are removed; Section 11.2). A deployed system would mirror or license its metadata, a public service would need regular refreshes of the snapshot, and the use of MyAnimeList data and cover images would need a rights review (Section 10.5). The saved Jikan queue is to be resumed when the service recovers. The coverage gaps in demographic labels, relations and scenes are described under Algorithm 2 and Algorithm 4.

Deployment and concurrency. The system is measured on one laptop, so response times under concurrent demand are unverified; the Algorithm 1 timings likewise cover a single machine. The Docker setup has not been run, and SQLite limits concurrent writers. For a public service, SQLite and a single process would need to give way to a database server and more workers (Section 10.5). The LLM answers of the final chat module also await an evaluation budget (see Algorithm 4). The next step is to test the Docker setup on a second machine; load tests, as described for Algorithm 1, would be needed before wider deployment.

12.6 Strategy for the Remaining Time

The remaining time is organized around the following five steps, each addressing an open point described above; the sections above give further next steps for each algorithm.

1. **Human evaluation.** Run the written human-evaluation protocol with a small group of classmates on fixed title pairs and chat sessions.
2. **Algorithm 2.** Re-run the frozen Algorithm 2 pipeline on real collections as they accumulate, then compare B1 with the tuned formula online.
3. **Algorithm 4.** Repeat the Section 7.4 comparison on the final chat module with the three top-tier models once a budget is approved.
4. **Algorithms 2 and 3.** Report coverage and novelty beside accuracy, and test lower novelty and freshness weights for the GA on validation users.
5. **Data and deployment.** Resume the saved Jikan queue when the service recovers, and test the Docker setup on a second machine.

13 Future Work

In the future, this project focuses on three main areas for further improvement: data collection and updating mechanisms, dynamic management of user behavior data, and system functionality expansion. These improvements aim to enhance data timeliness, recommendation responsiveness,

system scalability, and maintainability, thereby providing a solid foundation for the long-term operation and practical application of the system.

First, in terms of data collection and updating, anime content has several characteristics, including a large volume of works, frequent updates, scattered information sources, and continuous growth in related content. As new anime series are released, existing information changes, and user ratings and reviews continue to accumulate, static datasets become insufficient to meet the long-term requirements of the recommendation system for data timeliness and completeness. Therefore, future work focuses on exploring a data maintenance mechanism that combines automated data collection with incremental updates. This mechanism gradually supports the regular collection and updating of basic anime information, categories, ratings, reviews, and related associations while maintaining data quality. In addition, data deduplication, anomaly detection, and field completeness checks improve data consistency and reliability, reducing the impact of redundant and invalid information on recommendation results. The project also optimizes data collection workflows and resource scheduling to reduce the overhead caused by repeatedly processing the entire dataset. These improvements lower database maintenance costs and enable the system to accommodate the continuous growth of anime content.

Second, in terms of user behavior data management and recommendation responsiveness, some recommendation algorithms in this project rely on the statistical analysis of user interactions, such as favorites, ratings, and other activities. Therefore, the completeness, update frequency, and processing efficiency of user behavior data directly affect the accuracy of user interest profiles and, consequently, the relevance and timeliness of recommendation results. Future work further improves the automated collection, storage, analysis, and updating of user behavior data. It also explores different mechanisms, including scheduled batch updates and event-triggered updates, to ensure that statistical results and recommendation inputs reflect changes in user behavior in a timely manner. On this basis, comparative experiments evaluate how different updating strategies and intervals affect recommendation accuracy, recommendation diversity, computational resource consumption, and system response time. These experiments help determine a more appropriate updating strategy by balancing recommendation quality and operational efficiency. Furthermore, as user behavior data continues to accumulate, methods such as caching, incremental statistics, and data processing optimization help reduce redundant computations, improve recommendation responsiveness, and maintain system stability.

Finally, in terms of system functionality expansion and platform management, the current project mainly provides anime information retrieval, personalized recommendations, and related interactive services through a web interface. To broaden the system's applicability, future work includes developing a mobile application based on the existing system architecture, allowing users to access the platform across different devices and usage scenarios and enjoy a more convenient and consistent service experience. This process also addresses issues such as cross-device data synchronization, user authentication, interface adaptation, and API compatibility to maintain functional consistency and data integrity across different platforms. In addition, the project introduces a dedicated administrator mode and gradually improves functions such as anime data maintenance, user information management, data update task management, system status monitoring, and error log recording. By establishing a more comprehensive management mechanism, defining the access permissions of ordinary users and administrators, and improving access control and error handling, the system enhances its security, maintainability, and operational efficiency. These improvements also support subsequent feature development and system deployment.

14 Conclusion

This project addresses the challenges of anime content discovery and personalized recommendation through the design and implementation of AniLogic, a hybrid intelligent recommendation system. The project has evolved from an initial prototype containing only 24 manually entered anime titles into a fully functional application built upon the MyAnimeList 2023 dataset. The final system integrates 23,363 serviceable anime entries, approximately 24 million user rating records, 14 functional pages, a versioned JSON API, and an offline data processing pipeline covering raw data cleaning, attribute integration, data validation, and database construction. In terms of technical architecture, the system incorporates four categories of intelligent reasoning techniques: knowledge-based recommendation using attribute similarity, rule- and knowledge-based recommendation, item-based collaborative filtering combined with genetic algorithm optimization, and Cognitive Language Perception integrated with Retrieval-Augmented Generation (RAG). These techniques are applied to anime detail pages, user favorites, the personalized homepage, and natural language interaction, respectively. Through shared data resources, consistent constraint-handling mechanisms, and unified backend interfaces, the system demonstrates the feasibility of integrating multiple intelligent reasoning techniques within a single recommendation platform.

From an algorithmic research perspective, the primary contribution of this project lies in establishing a systematic evaluation methodology centered on comparative experimentation and data-driven model selection. For different recommendation tasks, key components, including similarity computation, feature representation, candidate filtering, ranking strategies, collaborative filtering-based retrieval, genetic algorithm-based re-ranking, and knowledge-enhanced generation, are treated as independent experimental variables. Baseline comparisons and ablation studies are conducted across alternative approaches using predefined evaluation metrics, experimental protocols, and model replacement criteria. For knowledge-based recommendation, the fixed five-dimensional weighted similarity method achieves the best performance across all evaluated metrics in experiments involving 1,000 target anime titles. For rule- and knowledge-based recommendation, the learning-based ranking model improves Hit Rate at 8 (HR@8) from 0.021 to 0.132 and is selected as the default approach after passing five confirmatory checks. For homepage recommendation, under the full-catalog evaluation setting, the genetic algorithm increases long-tail content exposure by approximately 51% at the cost of a reduction of approximately one percentage point in Hit Rate at 10 (HR@10). This result demonstrates a quantifiable trade-off between recommendation accuracy and content diversity while supporting the effectiveness of a two-stage architecture in which collaborative filtering performs candidate retrieval and the genetic algorithm conducts multi-objective re-ranking. For cognitive language perception and natural language interaction, comparative experiments involving 80 test queries and six language models show that introducing RAG improves the average evaluation score from 3.50 to 4.05 and increases episode-count accuracy from 0.39 to 0.86. These findings indicate that external knowledge retrieval can effectively enhance the factual reliability of language model outputs.

In terms of data engineering and system implementation, the project further demonstrates the engineering feasibility of large-scale data processing and multi-technique integration. A complete offline processing pipeline is established to transform raw data files into a deployable database. Through data cleaning, attribute completion, entity association, format standardization, and consistency validation, the pipeline provides a unified data foundation for different recommendation algorithms. The transformation rules and validation mechanisms applied at each

processing stage are documented to support reproducible dataset construction and experimental results. In the system-level evaluation, all functional pages pass validation tests using real data, with no observed violations of rule-based constraints, infeasible recommendation plans, or fabricated episode counts. Under the evaluated test conditions, the 95th-percentile request latency (p95 latency) remains below 0.35 seconds, and all 12 fault-injection tests are successfully handled through the intended graceful degradation mechanisms. These results indicate that the system not only supports the functional integration of multiple recommendation algorithms but also demonstrates operational efficiency, constraint consistency, and a degree of fault tolerance.

From an academic perspective, the significance of this project extends beyond performance improvements achieved by individual algorithms to the development of a testable, comparable, and reproducible methodology for algorithm evaluation. By employing consistent experimental designs and predefined model selection criteria, the project reduces the limitations associated with selecting algorithms based on isolated examples or subjective judgments. The experimental findings further indicate that increased algorithmic complexity does not necessarily lead to improved recommendation performance. For instance, under certain experimental conditions, semantic similarity methods do not outperform the fixed-weight similarity model, suggesting that model selection should consider task characteristics, data quality, evaluation objectives, and empirical performance rather than algorithmic complexity alone. Furthermore, by incorporating modular system design and graceful degradation mechanisms, the project connects algorithmic performance evaluation with system reliability assessment, providing a practical reference for the experimental design, model selection, and engineering implementation of intelligent recommendation systems that integrate multiple techniques.

From the perspective of practical applications and potential market impact, AniLogic demonstrates the technical feasibility of developing a personalized, explainable, and interactive content discovery system with limited computational resources and without relying on licensed content from paid streaming services. Compared with conventional content retrieval approaches that primarily depend on genre labels and keyword filtering, the system integrates historical user behavior, structured anime knowledge, and natural language requirements to generate recommendations. Through data-driven recommendation explanations, knowledge source citations, and consistent enforcement of hard constraints, the system improves the transparency and controllability of the recommendation process. This design is not limited to anime content discovery but also provides a potential technical reference for personalized recommendation and intelligent information retrieval in other digital content domains, including films, television series, books, and related media.

Supplementary Materials

Use of AI

Generative AI assistants were used in this project in the ways listed below. The team reviewed the AI-assisted output and remains responsible for the content of this report and of the delivered system.

- **Code formatting and optimization.** AI assistance was used to format the source code and to suggest optimizations, which the team reviewed before adopting them.
- **Organization of the final system files.** AI assistance was used to tidy and organize the final system files for delivery.
- **Translation support.** AI assistance was used to help translate drafts of the report into English.
- **Academic wording.** AI assistance was used to make the wording of the report more academic.
- **Layout and compilation of the report.** AI assistance was used to typeset the report and its appendices (styles, heading, figure and table numbering, cross-references and the cover page), to reorganize Chapter 12, and to compile Appendices A to C into this document.

Acknowledgements

We thank the teaching team of the Intelligent Reasoning Systems practice module for their guidance throughout the project and for the feedback on our proposal presentation.

We are grateful to the projects and communities whose open data and software this work builds on: the MyAnimeList dataset published on Kaggle by dbdmobile and the MyAnimeList community whose ratings it records; the Jikan, AniList and Kitsu services; Wikipedia and TheTVDB; and the open-source tools behind the system, including FastAPI, SQLite, LightGBM and PyTorch, and the BAAI BGE embedding models.

Appendices

The following appendices complete this report.

- **Appendix A**: Project Proposal
- **Appendix B**: Knowledge Use in the System
- **Appendix C**: User Guide and Installation

Appendix A Project Proposal

Intelligent Reasoning Systems Proposal for Group 18

Project Name：AniLogic -- Your ACG friend

Group number：Group18

Group member：

1. Gao Xinghe (A0305470M)
2. Lin Kai (A0352380J)
3. Sun Yancheng (A0335090L)
4. Yu Yuchen (A0356318Y)

A.1 Introduction

With the continuous expansion of the global pan-ACG (Animation, Comics, and Games) market, anime has grown into a billion-dollar mainstream entertainment industry. However, amid this explosion of content, existing major streaming platforms and rating databases still suffer from structural bottlenecks in their content discovery mechanisms:

- Homogeneous Recommendations: Traditional recommendation engines primarily compute relevance for individual titles, lacking mechanisms to globally optimize for diversity and novelty at the list level.
- Single-Dimension Retrieval: Current search inputs generally support only exact keyword matching for titles, failing to fulfill reverse-lookup needs based on plot descriptions, iconic scenes, or visual information.
- Inadequate Understanding of Complex Needs: Traditional filter-based tools cannot effectively parse vague natural language inputs that carry multi-constraint or non-standard user requirements.
- Lack of Planning Capabilities: Existing ranking approaches are structurally incapable of solving combinatorial optimization problems bounded by specific user time constraints.

To address these pain points, Group 18 introduces AniLogic—an intelligent anime discovery and viewing-plan system built on a hybrid reasoning architecture. The system integrates Knowledge-Based Reasoning, Item-Based Collaborative Filtering, Genetic Algorithms for global list optimization, and RAG-based Natural Language Retrieval powered by cognitive LLMs and an anime knowledge base. AniLogic aims to break away from rigid title-centric searches, delivering a high-accuracy, highly diverse, and conversational content discovery engine for modern users.

A.2 Problem Definition

A.2.1 Background for Market Perspective

Market scale: In 2024, pan-ACG (2D culture) users in China reached 503 million — nearly half of the country's total netizen population — making China the world's largest ACG market. Market size, growth and competitor detail are given in Section A.2.2.

How users find content today. Whether on Bilibili, Netflix, Disney+ or Crunchyroll, finding an anime title still relies heavily on only two entry points: homepage recommendations and title-keyword search.

Rating platforms fill only part of the gap. Douban and MyAnimeList provide quality-reference signals, but they likewise lack the ability to "help users locate their next watch".

The market gap this project targets. A world-leading consumption scale coexists with a structurally weak discovery experience — no mainstream platform offers **list-level viewing plans or language-level retrieval.**

A.2.2 Market Research

Because this project is an MVP product rather than a research study, this section reviews the market rather than the academic literature. It addresses three questions: how large the market is, who the key players are and where they fall short, and which unmet user needs create an opening for our system.

A.2.2.1 Market Size and Growth

The global anime market was valued at about USD 36.2 billion in 2025 and is projected to reach USD 66.0 billion by 2032 (CAGR ≈ 9%), with Asia-Pacific the largest region.

Anime is no longer a niche category. Netflix reported in July 2025 that more than 50% of its global members watched anime in 2024, with anime titles reaching the Top 10 in 33 countries.

In China, pan-ACG users reached 503 million in 2024 (Section A.2.1); Bilibili reported 376 million MAU in Q3 2025, with anime and ACG watch time up about 25% year on year.

A.2.2.2 Key Players and Their Gaps

Crunchyroll (Sony) — the dedicated anime streamer. Over 17 million paid members as of 31 March 2025 and a library of 50,000+ episodes; following the Funimation merger it accounts for roughly 80% of anime streaming. *Gap:* discovery is still catalogue browsing plus title search, and recommendations are title-to-title, with no list-level or time-budgeted planning.

Netflix, Disney+, Amazon Prime Video — generalist streamers. Anime is one vertical among many and a proven acquisition driver (Netflix: more than half of its members watched anime in 2024). *Gap:* no anime-specific discovery tooling; discovery reduces to homepage carousels and title search.

Bilibili — China's largest ACG community. 376 million MAU (Q3 2025) with ACG watch time growing about 25% year on year. *Gap:* its strengths are community and playback rather than planning; a user cannot state a time budget or ask for a deliberately mixed list.

MyAnimeList, Douban, Bangumi, AniList — rating and cataloguing databases. MyAnimeList alone has about 19.5 million registered users across 240 regions — 99% of them outside Japan, and roughly 90% of visitors aged 18–34. *Gap:* they supply quality signals and personal tracking, but not "help me find my next watch", which is precisely what the user complaints in 2.1 describe.

Trend — distribution is consolidating while catalogues keep growing. Content has concentrated into a handful of platforms (Crunchyroll absorbed Funimation; Netflix and Disney+ keep expanding their anime slates), so the number of discovery entry points is falling even as the number of titles rises. Access is no longer the bottleneck; discovery is.

A.2.2.3 User Needs and Opportunity

Diversity, not only relevance. Users complain that recommendations are homogeneous and that no platform optimises a list as a whole.

Retrieval by scene or description. Discovery increasingly starts from a clip, a screenshot or a half-remembered scene, yet every incumbent search box accepts exact titles only.

Vague, natural-language needs. Requests such as "something like Attack on Titan but shorter" are the normal register of conversation and the least supported by filter-based search.

Time-budgeted viewing plans. "15 hours this weekend, and don't repeat the styles" is a constrained combinatorial problem that no current product answers.

Opportunity. Incumbent catalogues are large and their metadata is publicly accessible — the MyAnimeList catalogue and rating data used in Section A.4, plus the Jikan API — so a system that layers knowledge-based matching, collaborative filtering, genetic-algorithm list optimisation and RAG-based language retrieval on top of that data can address the discovery gap without competing on content licensing. The audience is favourable too: MyAnimeList's users are 99% outside Japan and predominantly aged 18–34, the same segment whose unmet needs are documented in 2.1.

A.2.3 Background for Product Perspective

Existing platforms suffer from four core shortcomings.

(1) Homogeneous recommendations and the absence of a list-level experience. Current recommender systems optimise the relevance score of a single title, with no mechanisms in place to guarantee novelty or diversity.

> *Voice of the user — MyAnimeList official forum, June 2025*
>
> "Half the time, MAL's recommendations are: 'If you liked A (which I've watched), you might like B (which I've also already finished).' Recommending titles I've already marked as Completed — what's the point?" — forum user danmast94
>
> "The 'Recommended for You' section is hardly personalized at all. It just picks a score band and dumps every title in that band on you. I actually tested it: all 36 of my 'recommendations' were crammed into the 7.96–8.00 score range, ranked #665–#724, and mixed in were plenty of second seasons of shows I hadn't even seen — am I supposed to start with season two?" — forum user Darkonius (same thread)
>
> Source: https://myanimelist.net/forum/?topicid=2219140

(2) Search matches only exact title text, which is misaligned with how users remember. The search boxes on existing platforms essentially support only exact keywords such as titles or character names. Yet the real-world scenario of anime watching is this: a user catches a clip on a short-video platform, receives a screenshot in a group chat, or, years later, remembers only a certain scene or a line of dialogue — what they remember is the "scene", not the "title". Unable to search that way, users can only post questions in online communities or dig through lists manually, which is extremely inefficient.

> *Voice of the user — Baidu Zhidao, July 2013*
>
> Question: "Does any expert know which anime this image is from?!" More than a decade later, there is still no definitive answer under this question.
>
> Source: https://wen.baidu.com/question/564087842.html

(3) Vague needs cannot be handled by structured tools. A substantial share of user needs are expressed as vague natural language, such as "something like Attack on Titan but shorter". Existing anime sites, however, offer only filter-based search. This mismatch between how needs are expressed and what the tools can handle keeps a large number of light users out.

> *Voice of the user — complaints from both ends*
>
> On the demand side, users can only speak like humans: "I want an anime similar to Attack on Titan, but not too long, ideally with lots of fighting."
>
> Typed into any existing platform's search box, this sentence either returns no results at all, or returns every video whose title contains the word "Attack".

(4) Recommendation alone cannot handle "viewing plan" needs. Users' time budgets are limited (e.g. "I have 20 hours this weekend"). What they truly need is not a set of isolated high-rated titles, but a combined plan that balances overall quality, genre diversity and time constraints. This is a combinatorial optimisation problem that ranking-based recommendation is structurally unable to solve.

> *Voice of the user — a high-frequency question type in communities (e.g. r/Animesuggest)*
>
> "I have 15 hours this weekend — please recommend a batch of high-rated anime, and don't let the styles repeat too much."
>
> Typical replies: netizens each toss out a few titles, with no guarantee that the total duration stays within budget, that the genres don't overlap, or that the quality holds up — because this is inherently a constrained combinatorial optimisation problem (a knapsack problem) that crowd-sourced replies cannot solve, while the "ranking-style recommendation" of existing products is structurally incapable of answering "how does this batch add up as a whole?"
>
> Subreddit: https://www.reddit.com/r/Animesuggest/

A.2.4 Problems to Solve

To address the issues above, the system aims to accomplish three things.

First — accurate yet diverse recommendations. Deliver recommendations that are accurate while also ensuring diversity.

Second — reverse-lookup from vague descriptions or screenshots. Build a RAG-based chat system that supports natural-language descriptions — and even direct image input — to reverse-lookup the title of a work and the origin of iconic scenes. This feature targets the search blind spot described above and represents the most differentiated part of our system.

Third — an optimal viewing plan. Use a Genetic Algorithm to generate an optimal viewing plan tailored to each user's specific circumstances.

A.2.5 Goal

Build an anime-discovery system that "understands human language" — one in which users can:

browse recommendation lists; obtain a recommended viewing plan from a natural-language description; and identify an anime from a single screenshot or a vague description.

A.3 Core Algorithm Design

Algorithm 1: Similar Anime Recommendation Algorithm for Anime Detail Pages
Technology Used: Knowledge-based reasoning

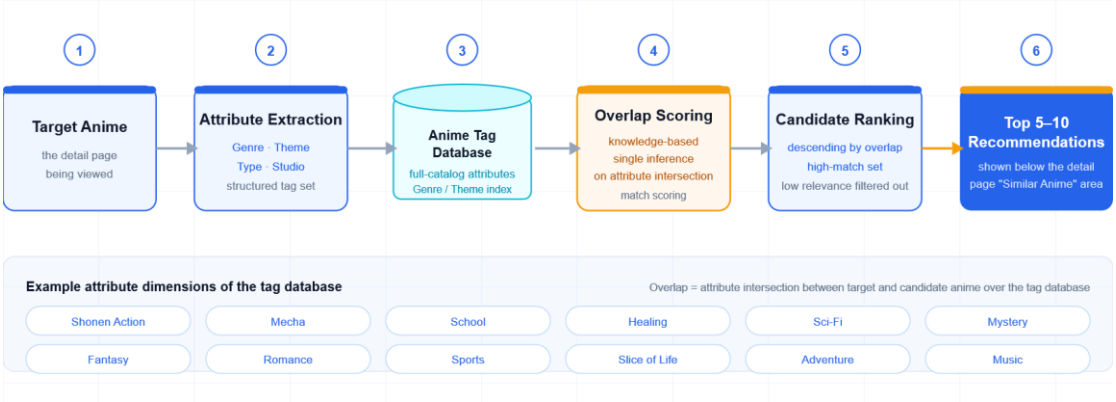


Figure A-1. Algorithm 1: Similar Anime Recommendation Algorithm for Anime Detail Pages

Detailed Description: At the bottom of each anime detail page, the system recommends 5–10 similar results based on attribute matching with the target anime. The system establishes an anime tag database containing attributes such as Genre and Theme, and uses knowledge-based single reasoning to calculate the attribute overlap between the target anime and all anime in the database, thereby generating candidate works with high matching scores.

Algorithm 2: Hybrid Recommendation Algorithm Based on User Favorites and Keyword Filtering for the User Favorites Page

Algorithm 3: Main Page Recommendation Algorithm Based on All User Ratings and Genetic Algorithm Optimization
Technology Used: Item-based CF + GA

Detailed Description:

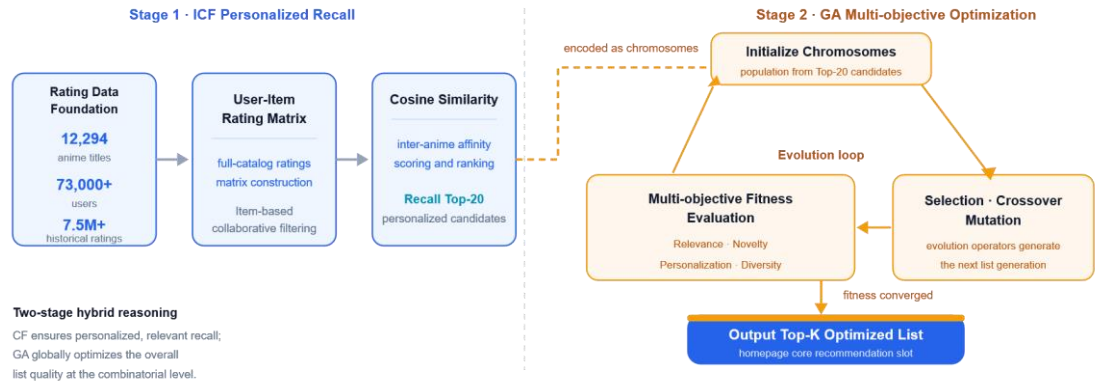


Figure A-3. Algorithm 3: Main Page Recommendation Algorithm Based on All User Ratings and GA Optimization

As the core recommendation algorithm of the main page, a hybrid reasoning mechanism combining Item-based Collaborative Filtering (ICF) and Genetic Algorithm (GA) is adopted. First, a User-Item rating matrix is constructed from the rating records described in Section A.4, after the filtering and sampling outlined there. Cosine similarity is then used to calculate the associations between anime titles, retrieving the Top-20 personalized candidate items.

The candidate set is subsequently modeled as the chromosomes of the GA. A multi-objective fitness function incorporating Relevance, Personalization, Diversity, and Novelty is designed.

Through evolutionary iterations involving selection, crossover, and mutation, the recommendation list is globally optimized, producing the final Top-K optimized recommendation list.

Algorithm 4: RAG-based Language-aware Search System

Technology Used: Cognitive LLM + Knowledge base + RAG

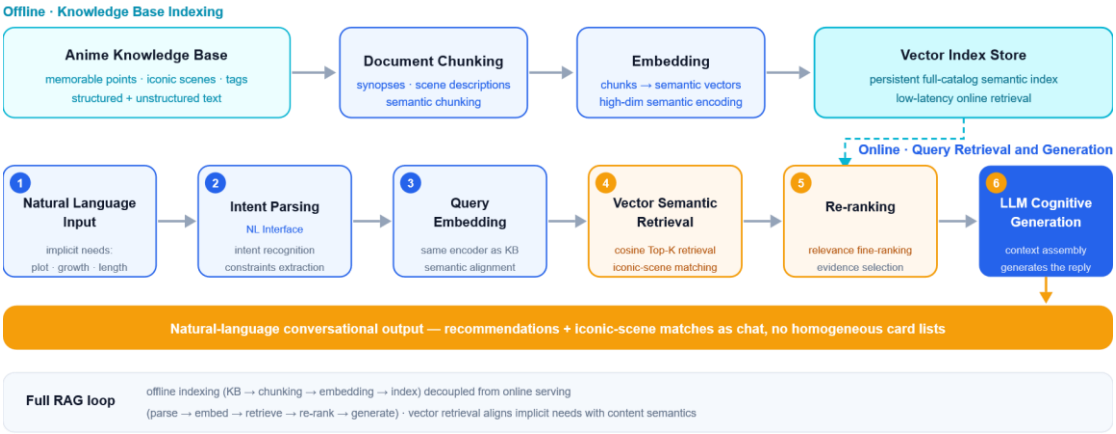


Figure A-4. Algorithm 4: RAG-based Language-aware Search System

Detailed Description: Based on a cognitive computing architecture, the system integrates a Large Language Model (LLM), an anime knowledge base containing memorable moments, iconic scenes, tag attributes/structured knowledge graphs, and RAG technology to construct a natural language conversational search system. The system receives and analyzes users' implicit requirements through a Natural Language Interface, such as plot style, character development, and length constraints. RAG is used to perform vector-based semantic retrieval over unstructured anime text and iconic scene descriptions, while the relational topology of the knowledge graph is incorporated for structured logical reasoning. Recommendation results and iconic scene matches are presented in the form of natural language chat conversations, enhancing interaction awareness and avoiding homogenization with traditional card-based list formats.

A.4 Data Collection and Preparation

A.4.1 Data Sources and Collection

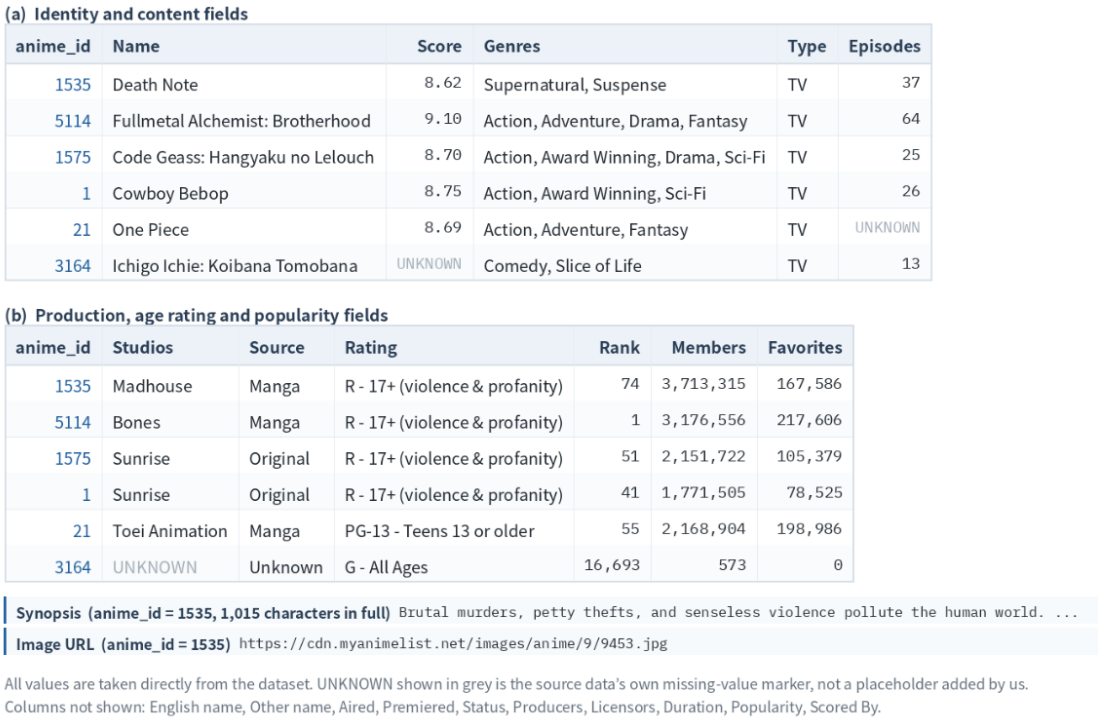
https://www.kaggle.com/datasets/dbdmobile/myanimelist-dataset

The dataset was scraped from MyAnimeList (MAL) and consists of three independent CSV files, linked through two identifiers, anime_id and user_id. Anime identifiers follow MAL's official numbering, so any additional data pulled from the same origin can be joined directly without an identifier mapping step.

File 1: anime-dataset-2023.csv (24,905 rows × 24 columns, one row per title)

- **anime_id** — official MAL title identifier; the primary key linking all three files.
- **Name / English name / Other name** — romanised title, English title and original-language title, used for display and title search.
- **Score** — MAL weighted average rating on a 10-point scale.
- **Genres** — genre labels, stored as a comma-separated multi-value field.
- **Synopsis** — official plot summary; the main text source for semantic retrieval in the chat module.
- **Type** — TV, Movie, OVA, ONA, Special,Music, or unspecified.

- **Episodes** — episode count.
- **Aired / Premiered / Status** — broadcast period, premiere season and airing status.
- **Studios / Producers / Licensors** — production and distribution companies.
- **Source** — adaptation origin (manga, light novel, original work, game and so on).
- **Duration** — running time per episode.
- **Rating** — age classification, used to filter out adult content.
- **Rank / Popularity / Members / Favorites / Scored By** — ranking and popularity indicators, used in the novelty calculation.
- **Image URL** — cover image address, consumed directly by the front end.



(a) Identity and content fields

anime_id	Name	Score	Genres	Type	Episodes
1535	Death Note	8.62	Supernatural, Suspense	TV	37
5114	Fullmetal Alchemist: Brotherhood	9.10	Action, Adventure, Drama, Fantasy	TV	64
1575	Code Geass: Hangyaku no Lelouch	8.70	Action, Award Winning, Drama, Sci-Fi	TV	25
1	Cowboy Bebop	8.75	Action, Award Winning, Sci-Fi	TV	26
21	One Piece	8.69	Action, Adventure, Fantasy	TV	UNKNOWN
3164	Ichigo Ichie: Koibana Tomobana	UNKNOWN	Comedy, Slice of Life	TV	13

(b) Production, age rating and popularity fields

anime_id	Studios	Source	Rating	Rank	Members	Favorites
1535	Madhouse	Manga	R - 17+ (violence & profanity)	74	3,713,315	167,586
5114	Bones	Manga	R - 17+ (violence & profanity)	1	3,176,556	217,606
1575	Sunrise	Original	R - 17+ (violence & profanity)	51	2,151,722	105,379
1	Sunrise	Original	R - 17+ (violence & profanity)	41	1,771,505	78,525
21	Toei Animation	Manga	PG-13 - Teens 13 or older	55	2,168,904	198,986
3164	UNKNOWN	Unknown	G - All Ages	16,693	573	0

Synopsis (anime_id = 1535, 1,015 characters in full) Brutal murders, petty thefts, and senseless violence pollute the human world. ...

Image URL (anime_id = 1535) https://cdn.myanimelist.net/images/anime/9/9453.jpg

All values are taken directly from the dataset. UNKNOWN shown in grey is the source data's own missing-value marker, not a placeholder added by us.
Columns not shown: English name, Other name, Aired, Premiered, Status, Producers, Licensors, Duration, Popularity, Scored By.

Figure A-5. Sample records from anime-dataset-2023.csv

File 2: users-score-2023.csv (approximately 24 million rows, one row per rating)

- The fields are user_id, Username, anime_id, Anime Title and rating, where rating is an explicit integer score from 1 to 10. Username and Anime Title duplicate information that can be recovered from the identifiers, so both are discarded at load time to keep memory usage manageable.

user_id	Username	anime_id	Anime Title	rating
20	vondur	36456	Boku no Hero Academia 3rd Season	9
20	vondur	1818	Claymore	8
20	vondur	1575	Code Geass: Hangyaku no Lelouch	8
20	vondur	1	Cowboy Bebop	9
20	vondur	5	Cowboy Bebop: Tengoku no Tobira	8
20	vondur	1535	Death Note	9
20	vondur	226	Elfen Lied	8
20	vondur	790	Ergo Proxy	8
20	vondur	356	Fate/stay night	8
20	vondur	10087	Fate/Zero	9

The excerpt shows consecutive records belonging to one user (user_id = 20). rating is an explicit integer score from 1 to 10.
Username and Anime Title can be recovered from user_id and anime_id. They are redundant and will be dropped at load time to limit memory use.

Figure A-6. Sample records from users-score-2023.csv

File 3: users-details-2023.csv (approximately 730,000 rows, one row per user)

The 16 fields are Mal ID, Username, Gender, Birthday, Location, Joined, Days Watched, Mean Score, Watching, Completed, On Hold, Dropped, Plan to Watch, Total Entries, Rewatched and Episodes Watched. The behavioural fields among these, in particular Mean Score, Total Entries and the status counts, will feed the rule-based recommendation module.

(a) Identity and profile fields

Mal ID	Username	Gender	Location	Days Watched	Mean Score
1	Xinil	Male	California	142.3	7.37
3	Aokaado	Male	Oslo, Norway	68.6	7.34
4	Crystal	Female	Melbourne	212.8	6.68
9	Arcane			30.0	7.71
18	Mad			52.0	6.27
20	vondur	Male	Bergen, Norway	73.1	8.06
23	Amuro		Canada	142.5	7.41
36	Baman	Male	Land of Ra...	272.1	5.90

(b) Viewing behaviour fields

Mal ID	Watching	Completed	On Hold	Dropped	Plan to Watch	Total Entries	Episodes Watched
1	1	233	8	93	64	399	8,458
3	23	137	99	44	40	343	4,072
4	16	636	303	0	45	1,000	12,781
9	5	54	4	3	0	66	1,817
18	1	114	10	5	23	153	3,038
20	11	94	11	2	20	138	4,374
23	20	298	5	19	50	392	8,565
36	27	1,144	11	55	338	1,575	16,309

All values are taken directly from the dataset; blank cells are missing in the source. Gender and Location have a high missing rate.
Columns not shown: Birthday, Joined, Rewatched. The rule-based module relies mainly on Mean Score, Total Entries and Dropped.

Figure A-7. Sample records from users-details-2023.csv

The dataset also ships a pre-merged wide table (final_animedataset.csv). It stores a large amount of duplicated content and is excessively large, so it will not be used in this project.

The three files are downloaded once from Kaggle as a fixed snapshot, so the data stays stable and the results stay reproducible. Two supplementary sources are planned on top of it: the Jikan API (v4), an open interface to MyAnimeList, for theme and demographic labels, character lists and sequel relations, none of which are present in the base dataset; and a second Kaggle dataset of user reviews as raw material for the chat module's knowledge base. Both are optional. If either proves impractical, the similar-title module falls back to the genre, studio and source fields already present, at reduced resolution. The labels themselves are contributed and moderated by the MyAnimeList community, which we treat as a reasonable stand-in for expert annotation, so no separate domain expert has been engaged at this scale.

A.4.2 Data Preparation

An initial read of the anime table shows two main quality issues: Studios is missing for 42.3% of records and Score for 37.0%. The two overlap heavily, and after removing unrated entries, adult-rated titles and entries without genre labels, around 12,000 titles remain usable, which is sufficient for the models we intend to build.

Neither field is imputed. Score is used as a filter and a ranking signal rather than a similarity feature, so unrated titles are excluded from the recommendable pool but kept in the database. Studio information is treated as a set that may be empty, with the remaining similarity weights redistributed, rather than filled with a placeholder.

The rating file is the largest of the three, so the current plan is to read it in chunks, keep only the three fields actually needed, and hold the result in a sparse structure; if memory remains the binding constraint, we would sample on the user dimension. The file carries no timestamp and no watching-status code, so sequential and implicit-feedback approaches are outside the scope of this project.

The dataset provides a single cover image per title and no frame-level images, so the screenshot identification listed as an additional feature would require a separate image source. This is being assessed, and the feature is scoped as optional accordingly.

These are planning decisions based on an initial inspection of the data, and will be adjusted as the implementation proceeds.

A.5 System Design

A.5.1 Technical Stack

- Frontend: HTML
- Backend: FastAPI (Python)
- Database: SQLite

A.5.2 MVP and Extensible

The system design is below, it may change after we get the suggestions from the proposal presentation.

Table A-1. System design and feature scope (MVP and additional features)

Scope	Feature	Description and Implementation
MVP	Log in and register	User can register and login to use our application.
MVP	Anime Browsing and Details	Search anime by title and display cover images, ratings, genres, synopses, episode counts, studios, and other basic information.
MVP	User Ratings and Favorites	Allow users to rate anime and save favorites, providing preference data for personalized recommendations.
MVP	Knowledge-Based Similar Anime Recommendations	Recommend similar titles on anime detail pages by comparing attributes such as genres, themes, demographics, studios, and episode counts.
MVP	Favorites-Based Recommendations with Rules	Build a preference profile from saved favorites and combine knowledge-based matching with filters such as genre, episode count, and minimum rating.
MVP	Collaborative Filtering Recommendations	Use **Item-based Collaborative Filtering** to identify similarities from user ratings and generate personalized candidates for the homepage.
MVP	Genetic Algorithm Recommendation Optimization	Apply a **Genetic Algorithm (GA)** to candidate titles and produce a Top-K list that balances relevance, personalization, diversity, and novelty.
MVP	RAG-Based Natural Language Search and Chat	Retrieve information from the anime knowledge base and use an LLM to answer questions, interpret viewing preferences, and provide basic recommendation reasons.
Additional Feature	Recommendation Explanations	Explain recommendations across modules using preference matches, anime attributes, collaborative filtering signals, and contributions to list diversity.
Additional Feature	Cold-Start Recommendations	Collect initial preferences through a short questionnaire or favorite-title selection, then introduce collaborative filtering as rating history grows.
Additional Feature	Viewing Plan Optimization	Use GA to select anime within the user's available viewing time while considering preferences, quality, and diversity.
Additional Feature	Anime Knowledge Graph	Add relationships involving characters, prequels, sequels, and other entities to support structured retrieval, relationship queries, and complex questions.
Additional Feature	Memorable Scene Search	Build a scene-level knowledge base to identify anime from vague scene descriptions and return characters, episodes, and plot details where supported by the data.
Additional Feature	Anime Screenshot Identification	Combine image encoding, multimodal retrieval, and model verification to identify anime screenshots and return candidate titles with relevant information.
Additional Feature	User Feedback and Adaptive Recommendations	Use additional feedback, such as likes, dislikes, and clicks, to continuously update user profiles and refine recommendations.

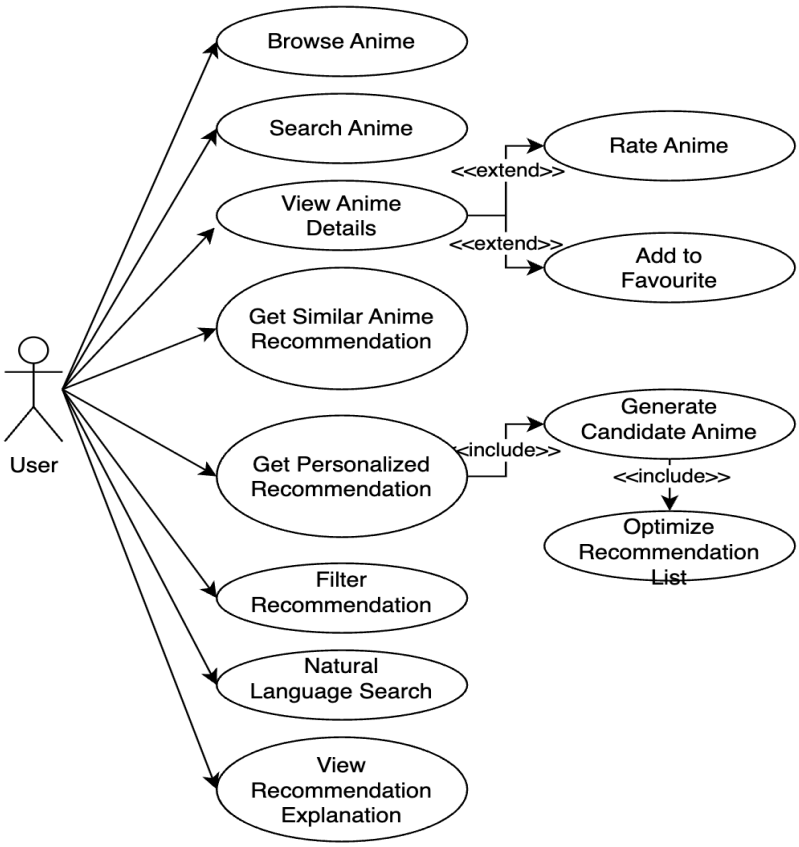


Figure A-8. System user case diagram

Appendix B Knowledge Use in the System

Mapped System Functionalities against the Knowledge, Techniques and Skills of the Modular Courses (MR, RS, CGS)

This appendix maps the functions of AniLogic to the knowledge, techniques and skills taught in the three modular courses of the practice module: Machine Reasoning (MR), Reasoning Systems (RS) and Cognitive Systems (CGS). **Table B-1** gives an overview of which course topics each function draws on. **Table B-2** gives the detail. There, the course topic is the knowledge and technique, and the right-hand column describes how we applied it in the system, which is the skill we practised. **Table B-3** reads the same mapping from the course side.

Course topics are cited by session. MR D1 to D4 and RS D1 to D5 are the teaching days of those courses, and CGS Day 1 is split into D1-AM (language) and D1-PM (vision cognition). F1 to F12 are the system functions. The parameters and behaviour described below are taken from the code in the project repository, and section numbers refer to the main report.

B.1 Overview

Table B-1. Course topics applied in each system function

System function	MR	RS	CGS
F1 Data preparation and knowledge base	D4 data pre-processing, missing values	D2 CRISP-DM, one-hot encoding; D1 data layer	D1-AM RAG indexing: splitting, embeddings
F2 Similar titles (Algorithm 1)	D2 analogical reasoning, frames; D4 logistic regression	D2 content-based filtering, similarity measures	D1-AM cosine similarity, TF-IDF versus embeddings
F3 Collection recommendation (Algorithm 2)	D2 rule-based reasoning; D4 tree ensembles	D1 rules + ML pattern; D3 cascade, two-tower	–
F4 Home page recall (Algorithm 3)	D2 nearest-neighbour reasoning; D4 neural networks	D2 item-based CF, matrix factorisation; D3 NCF, cold start	–
F5 Home page list optimisation (Algorithm 3)	D1 heuristic versus exhaustive search	D4 genetic algorithms; D3 re-ranking; D5 nature-inspired AI	–
F6 Chat: understanding the request (Algorithm 4)	D2 production rules, conflict resolution	D1 hard versus soft constraints	D1-AM NLU, slots, dialogue; D3 frames; D2 LLM reasoning limits
F7 Chat: retrieval and grounded answers (Algorithm 4)	D2 RAG versus classical reasoning	D5 RAG process; D1 validation, security; D3 conversational recommenders	D1-AM RAG, LLM challenges; D3 LLM reasoning failures
F8 Viewing plan (extension)	D1 constraints and search	D4 GA for scheduling; D1 search + optimisation	–
F9 Relation graph, scene search, screenshot identification	D3 knowledge graph traversal and its limits	D5 knowledge graph + RAG	D1-PM vision cognition, failure modes
F10 Explanations and user control	D3 explainable AI	D1 explainability levels, human oversight; D3 feedback	D1-PM design principles
F11 Architecture, security and reliability	–	D1 architecture, security, graceful failure	D3 cognitive architecture; D1-PM responsible deployment

System function	MR	RS	CGS
F12 Evaluation	D4 performance evaluation	D2 ranking metrics; D3 beyond accuracy	D2 model, task and system evaluation

B.2 Detailed mapping

Table B-2. Detailed mapping: course knowledge and technique, and how each is applied in AniLogic

Course knowledge and technique	Application in AniLogic (skill demonstrated)
F1 Data preparation and knowledge base *Chapter 6 · pipelines/, app/retrieval/documents.py*	
RS D2 Data-driven reasoning: modelling dataset, one-hot encoding, the CRISP-DM process	Six raw CSV files were turned into one data package by scripted, re-runnable steps. About 24 M ratings were read in chunks, explicit titles and titles without a public score were removed (**Table 2**), and multi-valued genre, theme and studio fields became tag sets used as multi-hot features. The order follows CRISP-DM: data understanding, preparation, modelling, evaluation.
RS D1 Data layer of a reasoning system and its failure modes	When the Jikan API stayed down for more than 24 hours, AniList became the main metadata source. Every filled value keeps a source label, so experiments can leave out inferred values, and a missing value stays missing and never passes a hard filter (Section 11.2).
MR D4 Data pre-processing and missing values (workshop 4.3.2)	Missing scores, studios and demographics are not imputed. They are passed on as missing values (LightGBM handles them natively), and the explanation layer tells the user which fields could not be compared.
CGS D1-AM RAG pipeline: document splitting, embeddings, index	The retrieval corpus holds 64,914 documents: attributes, synopsis chunks of 150–350 tokens with 40-token overlap, relation facts, character lists and 300 sourced scenes. Each document records its source type and locator and is indexed twice, in SQLite FTS5 for keywords and as BGE vectors for semantic search.
F2 Similar titles on the detail page (Algorithm 1) *Section 7.1 · app/recommendation/similarity/*	
MR D2 Analogical (similarity-based) reasoning; frames as slots and fillers	A title is a frame with five typed slots. Genre and theme are compared as sets (Jaccard), demographic and studio by exact match, and length by $1 / (1 + \lvert\Delta \text{episodes}\rvert / 12)$. The score is 0.35 genre + 0.30 theme + 0.15 demographic + 0.10 studio + 0.10 length. A missing slot never matches, and each Top-5 result shows the shared attributes as its reason.
RS D2 Content-based filtering; similarity and distance measures; semantic representation of content	Route A is the content-based baseline. We also built route B (logistic regression), route C (BGE embeddings compared by cosine) and route D (0.6 tag + 0.4 semantic) and compared the four on 1,000 targets. None of B, C or D passed the replacement gate fixed in advance, so the interpretable route A is served.
MR D4 Supervised learning, regularisation and generalisation	Route B feeds the same five similarities to a logistic regression. The regularisation strength is chosen on validation data, and its use is controlled by a replacement rule written before any test result existed.
CGS D1-AM Cosine similarity; TF-IDF versus neural embeddings	Route C is the embedding side of this trade-off. Semantic neighbours can appear without any tag overlap, but on our benchmark, whose labels are built from metadata overlap, it did not beat the explicit tag formula (Section 7.1).
F3 Collection recommendation (Algorithm 2) *Section 7.2 · app/recommendation/constraints.py, algorithm2/*	
MR D2 Rule-based reasoning: IF–THEN rules over facts, rule engines	Each hard filter (keyword, minimum score, maximum episodes, already saved, explicit content) is a rule with an id and a source, evaluated as a mask over the catalogue before any scoring. A missing value never satisfies a filter. If no title passes, the reply names the rule that removed everything and offers relaxations, and nothing is relaxed unless the user chooses it.
RS D1 Hybrid pattern “rules + ML”; constraint reasoning (find a valid answer)	This is the layering of the fraud example in the lecture: the learned scorer decides what is likely, and the rule layer decides what is allowed. The scorer only ranks titles that already satisfy every rule, and the evaluation found zero rule violations (**Table 26**).

Course knowledge and technique	Application in AniLogic (skill demonstrated)
RS D2 Content-based user profile (the "user prototype")	The collection is summarised as a profile: the share of saved titles carrying each genre, theme, demographic and studio tag, the median episode count and the ten most frequent tags. Candidate titles are compared with this profile.
MR D4 Ensemble learning with decision trees; feature importance	Scorer B1 is a LightGBM gradient-boosted tree model over eight profile features. On 3,000 held-out users it raised HR@8 from 0.021 (formula A0) to 0.132 and passed every check that was fixed before training.
RS D3 Cascade of retrieval and ranking; two-tower models with negative sampling	The rule layer narrows the catalogue to eligible titles and the scorer ranks within them, a small version of the retrieval-then-ranking funnel. C0, a content two-tower network trained on one positive and four sampled negatives, was built as an alternative. It reached HR@8 0.113 against 0.132 for B1 and is not served.
F4 Home page: candidate recall (Algorithm 3) *Section 7.3 · app/recommendation/recall/, services/recommend.py*	
RS D2 Collaborative filtering: item-based CF, Pearson and adjusted similarity, rating normalisation, scalability	Item-based CF with adjusted cosine similarity: ratings are centred by each user's mean, at least three co-raters are required, the similarity is shrunk by c / (c + 10), and 150 neighbours are kept per item. Item–item similarities are computed offline in blocks with sparse matrices, which is what makes the 24 M-rating matrix workable on one laptop. A prediction is the similarity-weighted average of the user's own centred ratings.
MR D2 Analogical reasoning: nearest-neighbour voting by similarity	ICF is a nearest-neighbour method over items. A title is scored from the user's ratings of its most similar titles, so each recommendation can show the rated title that supports it.
RS D2 Matrix factorisation: SGD, regularisation, handling new users	Biased matrix factorisation (32 factors, regularisation 0.04, early stopping) with a fold-in step for new users was compared with ICF as the recall engine. ICF was kept because it was the best personalised engine on the ranking protocol and serves a new user without retraining (Section 8.5).
RS D3 Neural collaborative filtering; hybrid switching and cold start	NCF (GMF and MLP branches fused into one score) was built as the deep alternative. It uses user IDs only, so it cannot serve an unseen user. Cold start is handled by a switching rule: with no usable ratings the list comes from a quality-controlled popular and content ranking, with fewer than eight ratings ICF and content scores are blended with weight n / 8, and from eight ratings ICF alone is used.
MR D4 Neural networks: MLP, back-propagation, overfitting	The MLP branch of NCF is trained by back-propagation with dropout 0.2, weight decay and early stopping on validation data.
F5 Home page: list optimisation with a genetic algorithm (Algorithm 3) *Section 7.3 · app/recommendation/rerank/*	
RS D4 Genetic algorithms: encoding, fitness, selection, crossover, mutation, rates, stopping criteria	A chromosome is ten distinct titles from the Top-20 recall window. Population 64, tournament selection of size 3, elitism 2, set crossover at 0.8 (shared genes are kept, the rest filled from both parents), mutation at 0.2 (replace one gene), repair to ten unique genes, and a stop after 20 stale generations or 250 ms. Fitness = 0.4 relevance + 0.3 diversity + 0.2 novelty + 0.1 freshness.
RS D4 Parameter trade-offs; handling constraints; when to use a GA	The offline benchmark used random-key encoding. For the online page we switched to set-based encoding with a wall-clock budget so that response time stays predictable (Section 12.3).
RS D3 Re-ranking after ranking; success is more than accuracy (diversity, serendipity)	The GA is the re-ranking stage after ICF recall. It gives up 1.5 points of Hits@10 for clearly more diverse, novel and fresh lists, and a separate labelled popular list serves users who mainly want accuracy (Section 9.2).
MR D1 Heuristic search compared with exhaustive search; combinatorial explosion	Choosing 10 of 20 titles is a combinatorial search problem. The GA is a heuristic for it, and exhaustive enumeration of small pools is used to check how close the GA gets to the optimum of its own objective.
RS D5 Four AI paradigms; choosing the right method	Symbolic rules, machine learning and nature-inspired search all appear in the home page pipeline. The lecture recommends nature-inspired methods for optimisation problems where traditional methods struggle, which describes the list objective.

Course knowledge and technique	Application in AniLogic (skill demonstrated)
F6 Chat: understanding the request (Algorithm 4) *Sections 7.4, 8.2 · app/chat/intent.py, conversation.py*	
CGS D1-AM NLU pipeline: intent, slots, resolving references; domain rules for slot values	A hand-written parser turns a request into typed slots: tags to include or exclude, soft preferences, episode limits, score range, start-year range, studio, type and a reference title. “At most 12” is inclusive and “under 12” exclusive, “no X” is a hard exclusion, and “not too long” stays a soft preference instead of becoming a silent 12-episode filter. Titles are resolved by alias, prefix search and then fuzzy match. The lecture notes that domain rules remain useful for non-standard slot values.
CGS D1-AM Dialogue management: wait and observe, seek information, act and reply	Contradictory limits (“at least 50 and at most 3 episodes”) or an ambiguous title lead to a clarification question. A follow-up such as “at most 6 episodes” refines the stored constraint state, and a new request resets it.
MR D2 Rule matching and conflict resolution (most specific rule first)	When several cue patterns match, the longest one wins (“science fiction” over “science”), which is the “most specific rule” strategy on the conflict-resolution slide.
CGS D3 Frames: slots, fillers and defaults; thematic roles	The constraint state is a frame whose slots are filled by the parser and updated by the dialogue. A studio named after a strong cue but missing from the catalogue is reported as unknown and never guessed.
CGS D2 Reasoning abilities of LLMs are often overestimated	Numeric limits are parsed by rules and checked against the catalogue instead of being left to an LLM. The LLM is optional and only words an answer after it has been validated (Section 8.2).
F7 Chat: retrieval and grounded answers (Algorithm 4) *Section 7.4 · app/retrieval/, app/chat/grounded_answer.py*	
CGS D1-AM RAG: retrieve chunks, build the prompt, generate; TF-IDF versus embeddings	Hard constraints first select the eligible titles. Retrieval over them uses two channels, BM25 over FTS5 (top 50) and BGE dense cosine (top 50), fused by reciprocal rank fusion with $k = 60$. A title scores as its best document, so one title cannot fill the list. If the dense channel fails, chat continues on keyword search and says so (Section 9.4).
RS D5 RAG process flow (query, retrieval, augmentation, generation, post-processing); RAG versus fine-tuning	The pipeline follows the five steps, and its last step is a validator. New knowledge is added by rebuilding the index and not by retraining a model (Section 8.1).
MR D2 RAG as classical reasoning; rule-based versus pure LLM versus RAG	Section 7.4 compares a template reply (T), an LLM alone (N) and retrieval-augmented generation (R), the same three rows as the comparison table in the lecture. R was chosen: the judge score rose from 3.50 to 4.05, and the stated episode count matched the knowledge base in 86% of replies against 39%.
CGS D3 LLM reasoning failures; validity and soundness	The LLM sees only delimited evidence and verified facts and must return JSON. Item keys must be candidates that passed the constraints, cited document ids must have been in its context, and every number it states is checked against the database. One repair round is allowed. After that the local template answer is shown with a warning, and unsupported details become “unknown”.
RS D1 Explainability and validation workflow (evidence, reasoning, explanation, validation); instruction isolation, output checks	Evidence is given to the model as delimited, untrusted text, and every claim carries a document id, which is the evidence-to-validation chain of the lecture. Chat answers list their sources, and no fabricated episode number appeared in 30 unanswerable episode questions (Section 9.5).
RS D3 Conversational recommenders and RAG	The chat page follows the conversational-recommender pattern of Day 3, in which the model works against an external catalogue to prevent hallucinated recommendations.
CGS D1-AM Challenges of LLM-powered applications: privacy, cost, latency	At most two model calls per request with a 30 s timeout, cost reported when it is known, chat text kept out of the logs by default, and a local mode that needs no model at all.

Course knowledge and technique	Application in AniLogic (skill demonstrated)
F8 Viewing plan (extension) *Sections 8.2, 9.2 · app/planning/*	
MR D1 Problem formulation as search under constraints; combinatorial explosion; heuristic versus exhaustive search	Given a time budget, the planner chooses titles and an order to watch them, a knapsack-style problem. Budget, title cap, must-include titles and "prequel first" are hard constraints, every candidate plan is repaired, and the final plan is re-validated independently. For pools of up to 14 units the exact optimum is computed, so the gap of the GA is known.
RS D4 GA for scheduling and TSP-style problems; handling constraints	In complete mode the chromosome has one bit per unit (a title plus its unmet prequels), and in partial mode each gene is the number of next episodes of a title. Population 60, 120 generations, 1.5 s budget. The objective is 0.45 relevance + 0.20 quality + 0.20 diversity + 0.05 novelty + 0.10 utilisation. Episodes are then packed into daily slots, and a movie is never split.
RS D1 Hybrid pattern "search + optimisation"	Same structure as the delivery-route example: business constraints, a search over candidate plans and an optimisation step that returns the best feasible one. In the evaluation of 600 requests no plan was infeasible (**Table 26**).
F9 Relation graph, scene search and screenshot identification (extensions) *Sections 6.6, 10.2 · app/graph/relations.py, app/multimodal/image.py*	
MR D3 Knowledge graphs: traversal (one-hop, multi-hop, BFS/DFS) and its disadvantages	Sequel and prequel edges carry a source reference. Traversal is bounded and cycle-safe, only "prequel" counts as a viewing prerequisite and only in release order, and a missing edge is reported as unknown and not as "no prerequisite", because the lecture points out that graphs are never complete. Relations cover 10,883 of 23,363 titles, and the status page reports this coverage.
RS D5 Knowledge graph + LLM; graph-enabled RAG	Relation facts are packed into retrieval documents, so a question about a sequel is answered from a sourced edge and not from the model's memory.
CGS D1-PM Common failure modes (hallucinated descriptions, overconfident answers); uncertainty-aware design	Scene records (300 scenes for 100 titles) are searched with both retrieval channels, and an episode is named only when the record ranks first in both. This cut wrong scene claims from 13 of 60 to 0 of 60 (Section 10.4).
CGS D1-PM Vision cognition pipeline (seeing to knowing); visual grounding	Screenshot identification encodes an uploaded frame with a CLIP image encoder and searches a frame gallery. No acceptance threshold is set until one has been calibrated on validation data, so every hit is shown as an unverified candidate. The function is switched off in the real-data build because no frame images exist (Section 10.2).
F10 Explanations and user control *Section 8.3 · app/recommendation/explanations.py, services/user_data.py*	
MR D3 Explainable AI: intrinsic and natural-language explanations	Every recommendation carries reasons computed from the model's own inputs: shared tags, the user's own supporting rating with its similarity and number of co-raters, and what the title adds to the list. An LLM never writes a reason.
RS D1 Explainability at several levels; human oversight and accountability	Data level: the supporting ratings. Process level: chat shows the conditions it understood. Output level: plain-English reasons. Governance level: every run is stored with its configuration and dataset version. Users can mark a title as not interested or undo a rating, and the next list reflects it immediately.
RS D3 Explicit feedback; a negative signal is not a positive seed	Low ratings and not-interested titles are never used as seeds for recommendations, and every rating increments the profile version so a stale list is never served (Section 8.4).
CGS D1-PM Design principles: explainable, uncertainty-aware, human-in-the-loop	Low metadata coverage is shown as a note on the card (**Figure 20**), and a request the data cannot answer receives a stated reason in place of a guess.

Course knowledge and technique	Application in AniLogic (skill demonstrated)
F11 Architecture, security and reliability *Chapters 8, 10 · app/core/security.py, app/services/serving.py*	
RS D1 Reasoning systems as architectures; design principles (ground, validate, monitor, fail gracefully)	One FastAPI process, with an offline pipeline that builds versioned bundles of data, models and indexes recorded with SHA-256 hashes. A failed check degrades one capability (the collection page falls back to the formula, chat to keyword search), and the status page shows the state of every capability.
RS D1 Security and ethical concerns: access control, output checks, accountability	Passwords are hashed with argon2id, session and CSRF tokens are stored only as hashes, and every query is scoped to the session user. A second account cannot see another account's data (Section 10.4).
CGS D3 Cognitive system architecture: memory, reasoning, learning, reaction	The parts of the architecture slide line up with ours: SQLite and the knowledge base are memory, the rule engine, the GA and the validators are reasoning, offline training is learning, and request handling is reaction.
CGS D1-PM Responsible deployment questions; design principles (privacy-conscious, continuously evaluated)	Uploaded images are not kept by default, adult titles are excluded by one catalogue rule so the policy sits in one place, and every reported number comes from a stored evaluation run with its configuration.
F12 Evaluation *Chapters 7, 9 · evaluation/*	
CGS D2 Performance evaluation from model to task to system	We evaluate at the three levels of the lecture: algorithm benchmarks in Chapter 7, task-level Hits@5, MRR@10 and task success (**Table 26**), and system-level efficiency and cost, robustness and explainability (Sections 9.3 to 9.5).
RS D2 Ranking metrics (P@K, NDCG@K), MAE; overfitting and regularisation; CRISP-DM evaluation	Hits@K, Recall@K, NDCG@K and MRR for ranking, MAE and RMSE for rating prediction, and diversity, novelty and catalogue coverage for list quality. Hyper-parameters such as the ICF damping and the logistic-regression C are tuned on validation users only.
MR D4 Training, validation and test data; generalisation; no single metric is best	Frozen splits (users are split 70/15/15 for Algorithm 2), selection rules written before the test run, the test set scored once, and 95% bootstrap intervals with paired differences between methods.
RS D3 Measuring success beyond accuracy	On the home page a popularity list reaches Hits@10 0.137 against 0.062 for ICF but covers about 30 times less of the catalogue, so we report coverage and novelty next to accuracy (Section 9.2).

B.3 Course topics applied, by session

Table B-3 lists, for each course session, the topics that the system puts to use and the functions in which they appear.

Table B-3. Course sessions and the topics applied in the system

Session	Topics applied	Functions
MR D1	Search as reasoning: heuristic versus exhaustive search, combinatorial explosion, constraint formulation	F5, F8
MR D2	Rule-based reasoning and conflict resolution; analogical reasoning (kNN, case-based); frames and knowledge representation; RAG compared with classical reasoning	F2, F3, F4, F6, F7
MR D3	Knowledge graph traversal and its limits; explainable AI	F9, F10
MR D4	Supervised learning, tree ensembles, neural networks, performance evaluation, missing values	F1, F2, F3, F4, F12
RS D1	Hybrid patterns (rules + ML, search + optimisation, RAG + LLM), constraint reasoning, explainability levels, human oversight, security, graceful failure	F3, F7, F8, F10, F11

Session	Topics applied	Functions
RS D2	CRISP-DM and data preparation, content-based filtering, item-based CF, matrix factorisation, relevance and diversity, ranking metrics	F1, F2, F3, F4, F12
RS D3	Hybrid recommenders (cascade, switching, weighted), cold start, NCF, two-tower, re-ranking, conversational recommenders, success beyond accuracy	F3, F4, F5, F7, F10, F12
RS D4	Genetic algorithms: encoding, fitness, selection, crossover, mutation, parameters, constraint handling, scheduling	F5, F8
RS D5	Four AI paradigms and method selection, RAG process, knowledge graph + LLM	F5, F7, F9
CGS D1-AM	NLU with intents and slots, dialogue management, TF-IDF versus embeddings, RAG pipeline, challenges of LLM applications	F1, F2, F6, F7
CGS D1-PM	Vision cognition pipeline, failure modes, responsible deployment, design principles	F9, F10, F11
CGS D2	Evaluation from model to task to system; limits of LLM reasoning	F6, F12
CGS D3	Cognitive architecture, frames and thematic roles, LLM reasoning failures, validity and soundness	F6, F7, F11

B.4 How the three courses fit together

The courses play different roles in AniLogic. RS gives the overall design: the hybrid pattern each page follows (rules plus ML on the collection page, search plus optimisation for the GA and the viewing plan, RAG for chat), the recall-then-re-rank structure of the home page, and the habits of validating, explaining and degrading safely. MR provides the reasoning components inside that design, namely rule-based filtering, similarity-based reasoning, tree models, graph traversal and the search view of optimisation. CGS provides the language and perception side, which covers slot filling, dialogue state, RAG and the validation of LLM output, together with the evaluation levels and deployment questions that we applied to the whole system.

Several functions combine all three courses. The chat page uses MR rules for parsing, the RAG process and validation workflow of RS, and the NLU and grounding techniques of CGS. The viewing plan combines the constraints and search of MR with a genetic algorithm from RS, and its prerequisite ordering relies on the knowledge graph ideas of MR D3.

Appendix C User Guide and Installation

AniLogic Installation and User Guide

Use this guide to install AniLogic, find anime and keep a viewing list. It also explains how to use similar-title recommendations, collection recommendations, home recommendations and Chat to find, then turn your choices into a viewing plan.

If AniLogic is already running, open the address supplied by the installer and start at Section C.3. The screenshots use the English interface and the real catalogue. Your results will vary with the active catalogue, ratings and preferences.

C.1 Install AniLogic

Prepare the computer

Install Python 3.11 to 3.13, Git, Git LFS and uv. Python 3.12 is a suitable choice. Use a current browser such as Chrome or Edge. Allow several gigabytes of free space for the project, packages and downloaded models, and keep an internet connection available during installation.

Download tools from python.org/downloads, git-scm.com/downloads, git-lfs.com and docs.astral.sh/uv/getting-started/installation. On Windows, reopen PowerShell after installation so the commands are available.

Download the project and install packages

1. Open PowerShell in the folder where you want to keep AniLogic.
2. Run the following commands in order. Wait for each command to finish before entering the next one.

```
git lfs install

git clone --branch main https://github.com/gaoxinghe1024/IRS_practice-
module.git anilogic

cd anilogic

git lfs pull

uv sync --frozen --extra ml --extra llm
```

Git LFS downloads the catalogue databases and other large project files. Keep this step even if the source files have already downloaded. The supplied project includes prepared catalogues, so a normal installation does not require rebuilding the data.

> **Check:** the last command should finish without an error and create a .venv folder inside anilogic. If a command is not recognised, finish installing that tool, reopen PowerShell and try again.

C.2 Start and stop the system

1. In PowerShell, open the anilogic project folder. This is the folder containing pyproject.toml.
2. Select the real catalogue and start the server with the commands below.

```
$env:ANILOGIC_DATASET_MODE = "real"
uv run --no-sync python -m uvicorn app.main:app --host 127.0.0.1 --port 8000
```

3. When the terminal reports that the application has started, open http://127.0.0.1:8000 in your browser.
4. Open Discover and search for Steins;Gate. Open a result to check that the catalogue is available. Use Status in the footer if a feature reports a problem.

Keep the terminal open while using AniLogic. To stop the server, return to that terminal and press Ctrl+C. To start it again, repeat the start commands. Saved accounts, ratings, collections and plans remain in the selected catalogue database.

Use the demonstration catalogue

To use the fictional demonstration catalogue, stop the server, change the environment setting below and run the same server command. The two catalogues keep separate accounts and saved activity.

```
$env:ANILOGIC_DATASET_MODE = "synthetic"
```

In synthetic mode, sign in as demo_mystery, demo_action, demo_healing or demo_scifi with the password anilogic-demo. For the real catalogue, create your own account.

On macOS or Linux, use this start command from the project folder instead of the PowerShell commands:

```
ANILOGIC_DATASET_MODE=real uv run --no-sync python -m uvicorn app.main:app --
host 127.0.0.1 --port 8000
```

Choose where to go

Table C-1. Tasks and Page Navigation

Task	Open
Find a title or browse the catalogue	Discover — Section C.4
Record ratings, progress or saved titles	A title page — Section C.5
Find something similar to a title	Similar titles on a title page — Section C.6
Recommend from a group of saved titles	Collections — Section C.7
See recommendations for your account	Home — Section C.8
Describe what you want to watch	Chat to find — Section C.9
Fit a watchlist into available time	Viewing plan — Sections C.10 and C.11
Return to saved work or resolve a problem	History or Status — Section C.12

C.3 Create an account and sign in

You can use Discover and open title pages without signing in. Sign in to use your home recommendations, keep ratings and progress, manage collections, chat and save viewing plans.

Create an account

1. Select Create account at the top right of the page.
2. Enter a Username of 3–32 characters. Use letters, numbers, underscores, hyphens or full stops. Enter a Password of at least 8 characters.
3. Select Create account. Once the account is created, AniLogic signs you in and opens Tell us what you like.

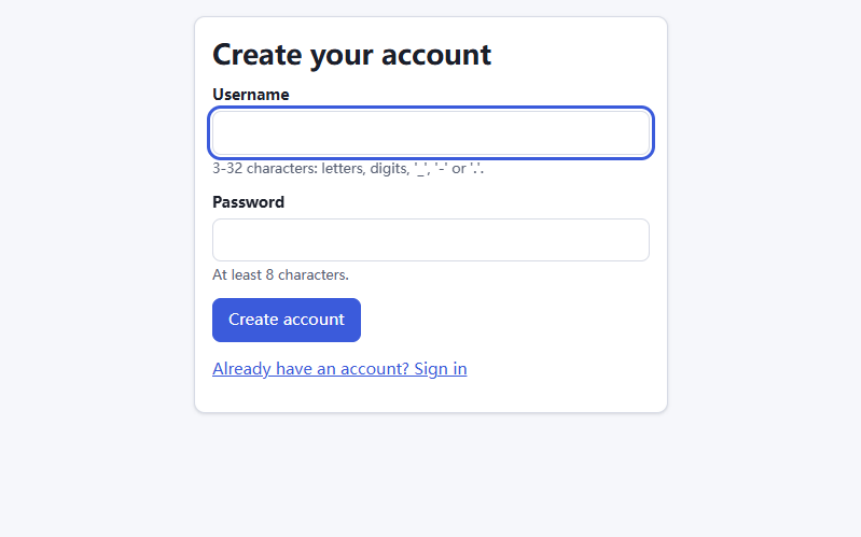


Figure C-1. The account form shows the username and password requirements.

Set your initial preferences

1. On Tell us what you like, select genres or themes you enjoy. You can select more than one.
2. For titles you have seen, choose your own rating from 1 to 10. Leave unfamiliar titles unrated.
3. Select Save and continue to open Home. To do this later, select Skip for now. You can return through Set preferences on Home or update preferences in Profile.

Sign in again or sign out

Select Sign in, enter the same username and password, then select Sign in on the form. Your username appears at the top right when you are signed in. Use Sign out there when you finish on a shared computer.

If registration reports that the username is taken, choose another name. If sign-in fails, check your spelling and password. The current interface does not offer a password reset; contact the system administrator if you cannot regain access.

C.4 Find an anime in Discover

Use Discover when you know a title, or when you want to browse by genre, length or score.

1. Select Discover in the top navigation.
2. In Search titles, enter a title or part of a title. For example, enter Steins;Gate. English, romaji and other recorded title names can be searched.
3. Select Search, then select a result title to open its details.

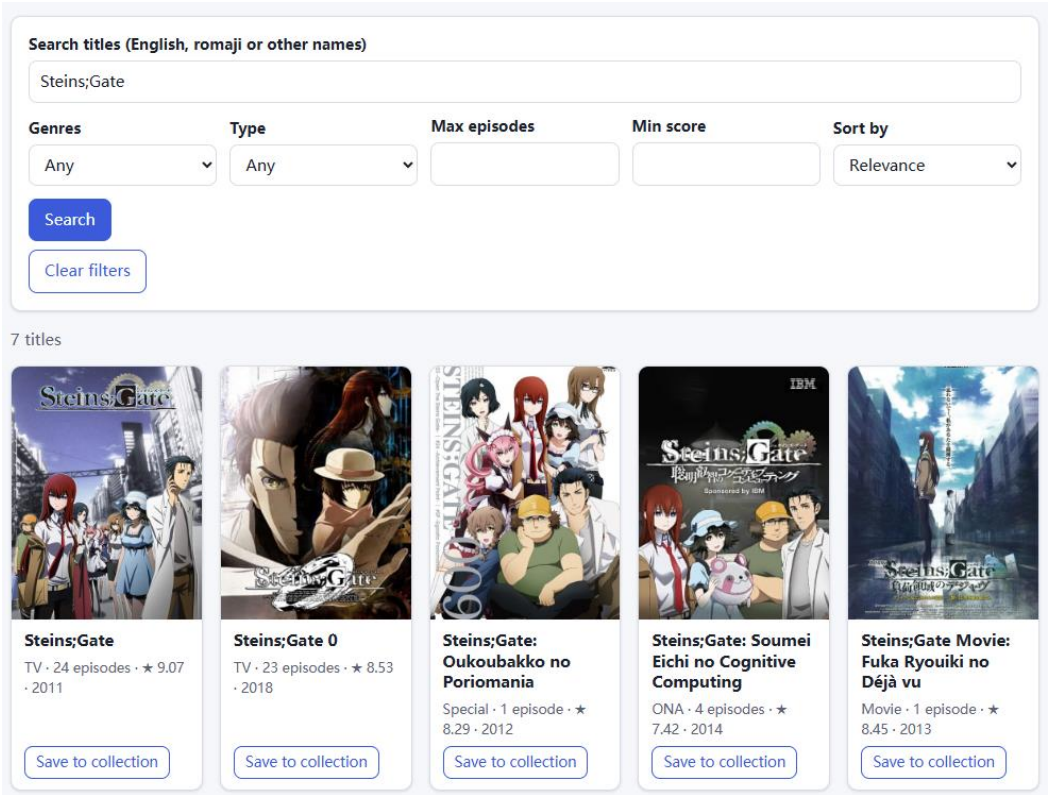


Figure C-2. A title search in Discover. Select a title to open its page.

Narrow the results

Choose Genres or Type, enter Max episodes or Min score, and select Search again. For example, set Max episodes to 12 for shorter series. Use Sort by to order the results by Relevance, Popularity, Score, Newest or Title.

Use Previous and Next if there is more than one page. Select Clear filters to start a new search. If no results appear, remove one filter or try a shorter title name.

Read the cards: the star value is the catalogue score, not your personal rating. “Unknown” or “No score” means the catalogue does not contain that value.

C.5 Rate a title and record your progress

Open a title from Discover or a recommendation. Its page contains the synopsis, available details and your own controls. Sign in before recording any changes.

1. Under Your rating, select a number from 1 to 10. The rating saves immediately. Select Clear rating to remove it.
2. Under Watch status, choose Plan to watch, Watching, Completed, On hold or Dropped. Use Not tracked if you do not want to track the title.
3. Enter the number of episodes you have watched and select Save. Check the saved status and episode count before leaving.

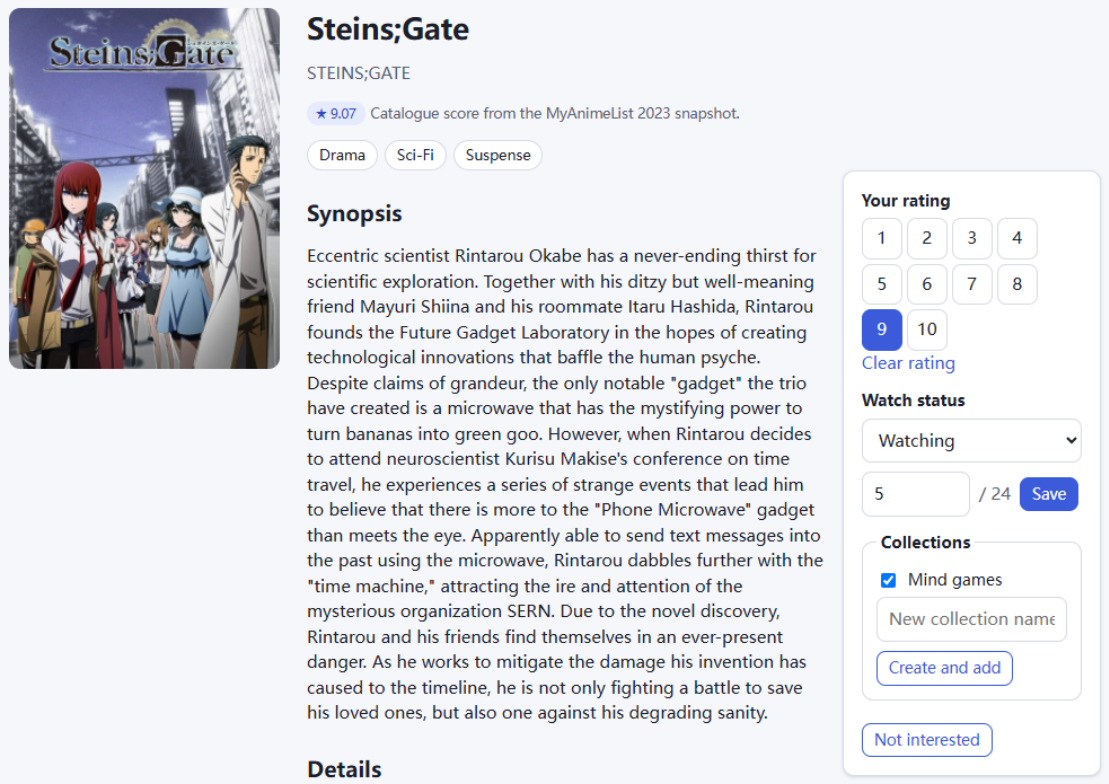


Figure C-3. A title page with a personal rating, watch progress and collection controls.

Save the title to a collection

In Collections on the title page, tick an existing collection to add the title; untick it to remove the title from that collection. These changes save immediately. To create a collection here, enter a name in New collection name and select Create and add.

On recommendation cards, select Save to collection and choose a collection from the menu. A title can belong to several collections. Use collections to keep separate lists, such as “Weekend choices” and “Mind games”.

Keep watch progress up to date before making a viewing plan. The plan can then take your remaining episodes into account.

C.6 Find titles similar to one you like

Start from a title you already enjoy. Similar titles helps you find other works with related features, such as shared genres or a similar length.

1. Find a title in Discover and open its page.
2. Scroll down to Similar titles, below the title details and source information.
3. Read the recommended titles and the short reasons on each card. Select a title to inspect its synopsis and length.
4. Select Show more to expand the initial five results to up to ten. Select Show fewer to return to the shorter list.
5. When signed in, tick Hide titles I rated or completed if you want to focus on titles you have not already rated or finished. Use Save to collection to keep a result.

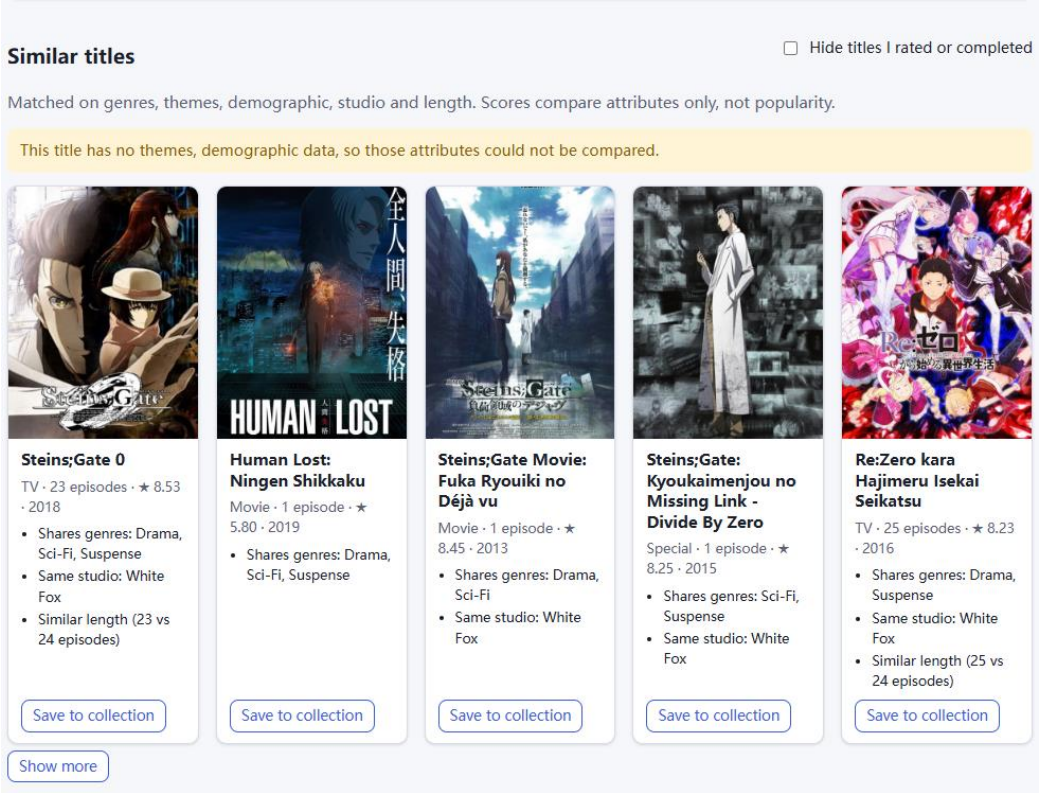


Figure C-4. The Similar titles results beneath Steins;Gate, including reasons and Show more.

A notice about missing themes or other details means those details were unavailable for comparison. You can still open the results and check whether they suit you. If the list is too short, clear the hide option or try another starting title.

C.7 Get recommendations from a collection

Use a collection when you want recommendations based on several saved titles, such as a group of mysteries you enjoyed.

Create and fill a collection

1. Open Collections. Enter a Name, optionally add a Description, and select Create collection.
2. Find titles in Discover and add them using the title-page controls or Save to collection. Return to Collections and open the collection name.

Choose recommendations

1. Check Titles in this collection. Add a title first if the collection is empty.
2. Scroll to Recommended from this collection. For an example search, enter 8 in Minimum score and 12 in Maximum episodes, then select Update recommendations.
3. Read the reasons on the result cards. Open a title to check its details, or save it to a collection.

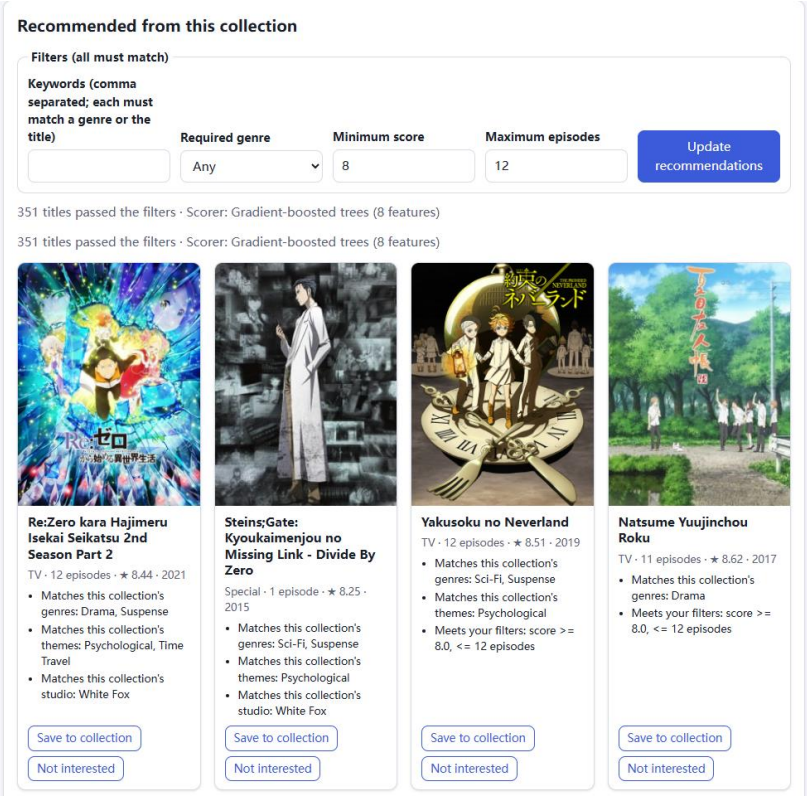


Figure C-5. Collection recommendations after setting a minimum score of 8 and a maximum of 12 episodes.

All filters must match. Required genre limits results to that genre. Each comma-separated keyword must match a genre or the title. If no titles qualify, revise a filter and select Update recommendations. Use an Apply: … suggestion only if you accept the change it describes.

Use Remove beside a saved title to take it out of this collection. On Your collections, use Rename to change a name or Delete to delete the collection after confirming.

C.8 Use your home recommendations

Home recommends titles for your account. Sign in, select Home, then look under Recommended for you.

1. Read the short reasons under each title. Open a title to inspect it, or select Save to collection to keep it.
2. If a title does not interest you, select Not interested. Use the displayed Undo action if you selected it by mistake.
3. Rate more titles you have actually seen and return to Home. Your saved ratings give the system more information for later recommendations.

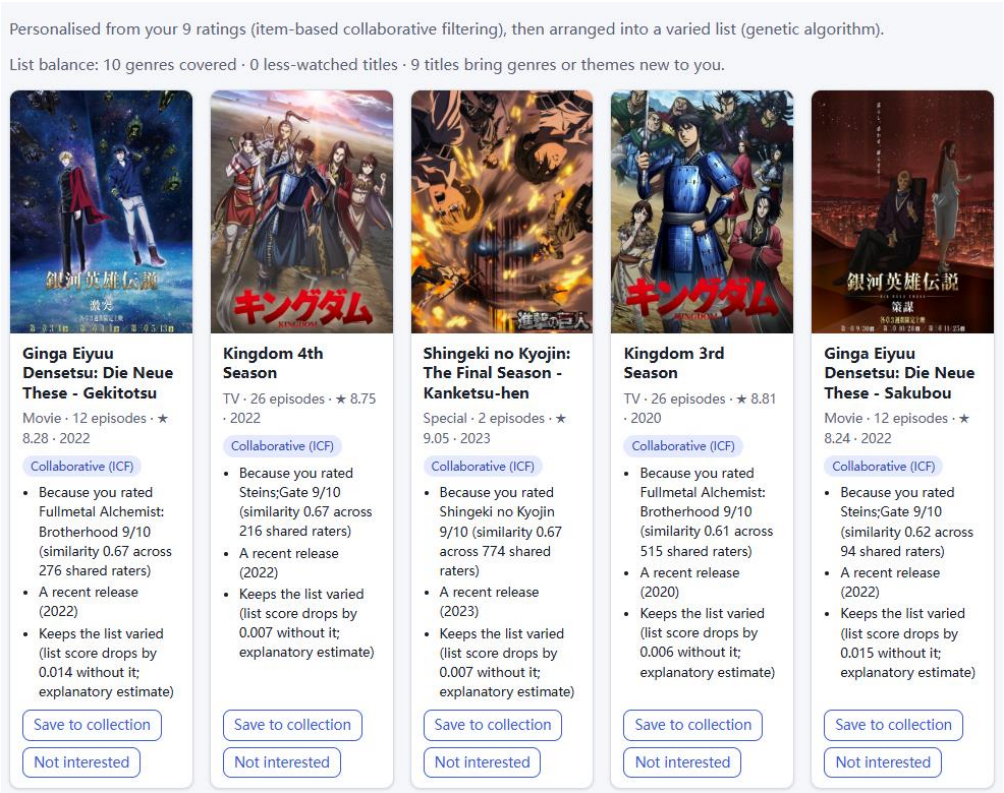


Figure C-6. Home recommendations for an account with saved ratings.

A new account may see Starter picks or Warming up before Personalised recommendations appear. Use Set preferences if prompted. Explore popular is another way to browse while you build your rating history.

Change your preferences or undo feedback

Open Profile. Under Preferences, select your liked genres and themes, choose a Preferred length, and set Hide titles I rated or completed as needed. Select Save to keep these changes.

In Your ratings, select Remove to delete a rating. To change its value, open the title and select a new rating. In Not interested, select Undo to make a title eligible for recommendations again. Excluding a title does not remove it from Discover.

C.9 Find anime by describing what you want

Use Chat to find when it is easier to describe your preferences than set filters.

1. Open Chat to find and select New chat to begin with no previous conditions.
2. Enter a comedy with at most 12 episodes and no horror, then select Send. Enter also sends a message; use Shift+Enter for a new line.
3. Check Understood conditions beneath the answer. For this request, look for Comedy, No Horror and a limit of 12 episodes.
4. To shorten the list further, send at most 6 episodes in the same chat. Check that the episode limit changes while the other conditions remain.
5. Open a result title to read more, or select Save to collection. Expand Sources used if you want to inspect the supporting information.

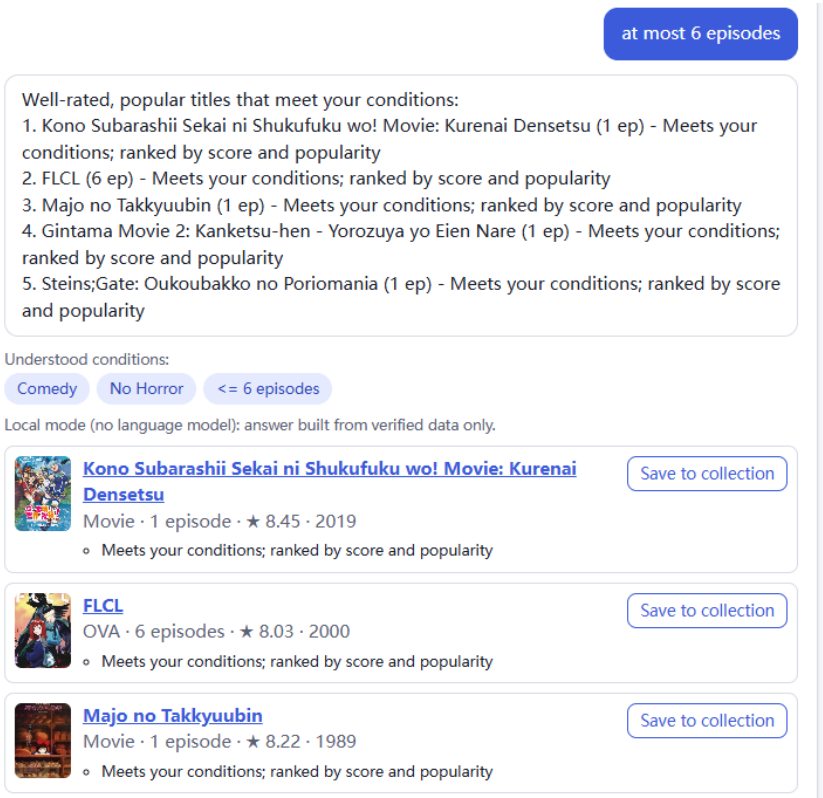


Figure C-7. A follow-up changes the episode limit to 6 while keeping Comedy and No Horror.

Use one follow-up at a time if the conditions are misunderstood. Select New chat before switching to an unrelated request. Reopen a previous conversation under Your chats or in History.

Select Save these conditions to my profile only if you want the liked or excluded genres and themes to influence future recommendations. Check Profile afterwards. A Local mode label is normal when an external language service is not configured; you can still search for titles.

If an answer lacks the detail you need, rephrase the request or open the title page. Scene and image searches depend on the data available in this installation.

C.10 Create a viewing plan

Viewing plan builds a watchlist within the time you have available. Update your watch progress first if you want to continue titles you have started.

Preview a weekend plan

1. Select Viewing plan. Enter 15 in Time budget (hours), 0 in + minutes, and 3 in Max titles.
2. Choose a Variety setting: Similar styles, Balanced or Different styles. Leave genre and score limits empty for the first attempt, or set the limits you need.
3. Under How to watch, keep Finish every title (default) for complete works. Choose Allow partial titles (next N episodes) if you are willing to watch only part of a series.
4. Keep Include required prequels selected if you want known prerequisites included in the time budget. Enter a Plan name, such as Weekend watchlist.
5. Select Preview plan. Review the titles, episode ranges, minutes and any Watch first information before saving.

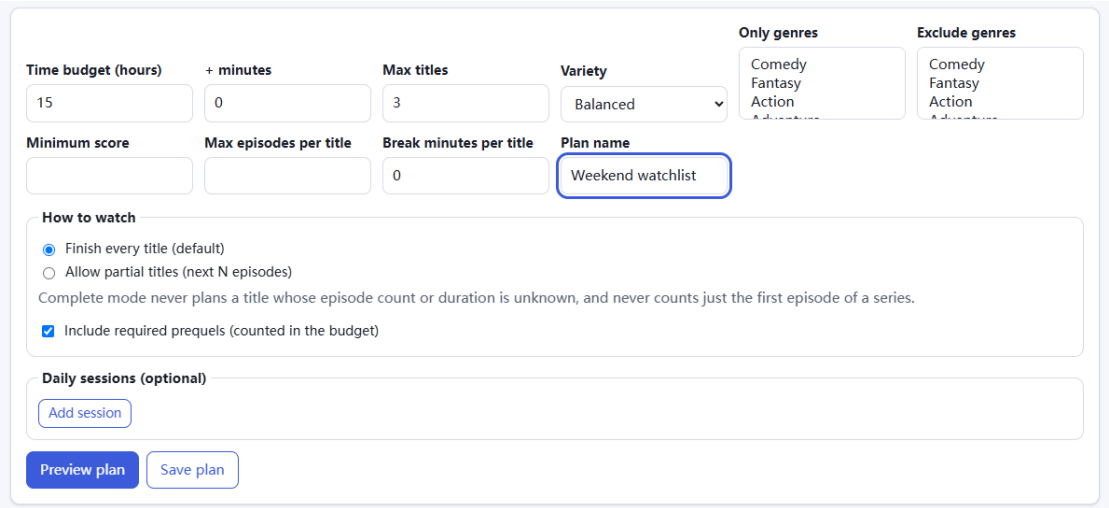


Figure C-8. The plan form with a 15-hour budget and a maximum of three titles.

Adjust the available time and limits

Use Only genres, Exclude genres, Min score and Max episodes per title to narrow the choices. On Windows, hold Ctrl when selecting more than one genre. Break minutes per title reserves break time within the budget.

For separate viewing sessions, select Add session under Daily sessions, enter a label and available minutes for each session, then preview again. Use Remove to delete a session.

Max titles is an upper limit. A plan can contain fewer titles. Durations are estimates based on the catalogue, so leave spare time if your schedule is tight. If no plan fits, increase the time, allow partial titles or relax one filter and preview again.

C.11 Save and manage a viewing plan

Check the preview and save it

1. Check the total minutes used, time left, title count and episode ranges. Read any warnings. The message Fits the budget and every constraint. confirms that the displayed plan passes its checks.
2. Select Save plan. The saved view shows editing and export controls. An unsaved preview will not appear in History.

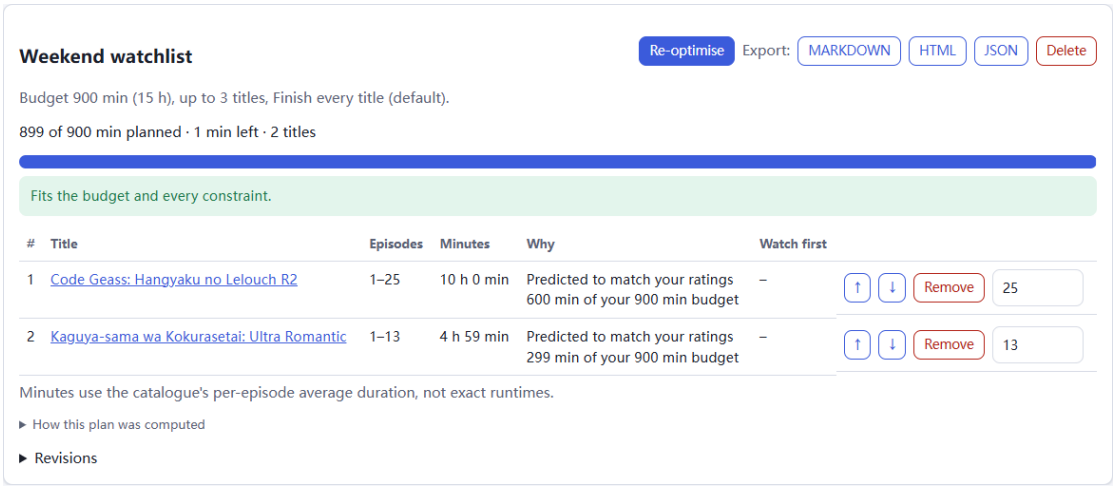


Figure C-9. A saved plan with its budget check, editing controls and export buttons.

Edit a saved plan

1. Use ↑ and ↓ beside a title to change its order, or select Remove to take it out.
2. To watch fewer episodes of a series, change the ending episode number on that row. Edits save automatically and the budget and constraint checks update.
3. Read the updated check after every change. If the plan no longer meets your needs, select Re-optimise to generate it again from the saved request. This can replace manual edits.

Reducing a complete series to fewer episodes changes the plan to allow partial viewing. Films are not divided into episode segments. Titles without usable episode or duration information may not be available for planning.

Export or return later

Select MARKDOWN for an editable text copy, HTML for a browser-readable copy, or JSON for structured data. Save the downloaded file using your browser. Exports are available after the plan has been saved.

Open History, then select the plan name to return to it. Expand Revisions to inspect earlier versions. To remove the saved plan, select Delete and confirm the deletion.

C.12 Return to saved work and resolve problems

Open your history

Select History at the top right while signed in. Under Chats, select a conversation to continue it. Under Viewing plans, select a saved plan to view, edit or export it. If an item is missing, check that you are signed in to the same account and using the same catalogue mode.

Resolve common problems

Table C-2. Common Problems and Solutions

What you see	What to do
The site will not open	Check that the server terminal is still running and use the port shown there. The guide uses http://127.0.0.1:8000.
The server says the port is in use	Stop another AniLogic server, or start with --port 8001 and open http://127.0.0.1:8001.
The catalogue is empty or fails to load	Installer: stop the server, run git lfs pull in the project folder, check the selected mode and restart. Do not replace a database containing activity you need to keep.
Search or recommendations return no results	Remove one filter or try another title name. In a collection, read an Apply suggestion before accepting it. An empty collection needs at least one saved title.
Home has few personal recommendations	Set your preferences and rate more familiar titles. Check whether the hide option or Not interested feedback is excluding choices.
A title image does not load	Check the internet connection and reload. Cover images use an external image service; the title text and controls can still be used.
Chat misunderstands a request	Check Understood conditions. State one correction at a time, or use New chat to clear previous conditions. English genre names and explicit numbers are useful for precise requests.
A viewing plan cannot be produced	Increase the budget, allow partial titles, or reduce a score, genre or length restriction. Preview again and read the displayed reason.
A change will not save or the session has expired	Copy any unsent message first. Reload the page and sign in again, then repeat the action and check its result.

Check feature availability

Open Status in the footer. ready means the feature is available; degraded means it is running with reduced capability; limited_by_data means required information is missing; not_configured means it has not been set up. If you need help, send the administrator the page name, the action you took and the exact message shown.

Scene details, sequel links and image identification depend on the supplied data. A missing result means the system has no supporting record; try Discover or a broader description.